

CSE460: VLSI Design

Lecture 1

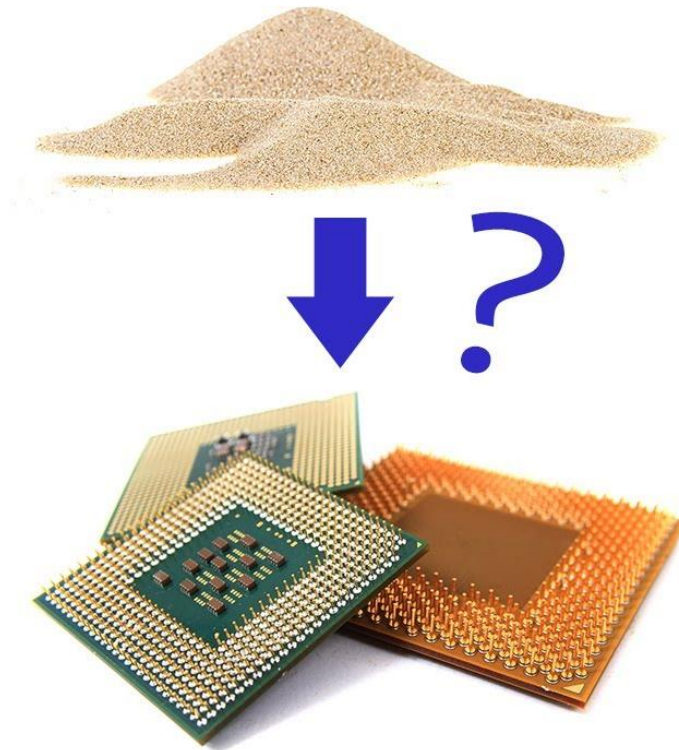
Very Large Scale Integration or VLSI

Contents

- Topics of this course
- History
- Transistors
- Moore's law
- Types of chips
- Design abstraction
- Design methodology

What will you learn in this course?

- You may have heard people say that electronics are made from sand, but do you know how?



What will you learn in this course?

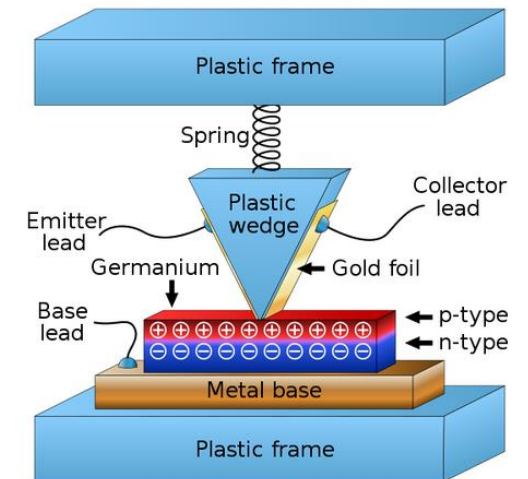
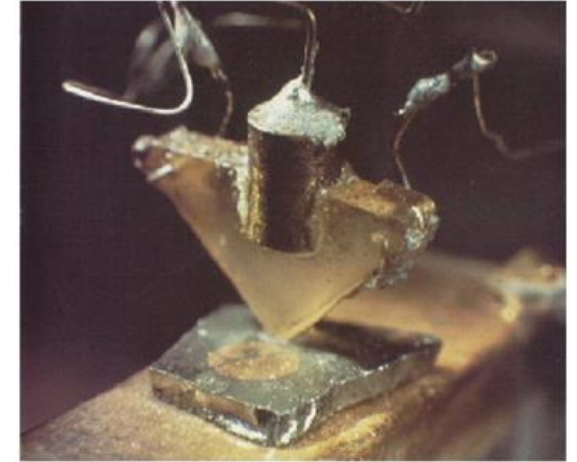
- A solid introduction to VLSI
- Different kinds of chips/ICs
- How to build one
- Theoretical background to making a good chip
- Quality measures of a chip
- The complete lifecycle of a chip

The Transistor

- Transistor concept
 - Transistors can be viewed as **electrically controlled switches** with a control terminal and two other terminals that are **connected** or **disconnected** depending on the *voltage* or *current* applied to the control terminal

Transistor Invention

- 1947: First transistor was built
 - John Bardeen and Walter Brattain built the first functioning point contact transistor at Bell Labs
 - Nearly classified as military secret
 - Later publicly introduced by Bell Labs
 - **Nobel Prize in Physics in 1956 for Bardeen, Brattain, and their supervisor William Shockley**
 - *We have called it the Transistor, T-R-A-N-S-I-S-T-O-R, because it is a resistor or semiconductor device which can amplify electrical signals as they are transferred through it from input to output terminals. It is, if you will, the electrical equivalent of a vacuum tube amplifier. But there the similarity ceases. It has no vacuum, no filament, no glass tube. It is composed entirely of cold, solid substances.*

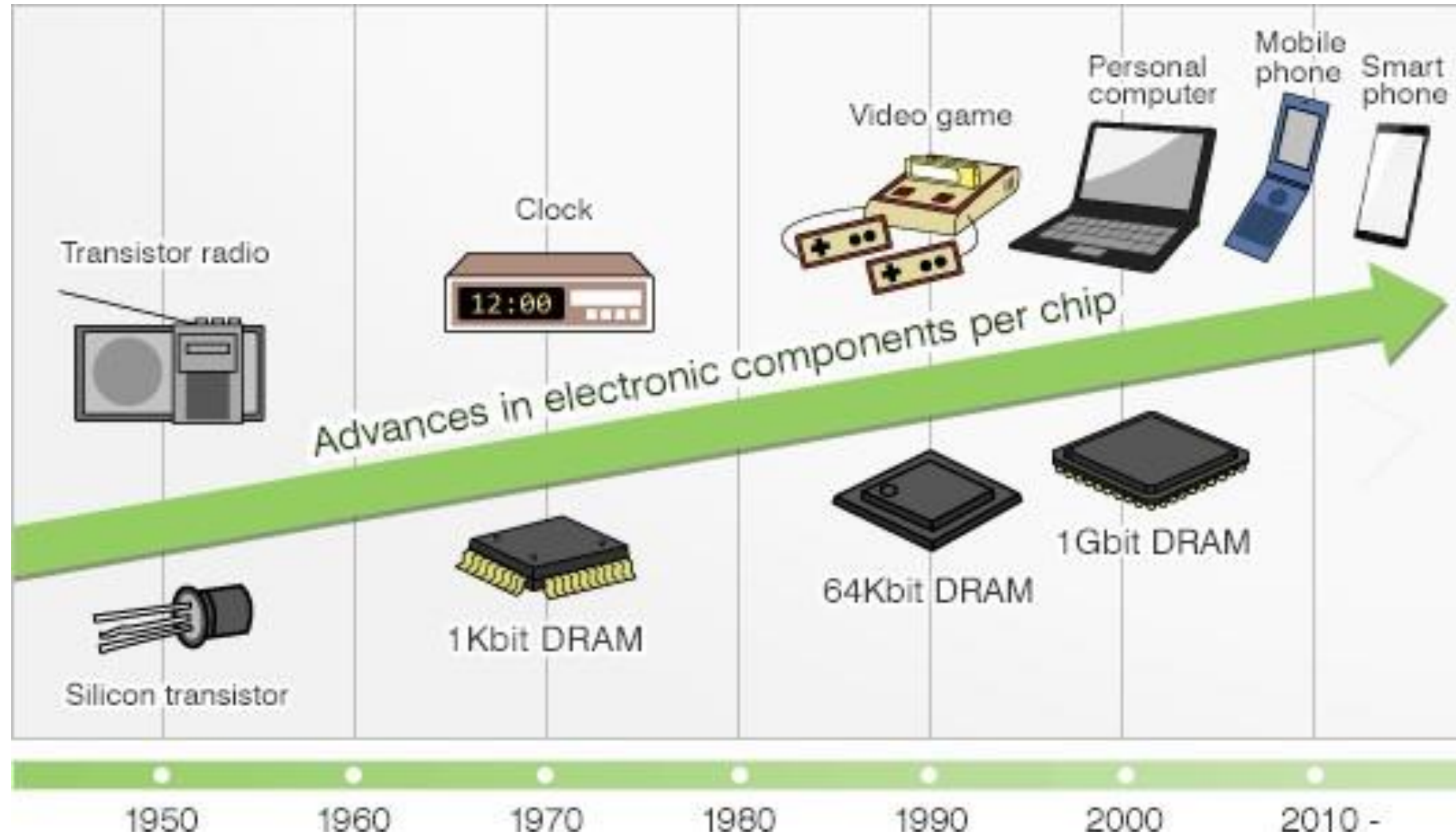


First IC

- 1959: U.S. Patent 3,138,743 for "Miniaturized Electronic Circuits"
 - Several years later after the invention of transistors, Jack Kilby at Texas Instruments realized the potential for miniaturization if multiple transistors could be built on one piece of silicon
 - **Kilby received the Nobel Prize in Physics in 2000 for the invention of the integrated circuit**



Evolution of Electronic Devices



Evolution of Electronic Devices



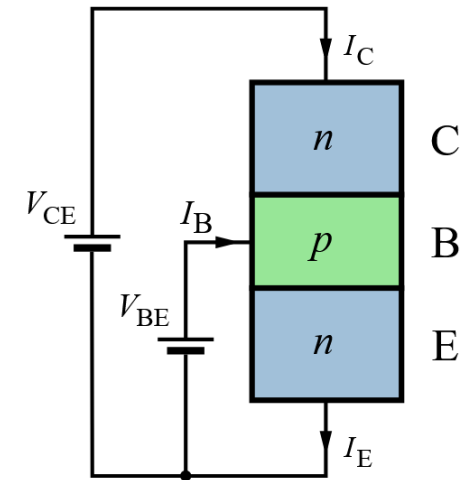
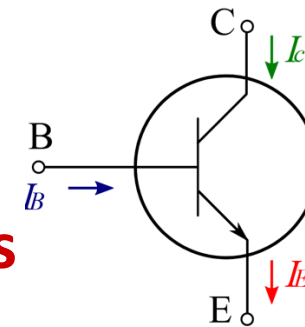
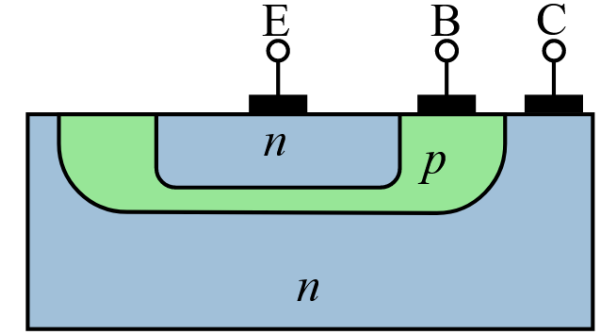
5MB (IBM)-1956, Cost - \$10,000/ 1 MB



4TB (WB)-2013

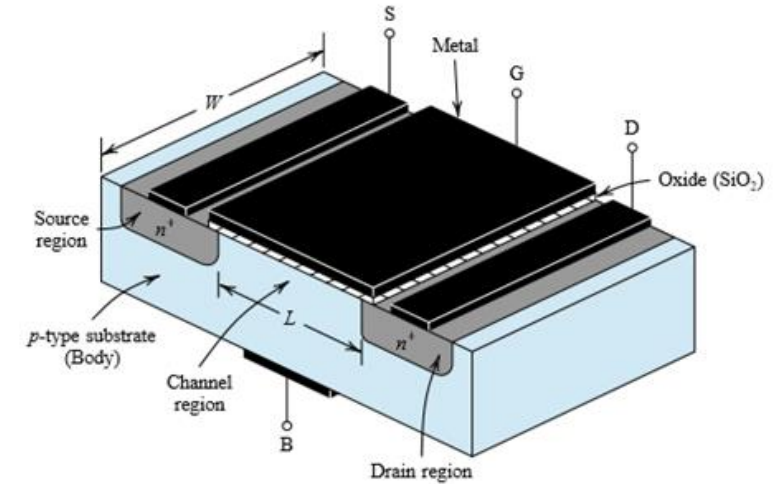
Bipolar Junction Transistor

- Bipolar Junction Transistor (BJT)
 - Developed by Bell Labs
- More reliable, less noisy, and more power-efficient than the point contact transistor
- BJTs require a small current into the control (*base*) terminal to switch much larger currents between the other two (*emitter* and *collector*) terminals
- **Power being dissipated by these base currents even when idle**



Metal Oxide Semiconductor Field Effect Transistor

- 1960s: (MOSFETs) began to enter production
- Metal Oxide Semiconductor Field Effect Transistors or MOSFETs offer the compelling advantage that they **draw almost zero control current while idle.**
- Two flavors: nMOS and pMOS
- Three terminals: *Gate* (control), *Drain* and *Source*



First MOSFET Gates

- 1963: First logic gate using MOSFETs
 - Frank Wanlass at Fairchild described the first logic gates using MOSFETs
- Fairchild's gates used both nMOS and pMOS transistors, earning the name Complementary Metal Oxide Semiconductor, or CMOS
- The circuits used discrete transistors but consumed six orders of magnitude less power than their bipolar counterparts
- With the development of the silicon planar process, MOS integrated circuits became attractive for their low cost because each transistor occupied less area and the fabrication process was simpler

Integration Levels

SSI: 1-12 gates

MSI: 13-99 gates

LSI: 100-9999 gates

VLSI: 10-99.9 k gates

ULSI: >100k gates

Integration Levels

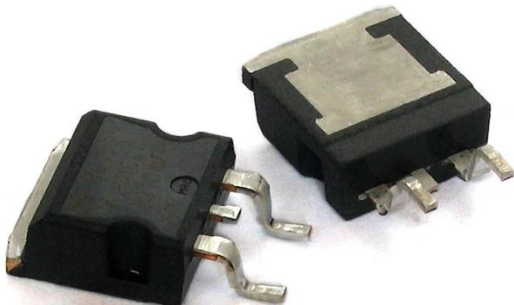
Name	No of Transistors	No of Logic Gates
Small-Scale Integration (SSI)	1 to 10	1 to 12
Medium-Scale Integration (MSI)	10 to 500	13 to 99
Large-Scale Integration (LSI)	500 to 20,000	100 to 9,999
Very Large Scale Integration (VLSI)	20,000 to 1,000,000	10,000 to 99,999
Ultra Large Scale Integration (ULSI)	1,000,000 and more	100,000 and more

Trend in MOSFET IC Design

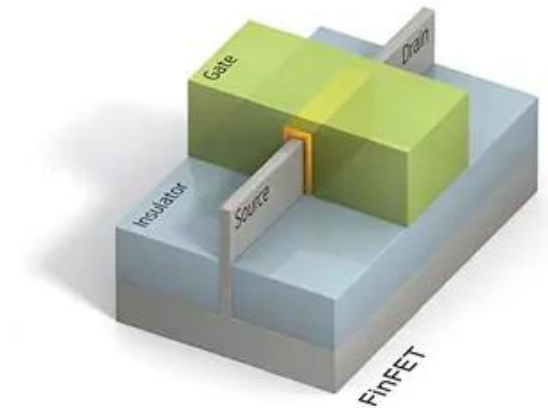
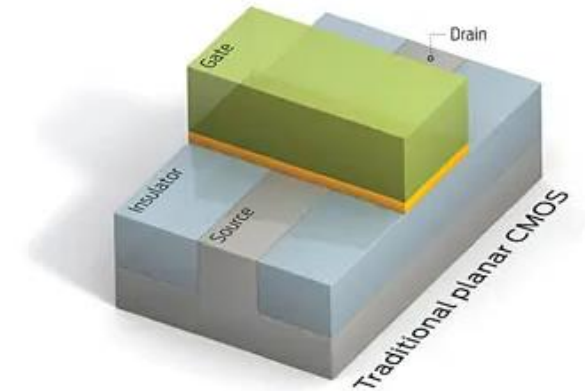
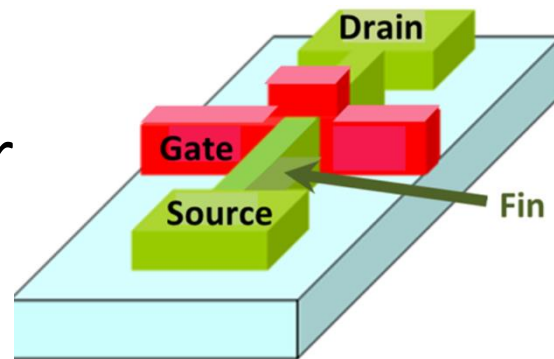
- Early commercial processes used only pMOS transistors and suffered from poor performance, yield, and reliability
- Processes using nMOS transistors became common in the 1970s
- While nMOS process was less expensive than CMOS, nMOS logic gates still consumed power while idle
- Power consumption became a major issue in the 1980s as hundreds of thousands of transistors were integrated onto a single die
- CMOS processes were widely adopted and replaced all the other technologies (pMOS, nMOS)

Types of MOSFET

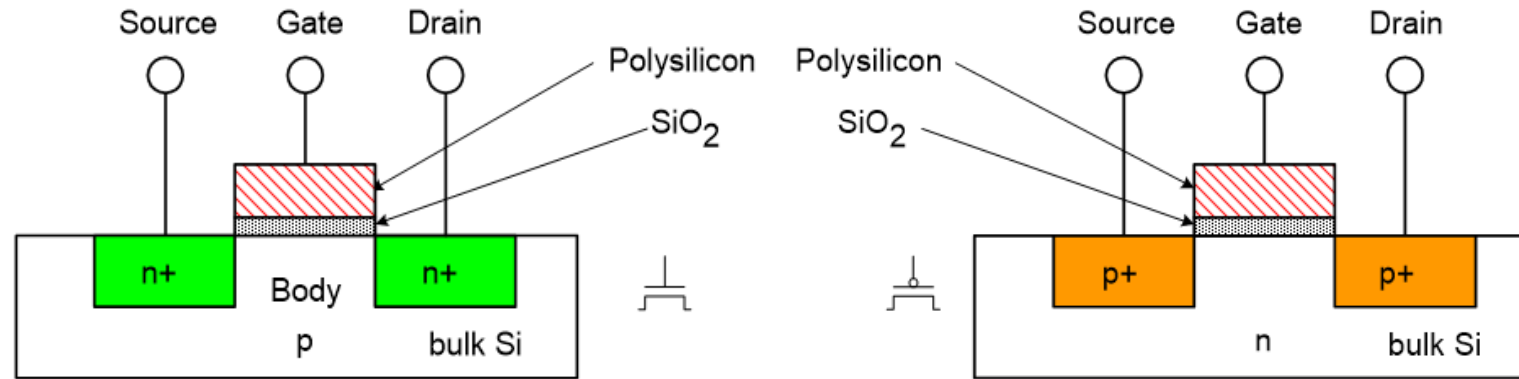
- pMOS, nMOS, CMOS
- Metal–insulator–semiconductor field-effect transistor (MISFET)
- Floating-gate MOSFET (FGMOS)
- Power MOSFET
- Multi-gate field-effect transistor (MuGFET)
- Quantum well MOSFET



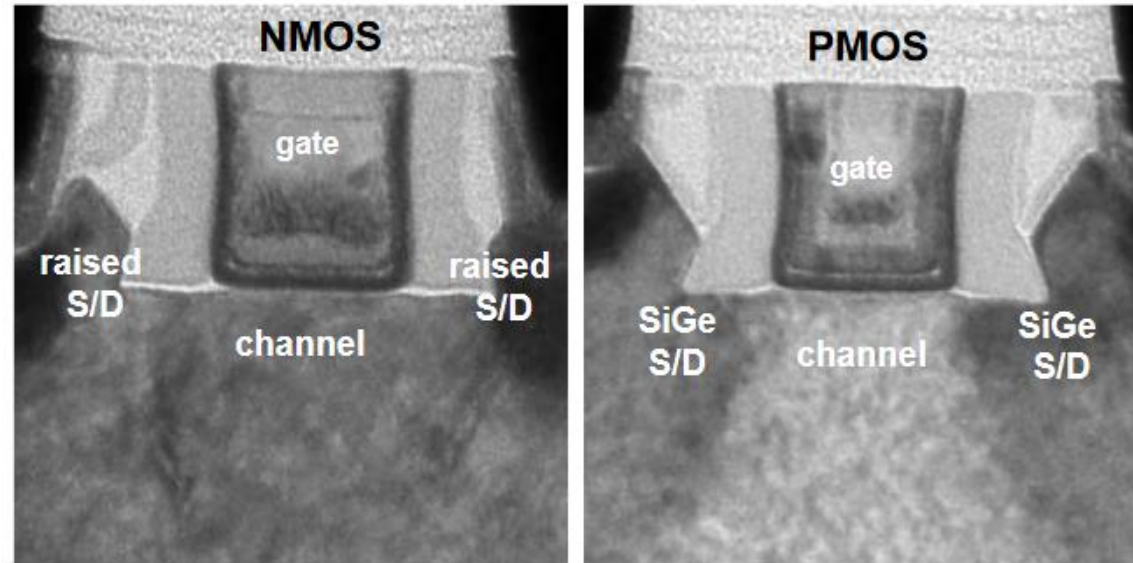
transistor



A Closer Look

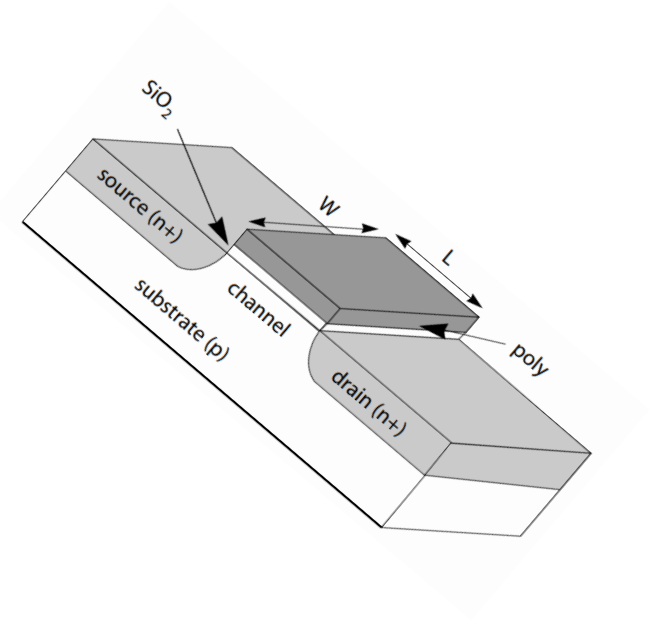
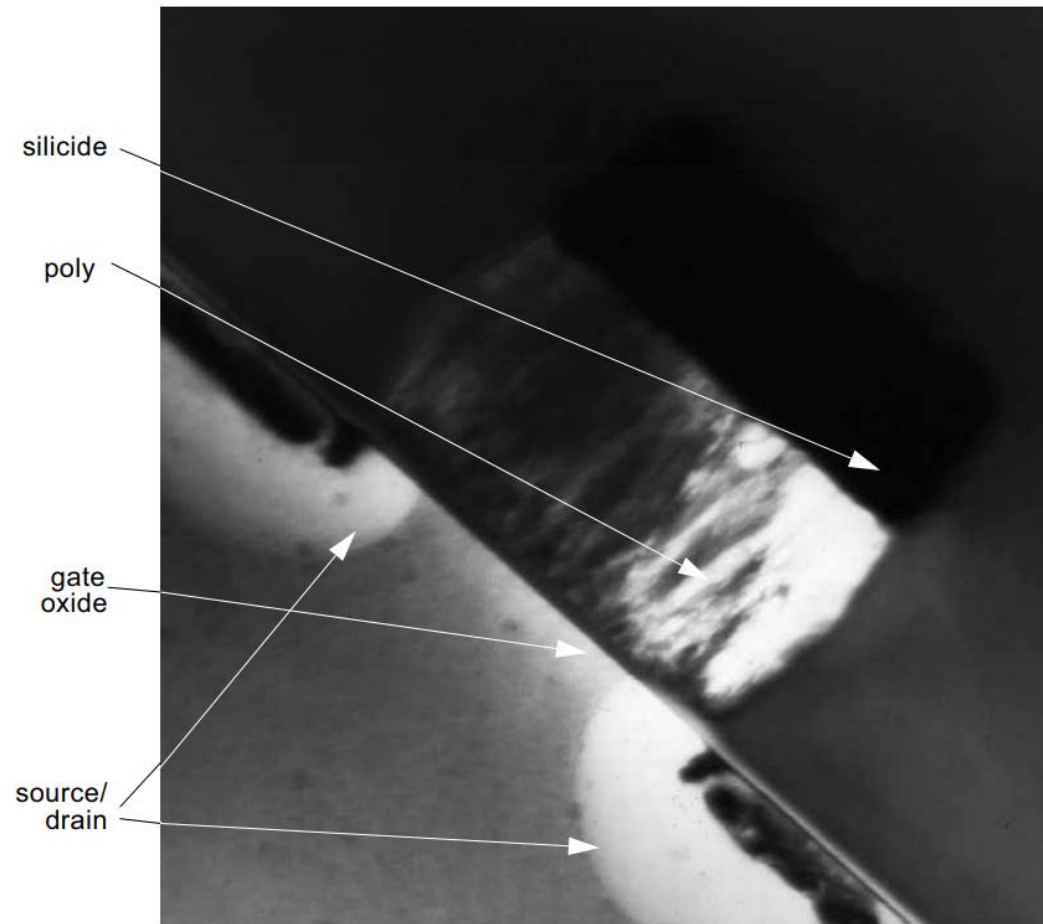


Packan P, Akbar S, Armstrong M, et al. High performance 32nm logic technology featuring 2nd generation High-k + metal gate transistors. In: International Electron Devices Meeting, IEEE, 2009. 659–662



Cross sectional view of intel's 32nm CMOS devices

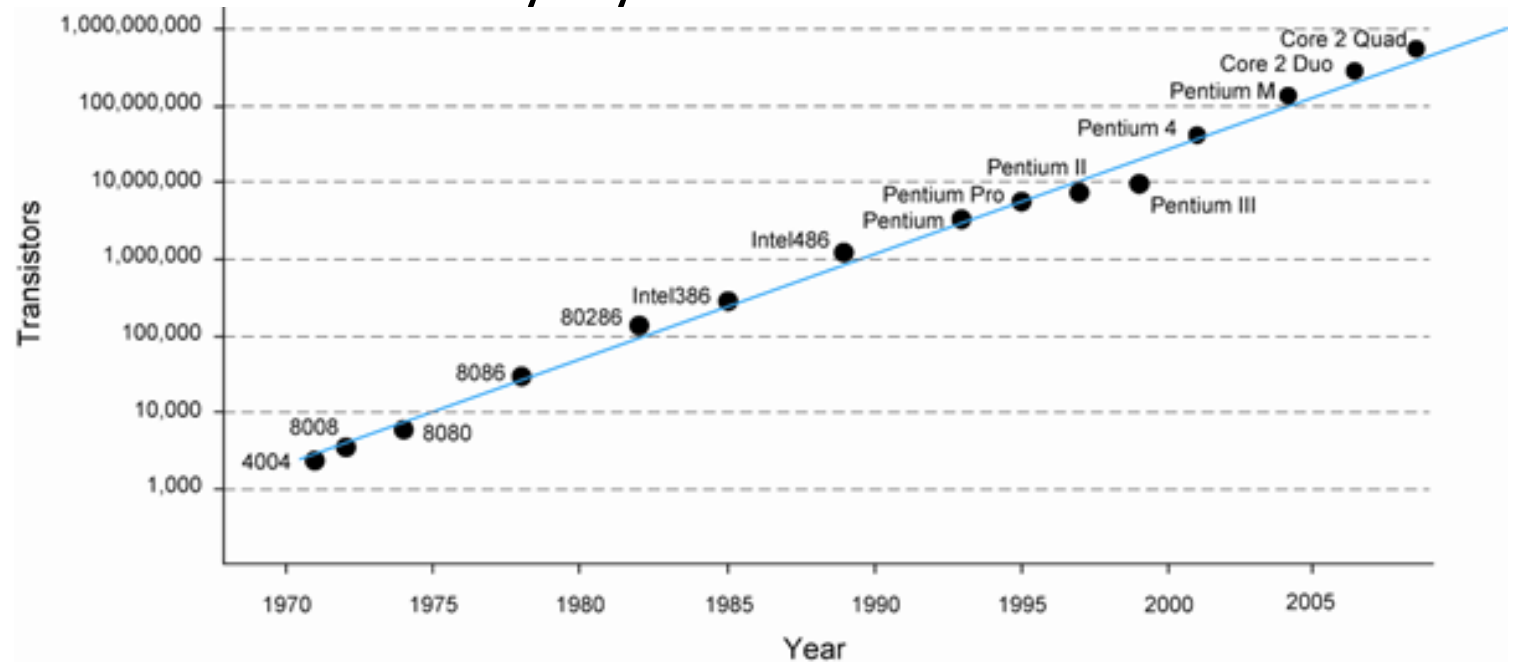
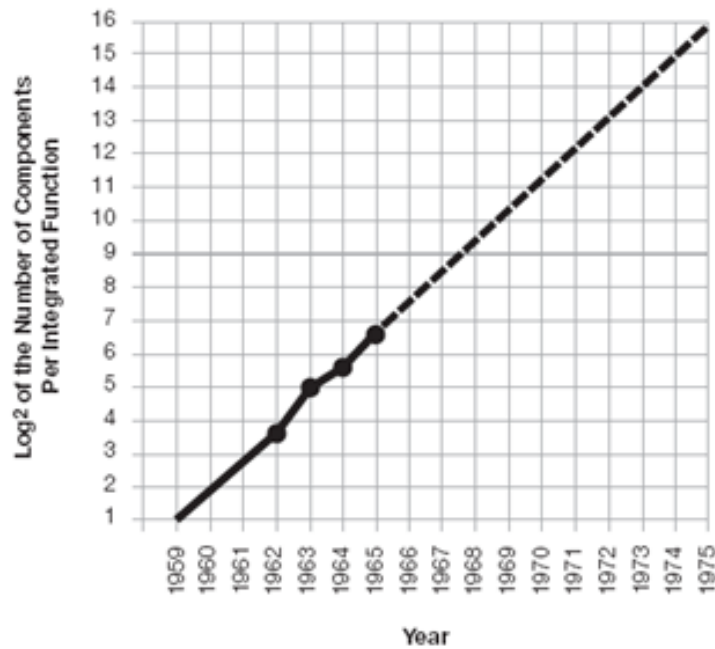
A Closer Look



Photomicrograph of a submicron MOS transistor (courtesy Agere)

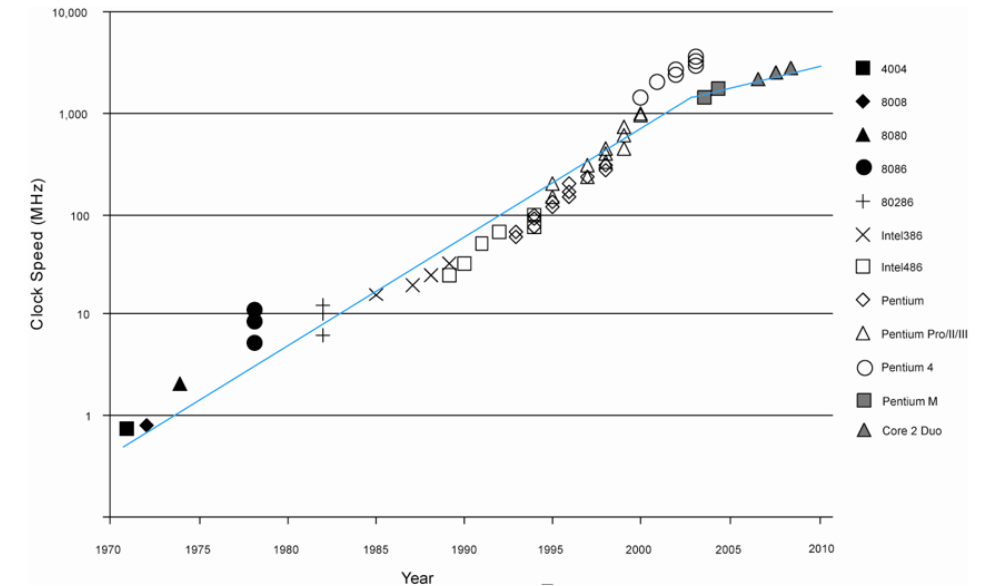
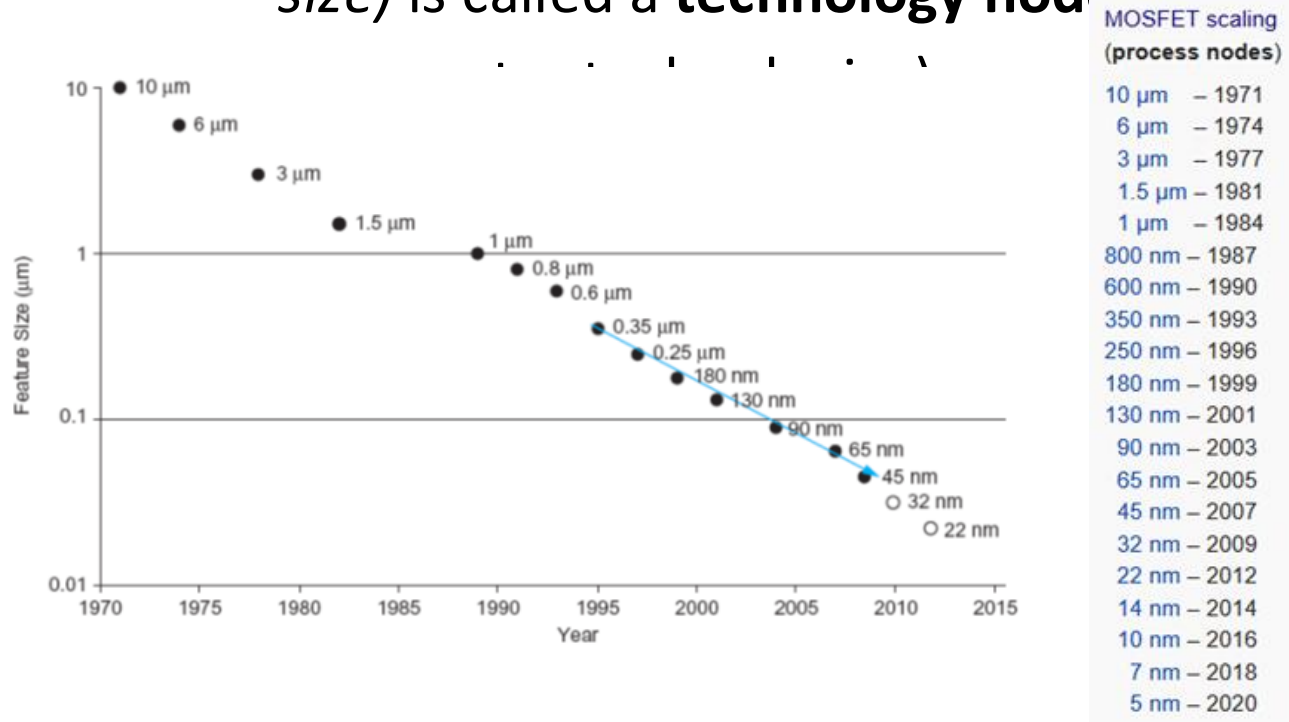
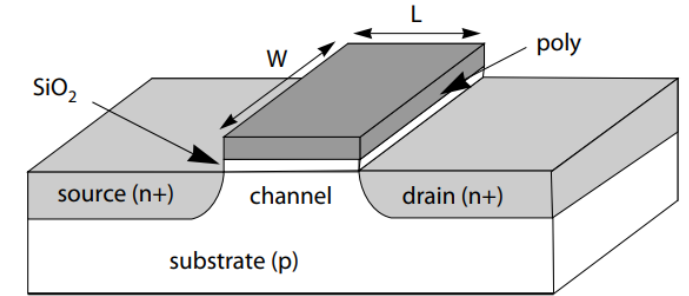
Moore's Law

- 1965: Gordon Moore plotted # transistors on each chip vs year
 - Fit straight line on semi-log scale
 - Transistor counts have doubled about every 2 years



Moore's Law

- Process generations
 - A manufacturing technology at a particular *channel length* (*feature size*) is called a **technology node** / **process node** (micron, submicron,



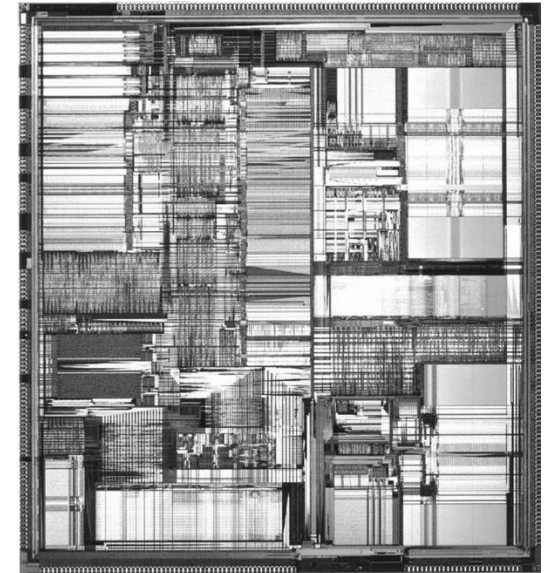
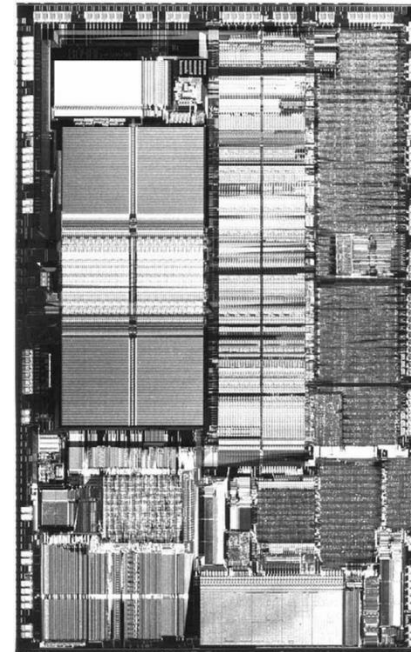
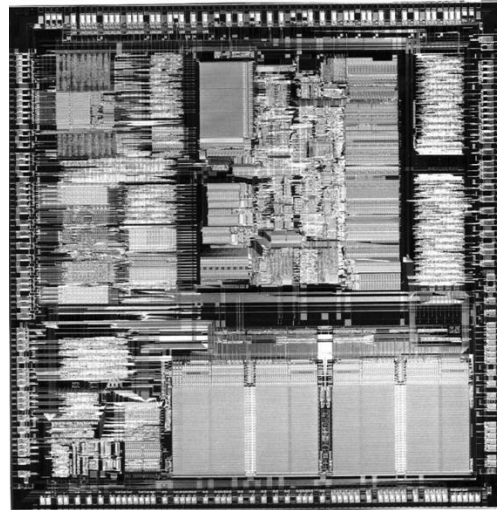
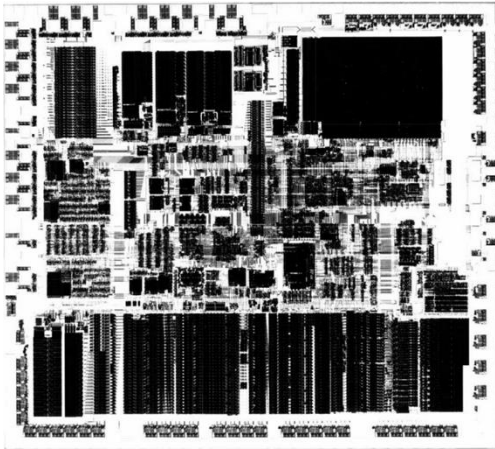
Moore's Law

- Moore's law and intel microprocessors

microprocessor	date of introduction	# transistors
80286	2/82	134,000
80386	10/85	275,000
80486	4/89	1,200,000
Intel Pentium™	3/93	3,100,000
Intel Pentium Pro™	11/95	5,500,000
Intel Pentium II™	1997	7,500,000
Intel Pentium III™	1999	9,500,000
Intel Pentium 4™	2000	42,000,000
Intel Itanium™	2001	25,000,000
Intel Itanium 2™	2003	220,000,000
Intel Itanium 2™ (9 MB cache)	2004	592,000,000

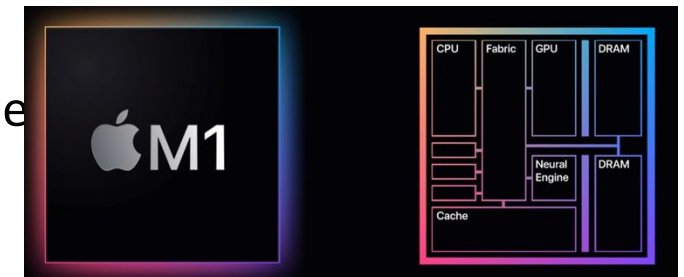
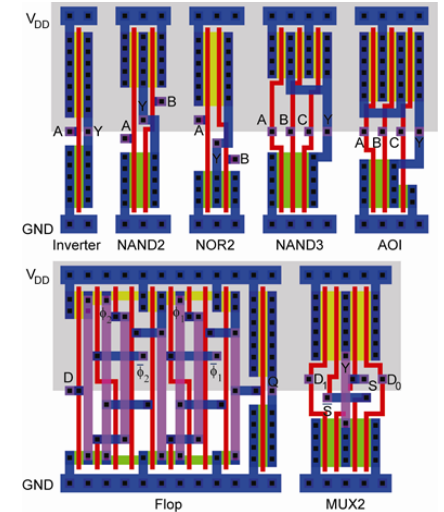
Intel Processors over the years

- 80286, 80386, 80486, Pentium



Types of chips

- Cell-based design
 - In the 1960s, standard parts were logic gates; in the 1970s they were LSI components. Today, standard parts include fairly specialized components: communication network interfaces, graphics accelerators
- Full custom
 - Designed manually piece by piece, fully custom layouts
- Application-specific integrated circuits (ASICs)
 - Rather than build a system out of standard parts, designers can now create a single chip for their particular application
- Systems-on-chips (SoCs)
 - a single-chip computer can include a CPU, bus, I/O devices, and me

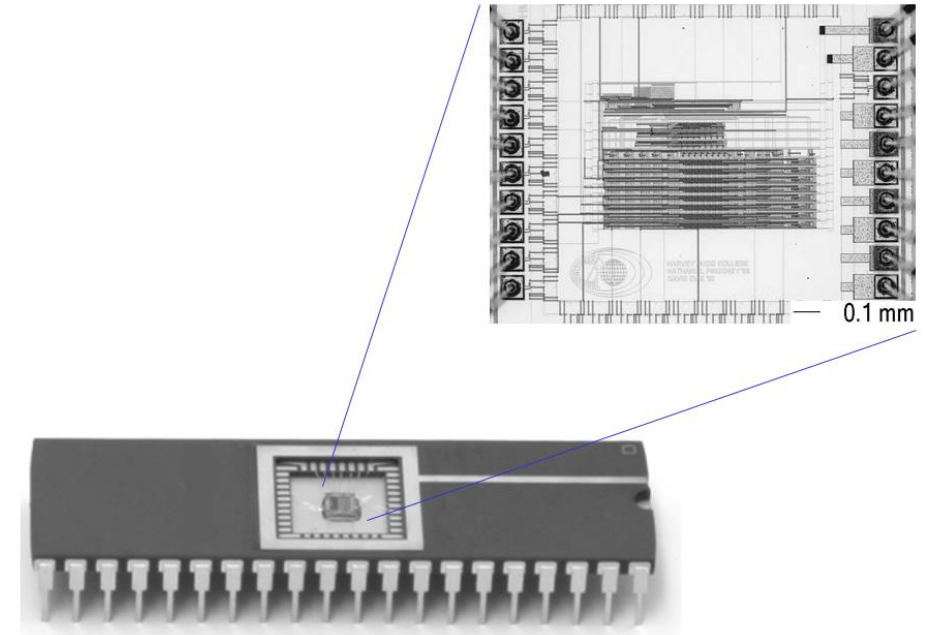
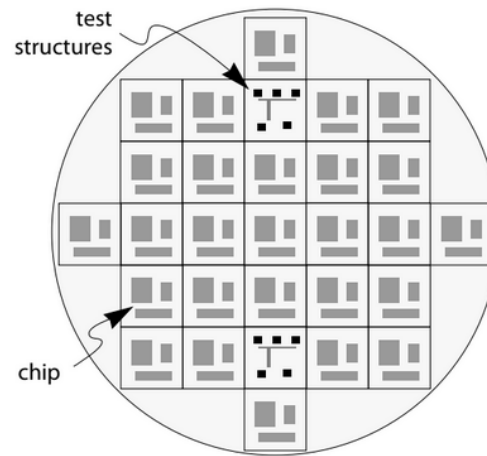
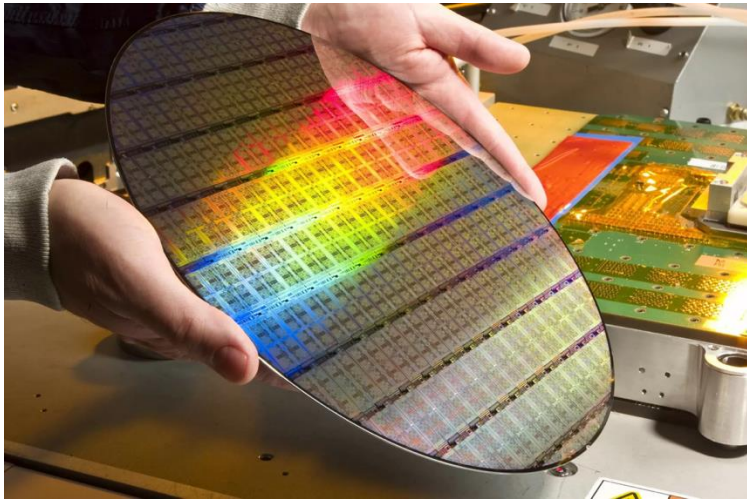


VLSI Chip Types

- ❑ VLSI chips are classified by the approach used to implement and build the circuit
- » **Full-custom Design**: where every circuit is custom designed for the project
 - Extremely tedious
 - Time-consuming process
- » **Application-Specific Integrated Circuits (ASICs)**: using an extensive suite of CAD tools that portray the system design in terms of standard digital logic constructs
 - Including state diagrams, functions tables, and logic diagram
 - Designer does not need any knowledge of the underlying electronics or the physic of the silicon chip
 - Major drawback is that all characteristics are set by the architectural design
- » **Semi-custom Design**: between that of a full-custom and ASICs
 - Using a group of primitive predefined cells as building blocks, called *cell library*

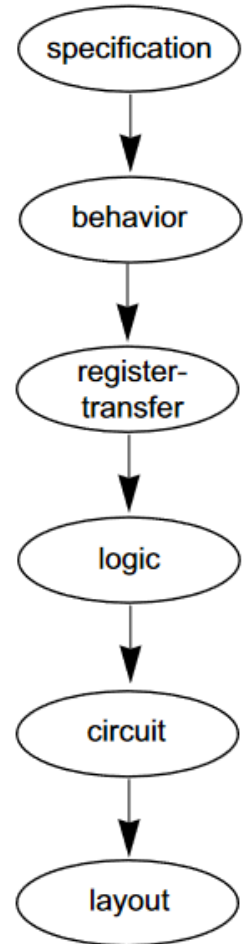
Chip Lifecycle

- A wafer > fabrication > chips > packaging



Abstraction levels

- Specification
 - A set of requirements, not a design
- Behavior
 - generally modeled as some sort of executable program
- Register-transfer
 - The system is specified as Boolean functions stored in abstract memory elements
- Logic
 - The system is designed in terms of Boolean logic gates, latches, and flip-flops
- Circuit
 - The system is implemented as transistors
- Layout
 - The final design for fabrication



Design Hierarchy (1/2)

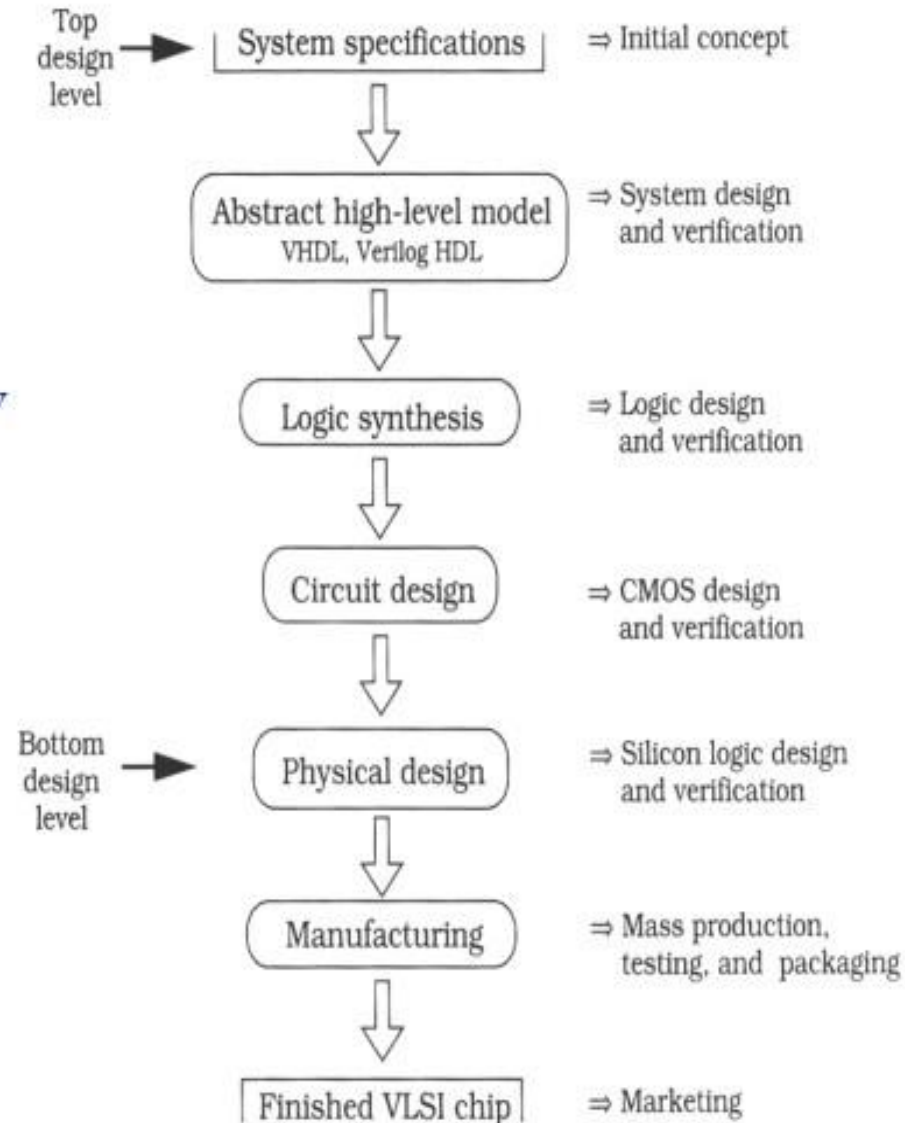


Figure 1.2 General overview of the design hierarchy

A Simple Design Flow

- An example of a design hierarchy
- – an instruction design of a
- *Register_X* $\square A \square B$

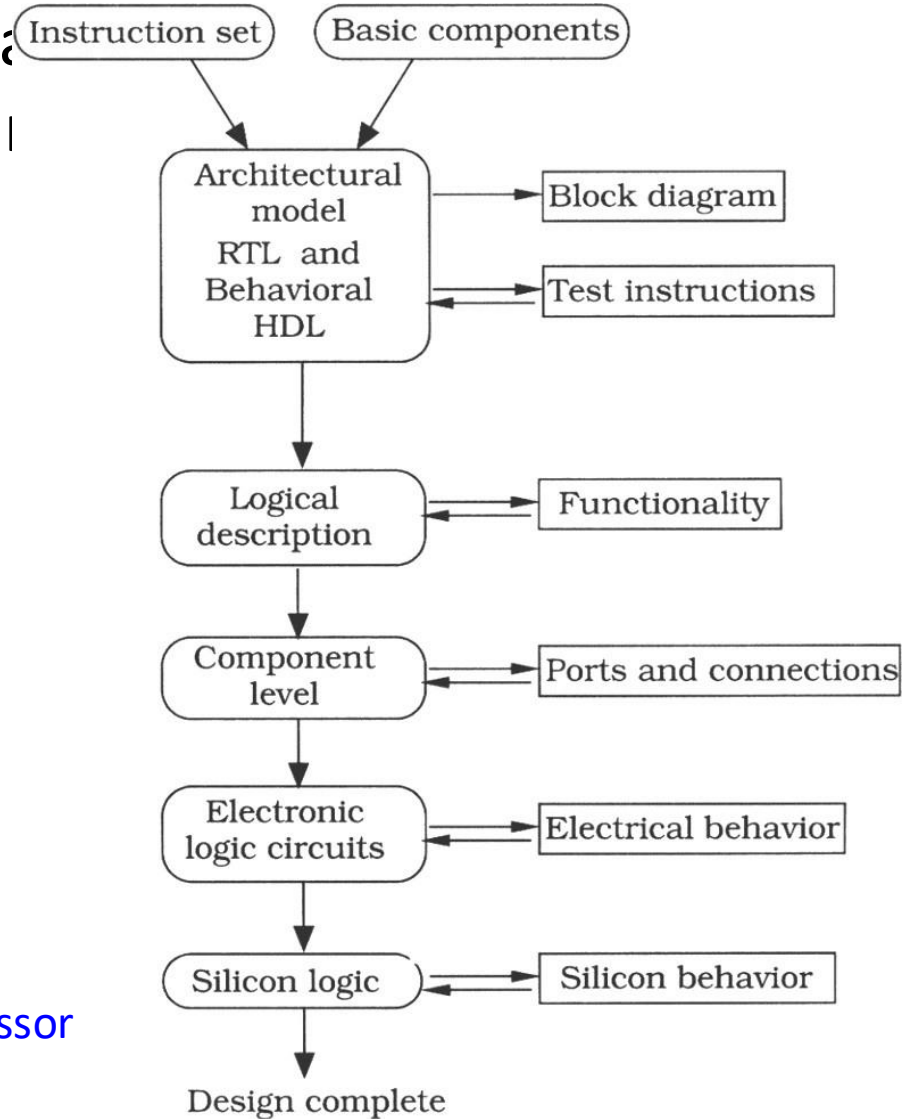


Figure 1.3 A simple design flow for a microprocessor
Reading from the text book.....

Design Hierarchy (2/2)

❑ Hierarchical design

» Top-down design

- the initial work is quite abstract and theoretical and there is no direct connection to silicon until many steps have been completed

» Bottom-up design

- starts at the silicon or circuit level and builds primitive units such as logic gates, adders, and registers as the first steps

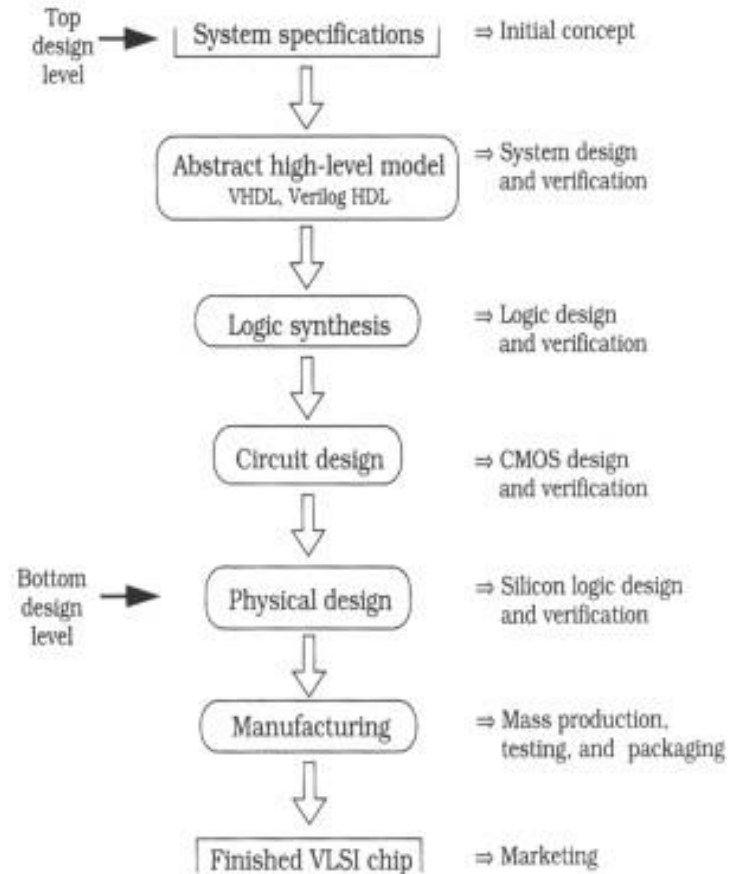
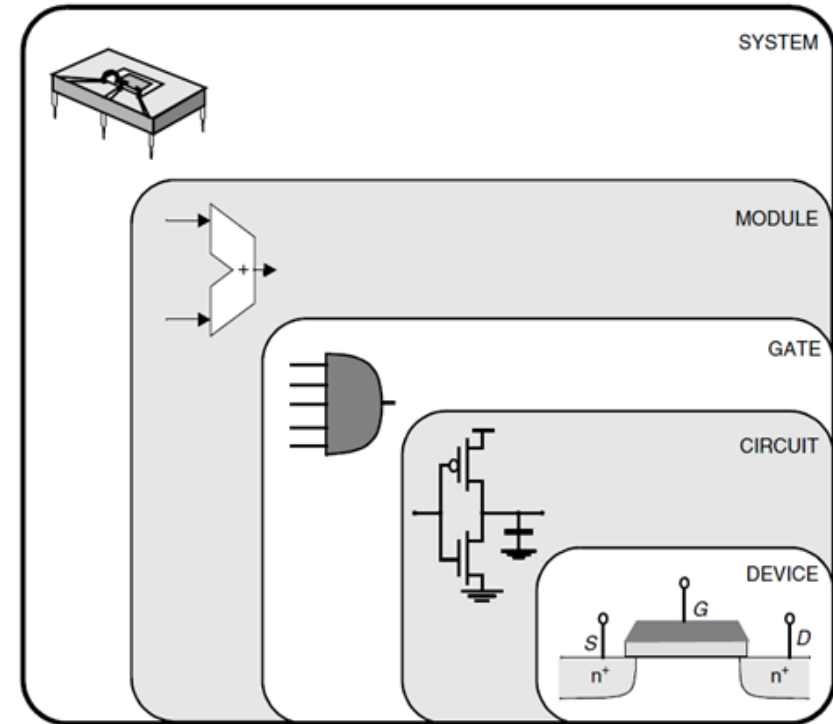


Figure 1.2 General overview of the design hierarchy

Design methodologies

- Top-down approach
 - Start from the specification and reach device specification
- Bottom-up approach
 - Start from the device optimization and go up to meet specification



Text

#	Title	Author(s)	Edition
1	CMOS VLSI Design	N.H.E. Weste, D. Harris & A. Banerjee	4th ed.
2	Fundamentals of Digital Logic with Verilog Design	Stephen Brown & Zvonko Vranesic	2nd/3rd ed.

Transistors = Switches

Slide References

1. <https://en.wikipedia.org/wiki/MOSFET>
2. <https://www.techspot.com/article/1840-how-cpus-are-designed-and-built-part-3/>
3. <https://ieeexplore.ieee.org/document/5424253>
4. Wayne Wolf - Modern VLSI Design IP based design
5. Jan M. Rabaey - Digital Integrated Circuits
6. Weste and Harris - CMOS VLSI Design

Thank you!

CSE460: VLSI Design

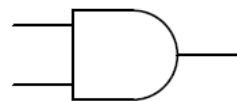
Lecture 2

Review of digital logic design

Background

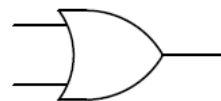
- Logic gates (AND, OR, NOT, XOR, etc.)
- Boolean algebra
- Truth tables
- Logic functions
- Logic function synthesis by
 - Sum of Products (SOP)
 - Product of Sums (POS)
 - K-maps
- Logic blocks (MUX, DEMUX)
- Sequential elements (Latch, Flip-flop)

Logic gates



AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



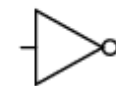
OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



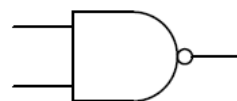
XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



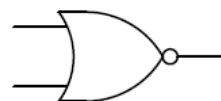
NOT

Input	Output
0	1
1	0



NAND

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0



NOR

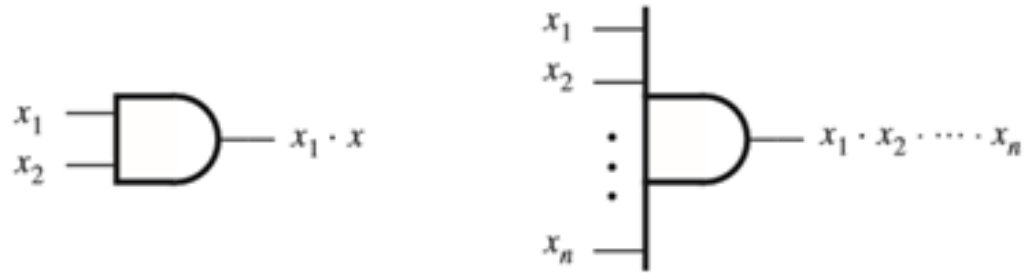
A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0



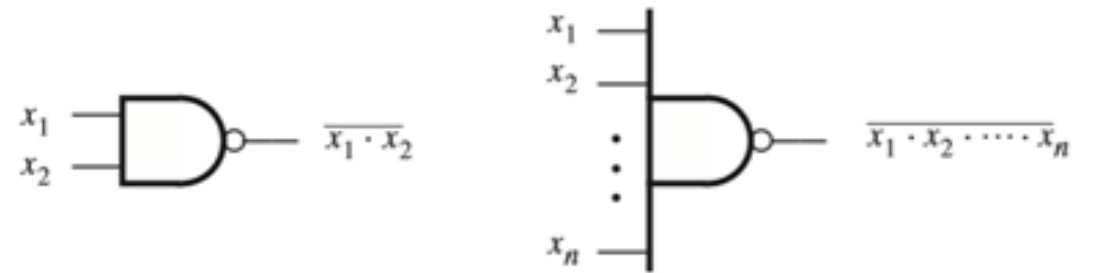
XNOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

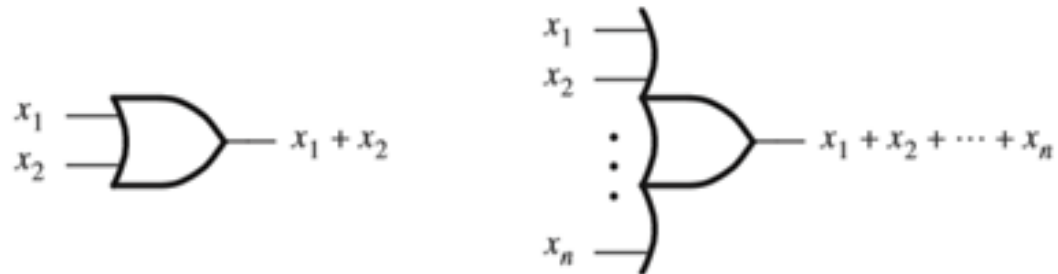
Generalized n-input logic gates



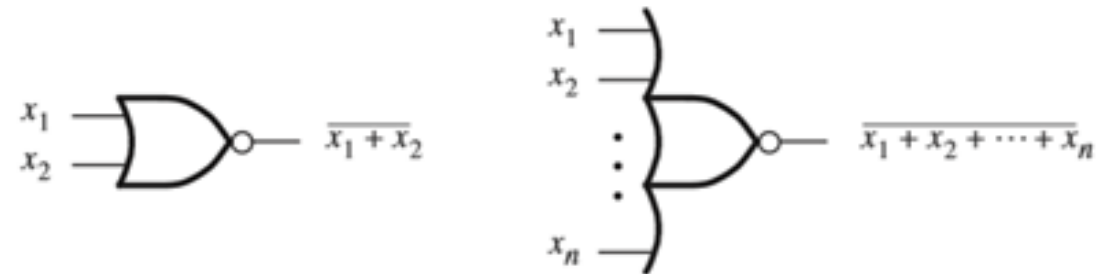
AND gates



NAND gates



OR gates



NOR gates

Axioms of Boolean Algebra

- 1a. $0 \cdot 0 = 0$
- 1b. $1 + 1 = 1$
- 2a. $1 \cdot 1 = 1$
- 2b. $0 + 0 = 0$
- 3a. $0 \cdot 1 = 1 \cdot 0 = 0$
- 3b. $1 + 0 = 0 + 1 = 1$
- 4a. If $x = 0$, then $x' = 1$
- 4b. If $x = 1$, then $x' = 0$

Boolean Algebra - Single Variable Theorems

- 5a. $x \cdot 0 = 0$
- 5b. $x + 1 = 1$
- 6a. $x \cdot 1 = x$
- 6b. $x + 0 = x$
- 7a. $x \cdot x = x$
- 7b. $x + x = x$
- 8a. $x \cdot x' = 0$
- 8b. $x + x' = 1$
- 9. $(x')' = x$

● Boolean Algebra - Two Variable Properties

- 10a. $x \cdot y = y \cdot x$ Commutative
- 10b. $x + y = y + x$
- 11a. $x \cdot (y \cdot z) = (x \cdot y) \cdot z$ Associative
- 11b. $x + (y + z) = (x + y) + z$
- 12a. $x \cdot (y + z) = x \cdot y + x \cdot z$ Distributive
- 12b. $x + y \cdot z = (x + y) \cdot (x + z)$
- 13a. $x + x \cdot y = x$ Absorption
- 13b. $x \cdot (x + y) = x$
- 14a. $x \cdot y + x \cdot y' = x$ Combining
- 14b. $(x + y) \cdot (x + y') = x$

Boolean Algebra - Two & Three Variable Properties

- 15a. $(x \cdot y)' = x' + y'$ DeMorgan's theorem
- 15b. $(x + y)' = x' \cdot y'$
- 16a. $x + x' \cdot y = x + y$
- 16b. $x \cdot (x' + y) = x \cdot y$
- 17a. $x \cdot y + y \cdot z + x' \cdot z = x \cdot y + x' \cdot z$ Consensus
- 17b. $(x + y) \cdot (y + z) \cdot (x' + z) = (x + y) \cdot (x' + z)$

Logic Function Synthesis - Three variable SOP & POS

- Function synthesis from truth table

Row number	x_1	x_2	x_3	Minterm	Maxterm
0	0	0	0	$m_0 = \bar{x}_1 \bar{x}_2 \bar{x}_3$	$M_0 = x_1 + x_2 + x_3$
1	0	0	1	$m_1 = \bar{x}_1 \bar{x}_2 x_3$	$M_1 = x_1 + x_2 + \bar{x}_3$
2	0	1	0	$m_2 = \bar{x}_1 x_2 \bar{x}_3$	$M_2 = x_1 + \bar{x}_2 + x_3$
3	0	1	1	$m_3 = \bar{x}_1 x_2 x_3$	$M_3 = x_1 + \bar{x}_2 + \bar{x}_3$
4	1	0	0	$m_4 = x_1 \bar{x}_2 \bar{x}_3$	$M_4 = \bar{x}_1 + x_2 + x_3$
5	1	0	1	$m_5 = x_1 \bar{x}_2 x_3$	$M_5 = \bar{x}_1 + x_2 + \bar{x}_3$
6	1	1	0	$m_6 = x_1 x_2 \bar{x}_3$	$M_6 = \bar{x}_1 + \bar{x}_2 + x_3$
7	1	1	1	$m_7 = x_1 x_2 x_3$	$M_7 = \bar{x}_1 + \bar{x}_2 + \bar{x}_3$

- A **Minterm** m_i is a complete argument vector $(a,b,c,...,x)$ for which a Boolean function $f(a,b,c,...,x)$ delivers the value '1'. It has “**minimum**” satisfiability.
- A **Maxterm** M_i is a complete argument vector $(a,b,c,...,x)$ for which a Boolean function $f(a,b,c,...,x)$ delivers the value '0'. It has “**maximum**” satisfiability.

SOP: Sum of product

- $ABC' + B'CD + ABCD + AD'$
- $A \rightarrow 1, A' \rightarrow 0$

POS: Product of sum

- $(A+B+C')(A+D')(B'+C+D)$
- $A \rightarrow 0, A' \rightarrow 1$

Logic Function Synthesis - Three variable SOP & POS

Example: 01: Write down the Boolean expression in **SOP** form that describes this truth table

A	B	C	Q
0	0	0	0
0	1	0	1
1	0	0	1
1	1	0	0
0	0	1	1
0	1	1	0
1	0	1	0
1	1	1	1

$$\bar{A}.\bar{B}.\bar{C} + A.\bar{B}.\bar{C} + \bar{A}.\bar{B}.C + A.B.C = Q$$

Example: 02: Write down the Boolean expression in **POS** form that describes this truth table

Sensor Inputs			
A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Logic Function Synthesis - Three variable SOP & POS

Example: 01: Write down the Boolean expression in **POS** form that describes this truth table

A	B	C	Output	Output	
0	0	0	0	0	$(A + B + C)$
0	0	1	0	1	
0	1	0	0	1	
0	1	1	1	1	
1	0	0	0	1	$(\bar{A} + \bar{B} + \bar{C})$
1	0	1	1	1	
1	1	0	1	1	
1	1	1	1	0	

$$\text{Output} = (A + B + C) (\bar{A} + \bar{B} + \bar{C})$$

Logic Function Synthesis - 2/3/4 variable k-map

- Function synthesis using k-maps

x_1	x_2	
0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

x_1	x_2	
0	0	m_0
0	1	m_2
1	0	m_1
1	1	m_3

$x_1 x_2$	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5

$x_1 x_2$	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

Diagram illustrating a 4-variable K-map (4x4 grid) for variables x_1, x_2, x_3, x_4 . The map is labeled with $x_1 x_2$ on the top and $x_3 x_4$ on the left. The cells are labeled m_0 through m_{15} . The map is partitioned into four 2x2 quadrants by dashed lines, with labels x_1 (top), x_2 (bottom), x_3 (left), and x_4 (right) indicating the corresponding variable groups.

Logic Function Synthesis - 2/3/4 variable k-map

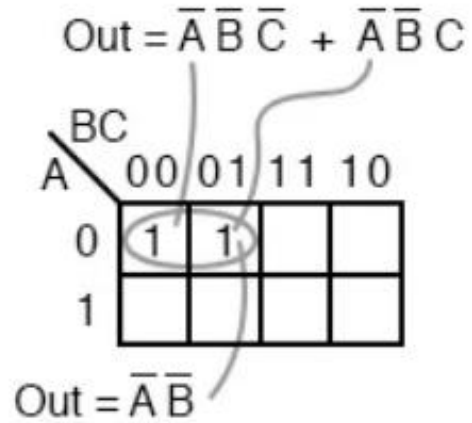
Function synthesis using k-maps

1. No zeros allowed.
2. No diagonals.
3. Only power of 2 number of cells in each group. ($2^0=1$, $2^1=2$, $2^2=4$, $2^3=8$, etc.)
4. Groups should be as large as possible.
5. Every 1 must be in at least one group.
6. Overlapping allowed.
7. Wrap around allowed.
8. Fewest number of groups possible.

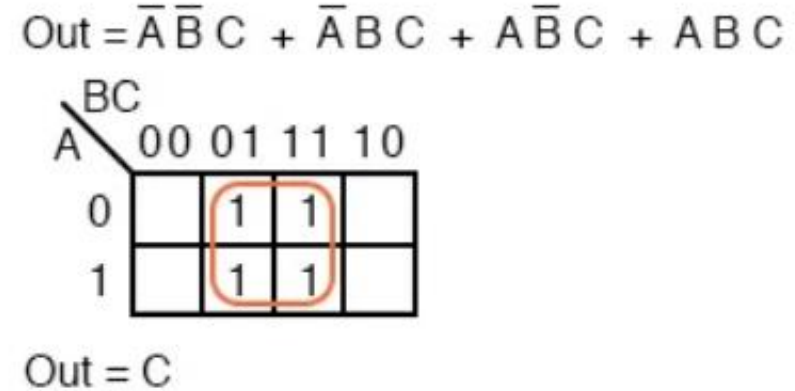
Visit: <http://www.ee.surrey.ac.uk/Projects/Labview/minimisation/karrules.html>

3 variable K-Map

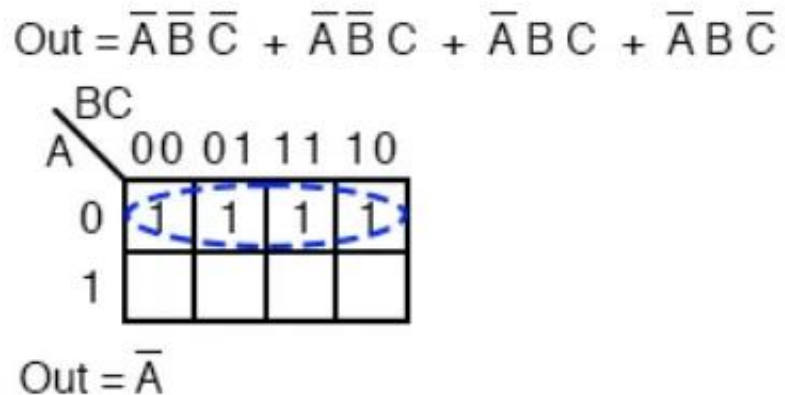
Example: 01



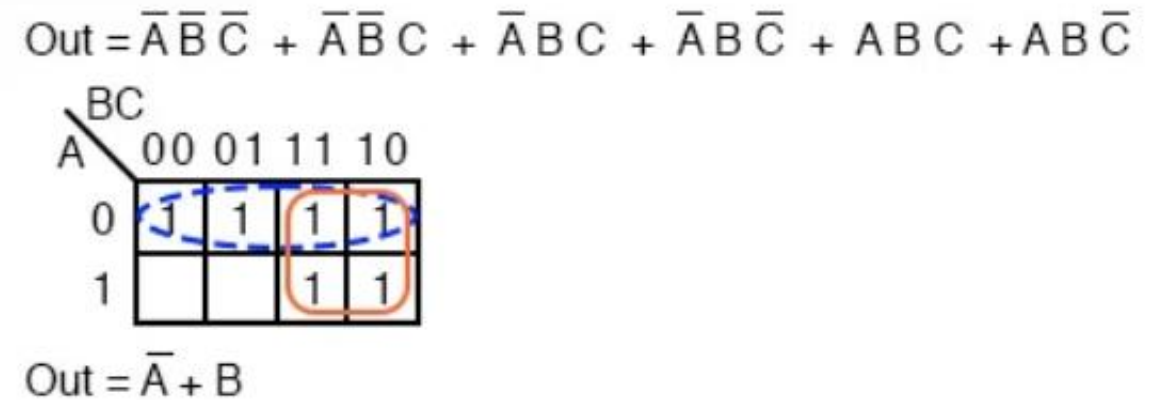
Example: 03



Example: 02



Example: 04



3 variable K-Map

Example: 05

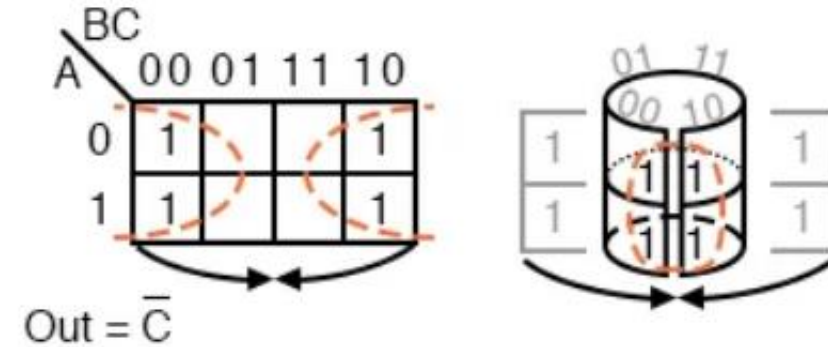
$$\text{Out} = \bar{A} B C + A B C$$

A \ BC	00	01	11	10
	0	1	1	0
0			1	
1			1	

$$\text{Out} = B C$$

Example: 06

$$\text{Out} = \bar{A} \bar{B} \bar{C} + A \bar{B} \bar{C} + \bar{A} B \bar{C} + A B \bar{C}$$



Example: 07

$$\text{Out} = \bar{A} \bar{B} \bar{C} + \bar{A} \bar{B} C + \bar{A} B C + \bar{A} B \bar{C} + A \bar{B} \bar{C} + A B \bar{C}$$

A \ BC	00	01	11	10
	0	1	1	0
0	1	1	1	1
1	1			1

$$\text{Out} = \bar{A} + \bar{C}$$

Example: 08

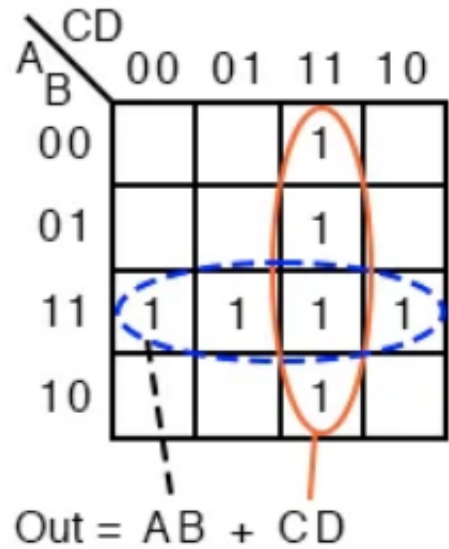
A \ BC	00	01	11	10
	0	1	1	0
0			1	
1		1	1	1

$$\text{Output} = A B + B C + A C$$

4 variable K-Map

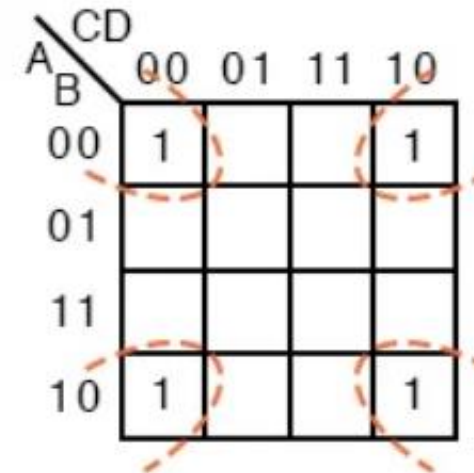
Example: 01

$$\text{Out} = \bar{A}\bar{B}CD + \bar{A}BCD + ABCD + A\bar{B}CD + AB\bar{C}\bar{D} + AB\bar{C}D + ABC\bar{D}$$

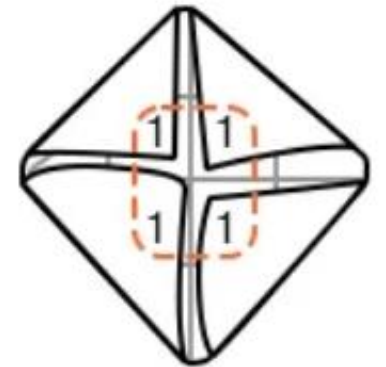


Example: 02

$$\text{Out} = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D}$$



$$\text{Out} = \bar{B}\bar{D}$$



4 variable K-Map

Example: 03

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD \\ + A\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}D + A\overline{B}C\overline{D} + A\overline{B}CD$$

A \ B	CD			
	00	01	11	10
00	1	1	1	1
01				
11				
10	1	1	1	1

$$\text{Out} = \overline{B}$$

Example: 04

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD \\ + B\overline{C}\overline{D} + B\overline{C}D + A\overline{B}\overline{C}\overline{D} + A\overline{B}D + A\overline{B}C\overline{D}$$

A \ B	CD			
	00	01	11	10
00	1	1	1	1
01	1			1
11	1			1
10	1	1	1	1

A \ B	CD			
	00	01	11	10
00	1	1	1	1
01	1			1
11	1			1
10	1	1	1	1

$\overline{B}$

$\overline{D}$

$$\text{Out} = \overline{B} + \overline{D}$$

4 variable K-Map

Example: 05

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + A\overline{B}\overline{C}\overline{D} + ABCD$$

A \ B		CD			
		00	01	11	10
00	1				1
01					
11				1	
10	1				

$$\text{Out} = \overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{D} + ABCD$$

Example: 06

$$\begin{aligned} \text{Out} = & \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} \\ & + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + \overline{A}BC\overline{D} \\ & + AB\overline{C}\overline{D} + AB\overline{C}D + ABCD \end{aligned}$$

A \ B		CD			
		00	01	11	10
00	1	1	1		
01	1	1	1		
11	1	1	1		
10					

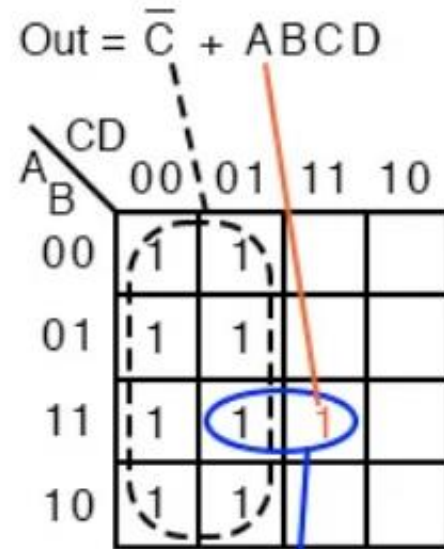
A \ B		CD			
		00	01	11	10
00	1	1	1		
01	1	1	1		
11	1	1	1		
10					

A \ B		CD			
		00	01	11	10
00	1	1	1		
01	1	1	1		
11	1	1	1		
10					

$$\text{Out} = \overline{A}\overline{C} + \overline{A}D + B\overline{C} + BD$$

4 variable K-Map

Example: 07



Out = $\bar{C}$ + ABD

Simplification by Boolean Algebra

Out = $\bar{C}$ + ABCD

Applying rule $A + \bar{A}B = A + B$ to the $\bar{C} + ABCD$ term

Out = $\bar{C}$ + ABD

Don't Care Condition

Example: 01

L1

A \ BC	00	01	11	10
	0	1	1	1
1	1	1	*	*

$$L1 = A + B + C$$

L2

A \ BC	00	01	11	10
	0	0	1	1
1	1	1	*	*

$$L2 = A + B$$

L3

A \ BC	00	01	11	10
	0	0	1	1
1	1	1	*	*

$$L3 = A + BC$$

L4

A \ BC	00	01	11	10
	0	0	0	0
1	1	1	*	*

$$L4 = A$$

L5

A \ BC	00	01	11	10
	0	0	0	0
1	1	1	*	*

$$L5 = AC$$

*** Don't care cells may be used as either 1s or 0s, whichever is useful

There are four sensors, A, B, C, and D that control the lights in a room. Each sensor transitions to a **HIGH** state when it detects an ambient light level exceeding a predefined threshold. The room's lights will turn **ON** when the majority of sensors are in the **LOW** state. If the number of sensors in the **HIGH** and **LOW** states is equal, the room's lighting condition remains **indeterminate** (i.e., a don't care condition). Answer questions (a-c) below based on this description.

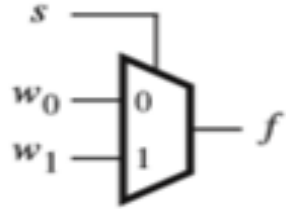
- (a) [2 marks] **Write** down the truth table for the system, describing all possible states of the sensors (A, B, C, D) and their corresponding output states.
- (b) [5 marks] **Draw** the Karnaugh map (K-map) for the system based on the truth table in part (a). **Derive** the logical expression that governs the system from the K-map.
- (c) [5 marks] **Draw** the CMOS logic circuit for the logical expression derived in part (b). **Specify** the number of MOSFETs required to implement your design.

A	B	C	D	Y	
0	0	0	0	0	→ OFF
0	0	0	1	0	→ OFF
0	0	1	0	0	→ OFF
0	0	1	1	d	→ don't care
0	1	0	0	0	→ OFF
0	1	0	1	d	→ don't care
0	1	1	0	d	→ don't care
0	1	1	1	1	→ ON
1	0	0	0	0	→ OFF
1	0	0	1	d	→ don't care
1	0	1	0	d	→ don't care
1	0	1	1	1	→ ON
1	1	0	0	d	→ don't care
1	1	0	1	1	→ ON
1	1	1	0	1	→ ON
1	1	1	1	1	→ ON

	00	01	11	10
00	0	0	d	0
01	0	d	1	d
11	d	1	1	1
10	0	d	1	d

Multiplexer

- Multiple inputs, single output. Output is chosen by selector pin/s

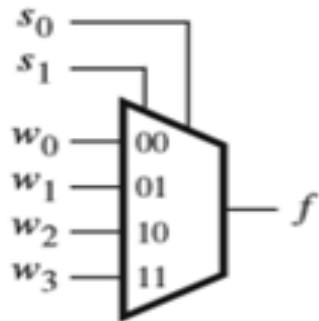


Graphical symbol

s	f
0	w_0
1	w_1

Truth table

2x1 Multiplexer



Graphical symbol

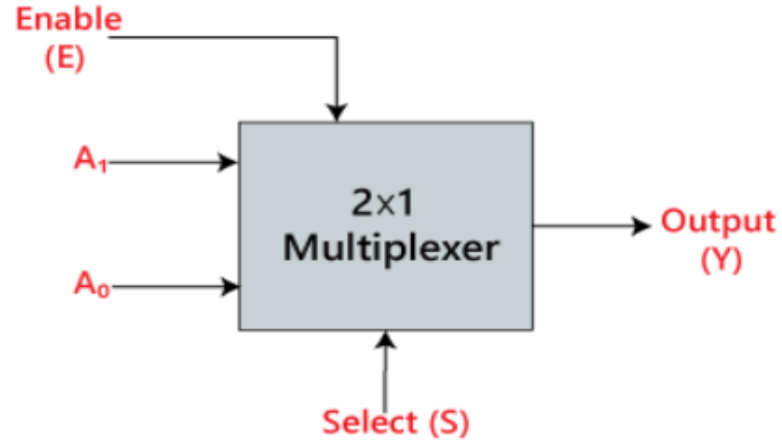
s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

Truth table

4x1 Multiplexer

Multiplexer

Block Diagram:



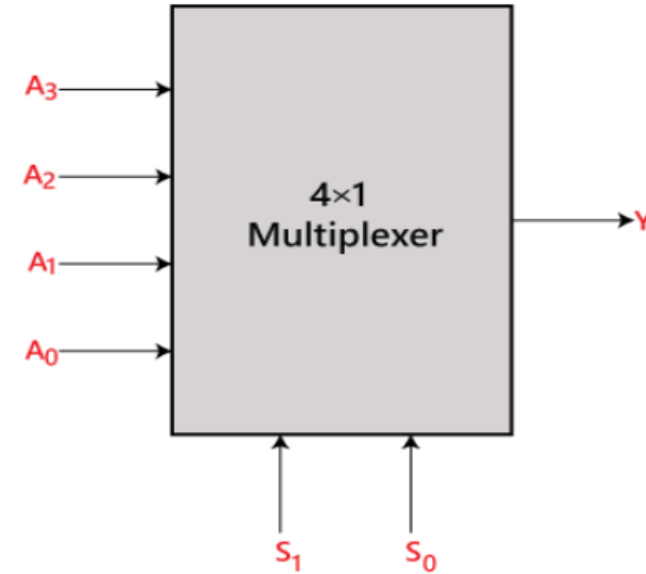
Truth Table:

INPUTS	Output
S_0	Y
0	A_0
1	A_1

The logical expression of the term Y is as follows:

$$Y = S_0' \cdot A_0 + S_0 \cdot A_1$$

Block Diagram:



Truth Table:

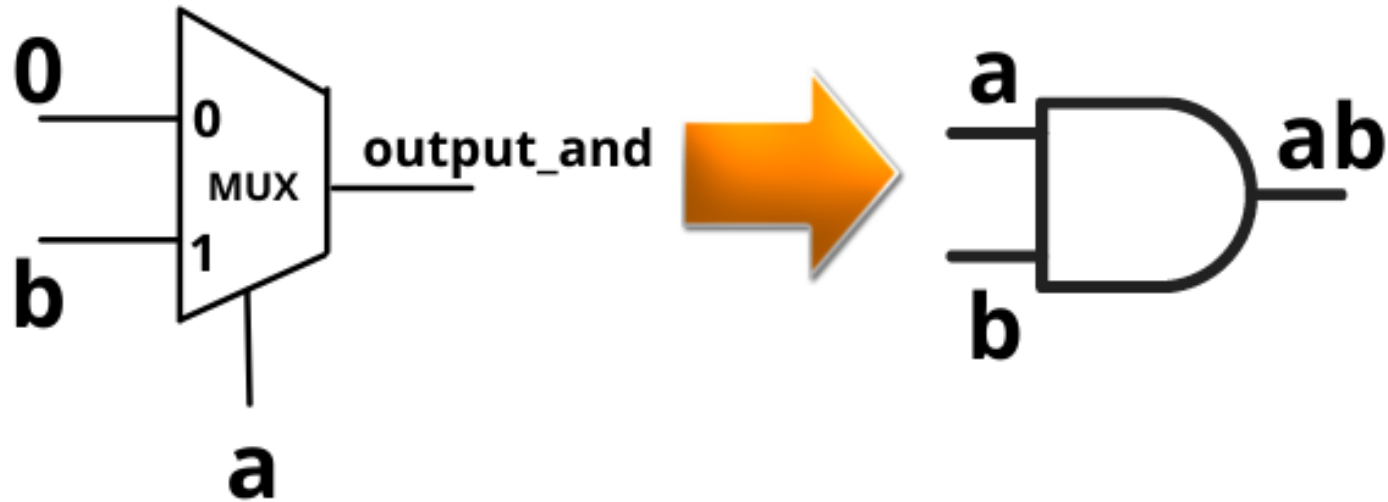
INPUTS		Output
S_1	S_0	Y
0	0	A_0
0	1	A_1
1	0	A_2
1	1	A_3

The logical expression of the term Y is as follows:

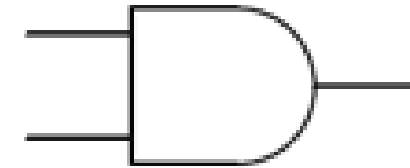
$$Y = S_1' S_0' A_0 + S_1' S_0 A_1 + S_1 S_0' A_2 + S_1 S_0 A_3$$

Logic Gates using Multiplexer

And Gate Using 2:1 MUX



$B \rightarrow I_1$
 $A \rightarrow S$
 $I_0 \rightarrow 0\text{ V}$

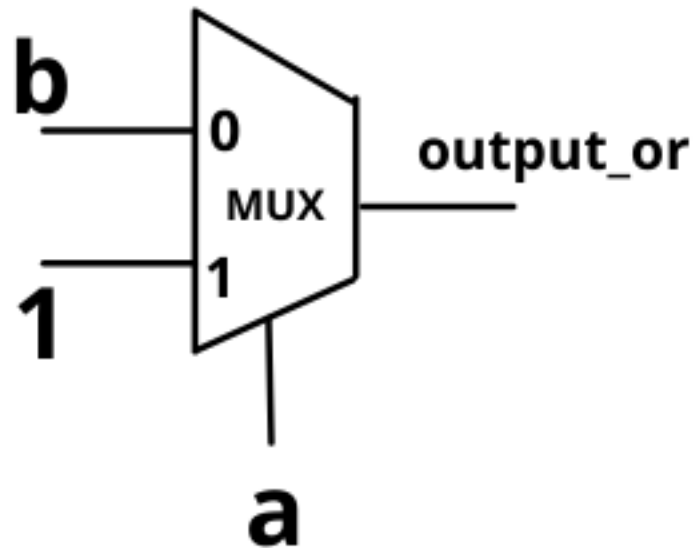


AND

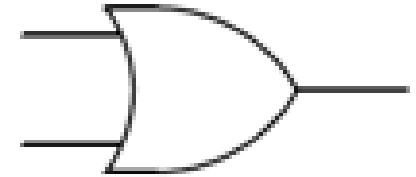
A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

Logic Gates using Multiplexer

OR Gate Using 2:1 MUX



$B \rightarrow I_0$
 $A \rightarrow S$
 $I_1 \rightarrow 5V$



OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

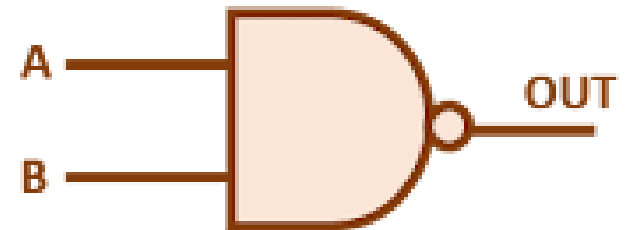
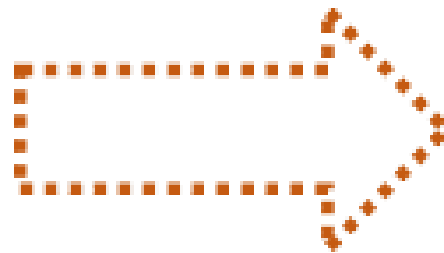
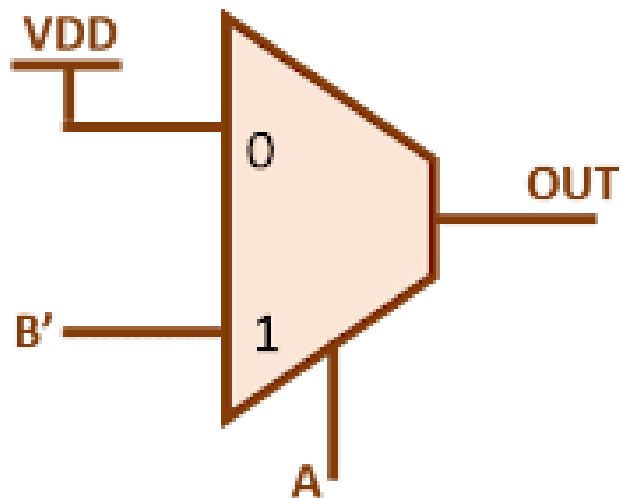
Logic Gates using Multiplexer

NAND Gate using MUX

A	B	OUT
0	0	1
0	1	1
1	0	1
1	1	0

OUT = 1
when A = 0

OUT = B'
when A = 1



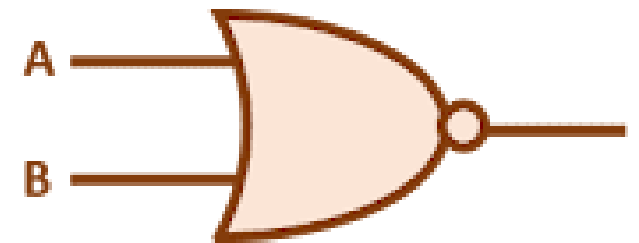
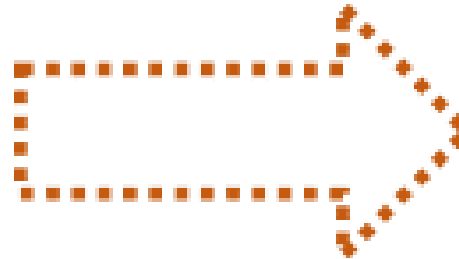
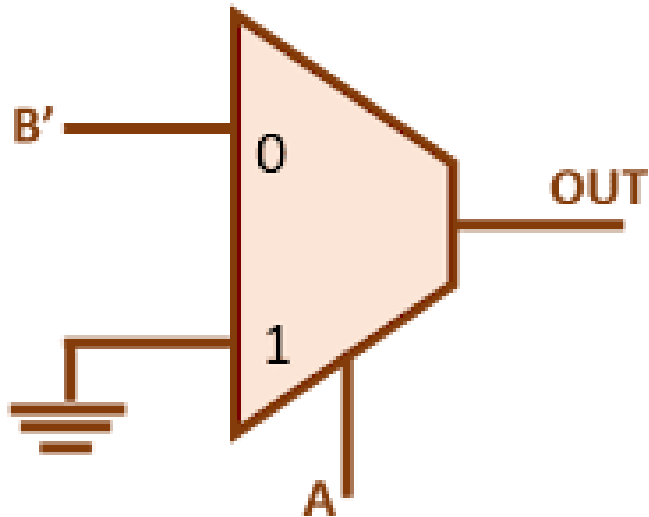
Logic Gates using Multiplexer

NOR Gate using MUX

A	B	OUT
0	0	1
0	1	0
1	0	0
1	1	0

OUT = B' when A = 0

OUT = 0 when A = 1



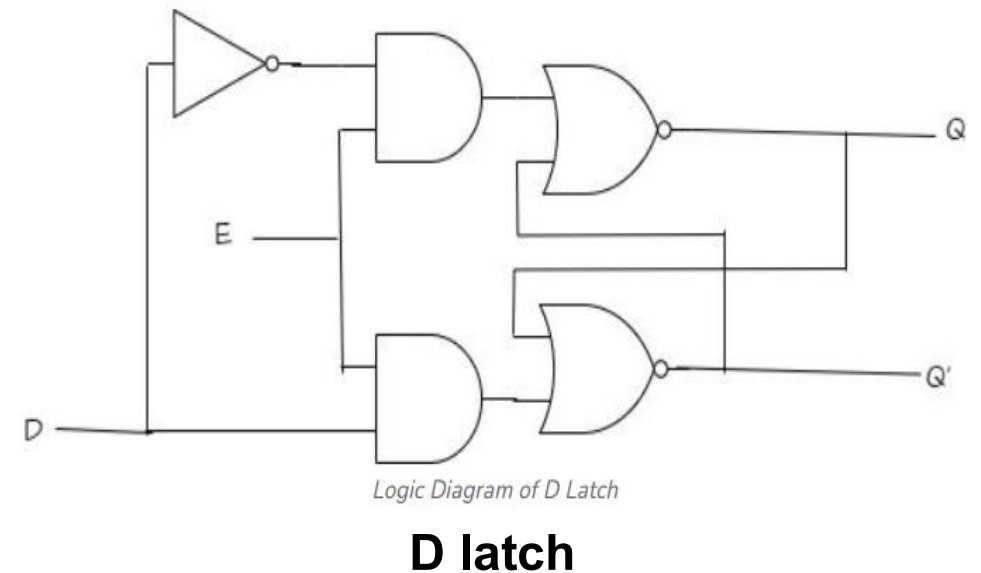
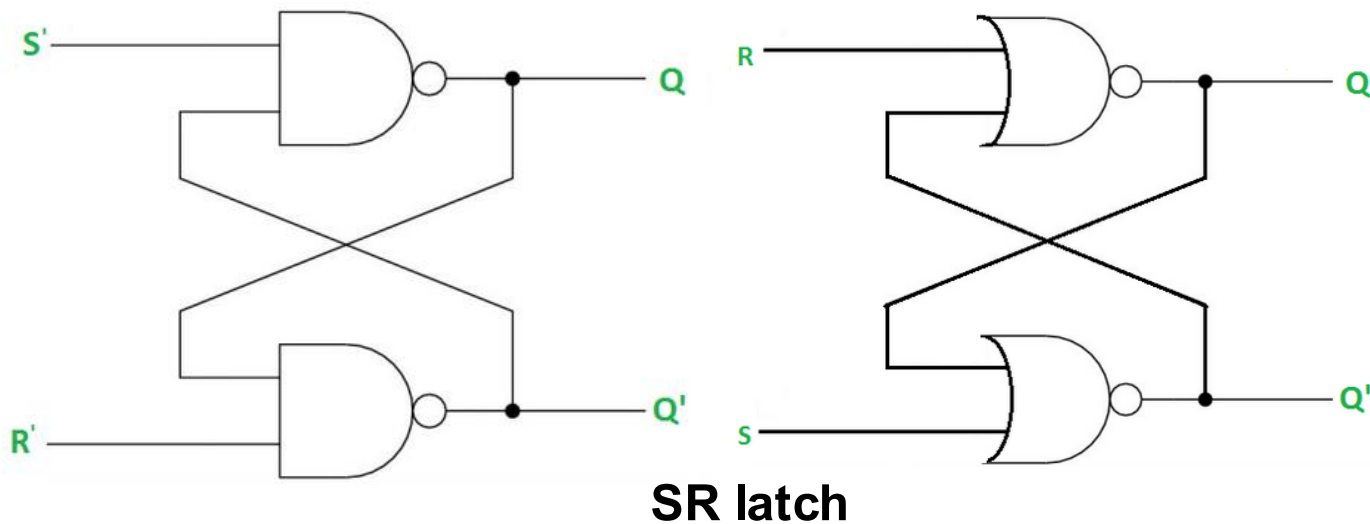
Difference between combinational and sequential circuits

Combinational Circuit	Sequential Circuit
This output is solely dependent on the current input.	This output is affected by both current and previous input.
The process is quick.	The process is slow.
It is intended to be simple.	When compared to combinational circuits, it is more efficiently designed.
There is no feedback from input to output.	A feedback path exists between input and output.
This is not dependent on time.	This is time-sensitive.
Basic building blocks: Logical gates	Basic building blocks: Flip-flops
Used for both arithmetic and boolean operations.	Mostly used for data storage.
Combinational circuits are incapable of storing any state.	Sequential circuits can store any state or retain previous states.
Combinational circuits do not require triggering because they lack a clock.	Sequential circuits require triggering because they are clock dependent.
These circuits lack a memory element.	Memory elements are used in these circuits.
It is simple to use and manage.	It is difficult to use and handle.

- **Combinational circuits** are built with logic gates such as AND, OR, NOT, NAND, and NOR. These logic gates serve as the foundation for combinational circuits.
- **Sequential circuits** are built with counters, shift registers, latches, etc.

Latch

- Latches are digital circuits that can store a single bit of information and hold its value until it is updated by new input signals. Latches are **asynchronous**.
- They are used in digital systems as temporary storage elements to store binary information.
- Latches can be implemented using various digital logic gates, such as AND, OR, NOT, NAND, and NOR gates.
- Different types of latches: SR (Set-Reset) Latches, Gated SR Latches, D Latches, Gated D Latches, JK Latches, T Latches.



D Latch

- Level sensitive element
- *A positive level triggered D latch*
 - copies D to output Q, if Clock=1, else preserves the previous output
- *A negative level triggered D latch*
 - copies D to output Q, if Clock=0, else preserves the previous output

D Flip-flop

Clk	D	$Q(t+1)$
0	x	$Q(t)$
1	0	0
1	1	1

Characteristic table

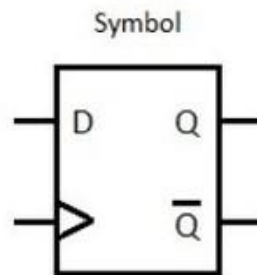
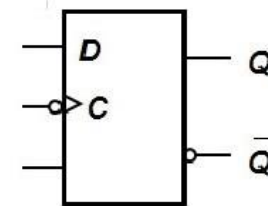


Table of truth:

clk	D	Q	$\bar{Q}$
0	0	Q	$\bar{Q}$
0	1	Q	$\bar{Q}$
1	0	0	1
1	1	1	0

Negative Edge triggering of D f/f

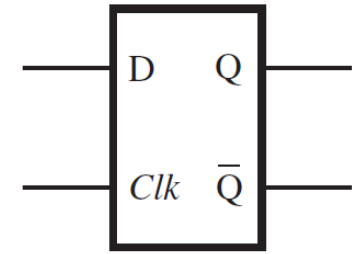
Symbol



Truth table

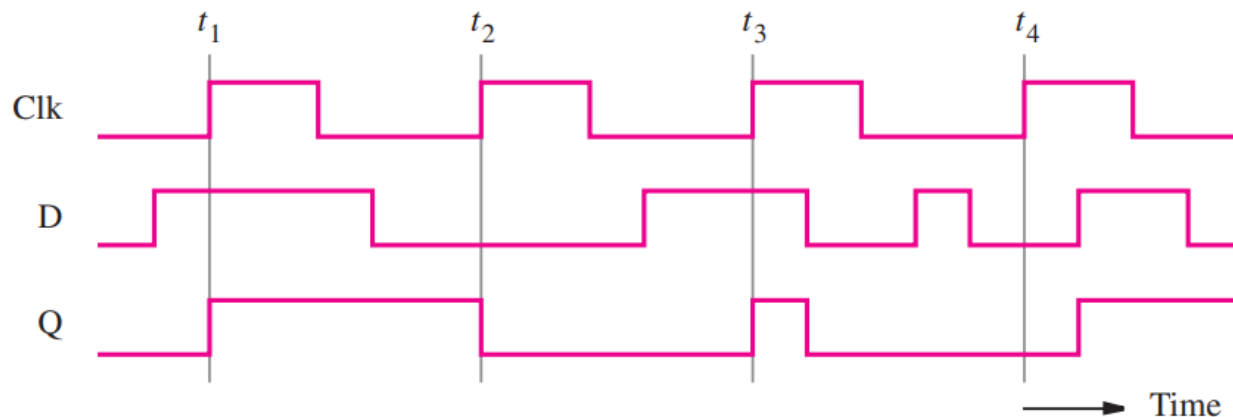
INPUTS		OUTPUTS		
CLK	D	Q	Q'	COMMENTS
0	0	0	1	NC
↑	1	0	1	NC
↓	0	0	1	RESET
↓	1	1	0	SET

D Latch



Graphical symbol

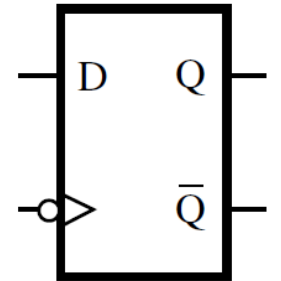
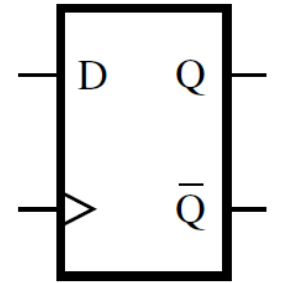
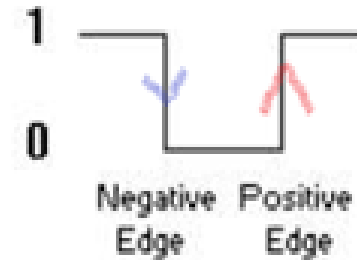
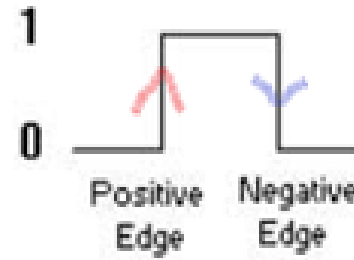
- Level sensitive element
- *A positive level triggered D latch*
 - copies D to output Q, if Clock=1, else preserves the previous output
- *A negative level triggered D latch*
 - copies D to output Q, if Clock=0, else preserves the previous output



Clk	D	$Q(t + 1)$
0	x	$Q(t)$
1	0	0
1	1	1

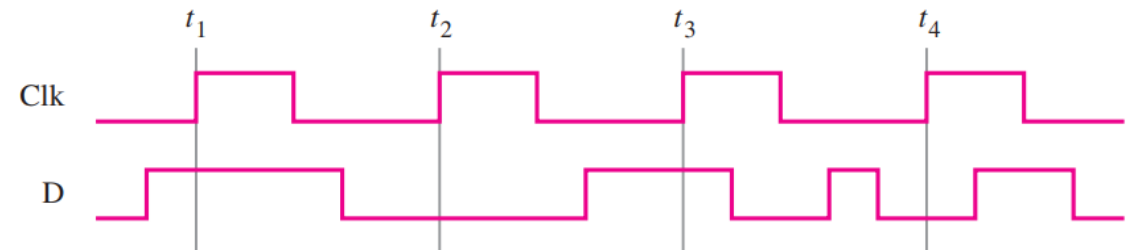
Characteristic table

D Flip-flop



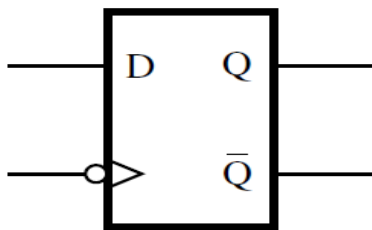
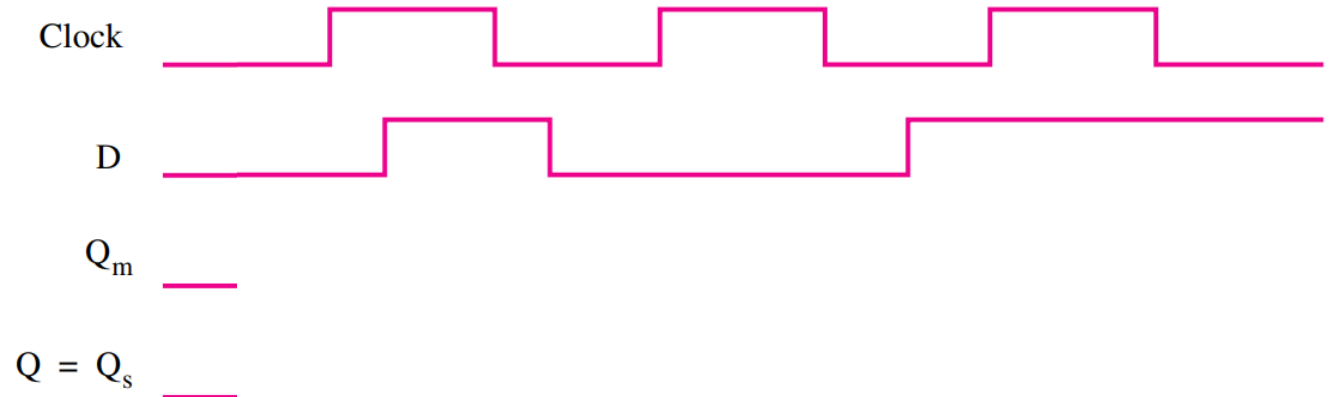
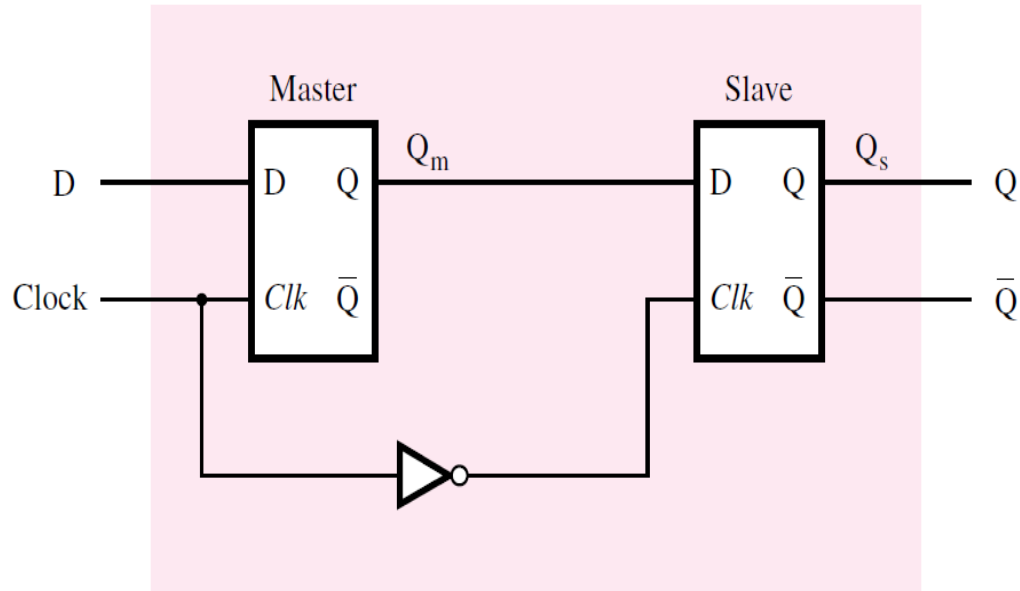
Graphical symbol

- Edge sensitive element
- **Flip-flops are synchronous** and operate based on clock signal
- *A positive edge triggered D flip-flop*
 - Sets $Q=D$ at all positive edges (rising edges) of the clock, retains the old value of Q otherwise
- *A negative edge triggered D flip-flop*
 - Sets $Q=D$ at all negative edges (falling edges) of the clock, retains the old value of Q otherwise



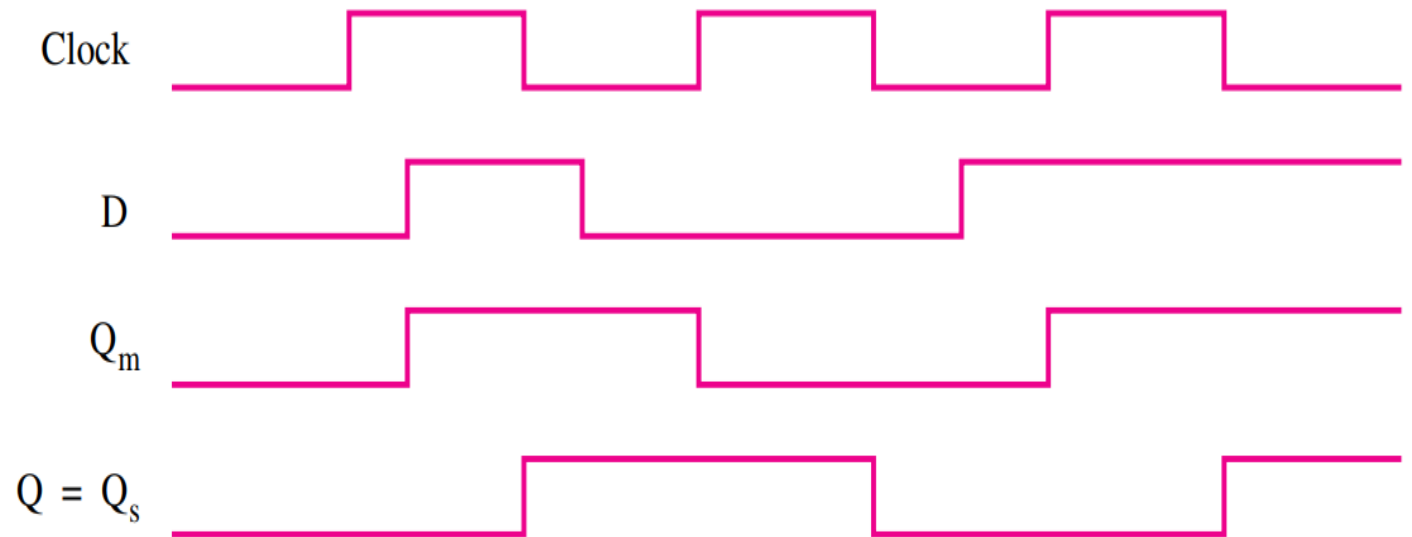
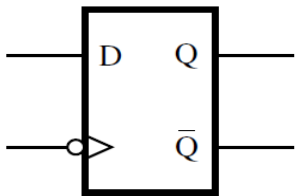
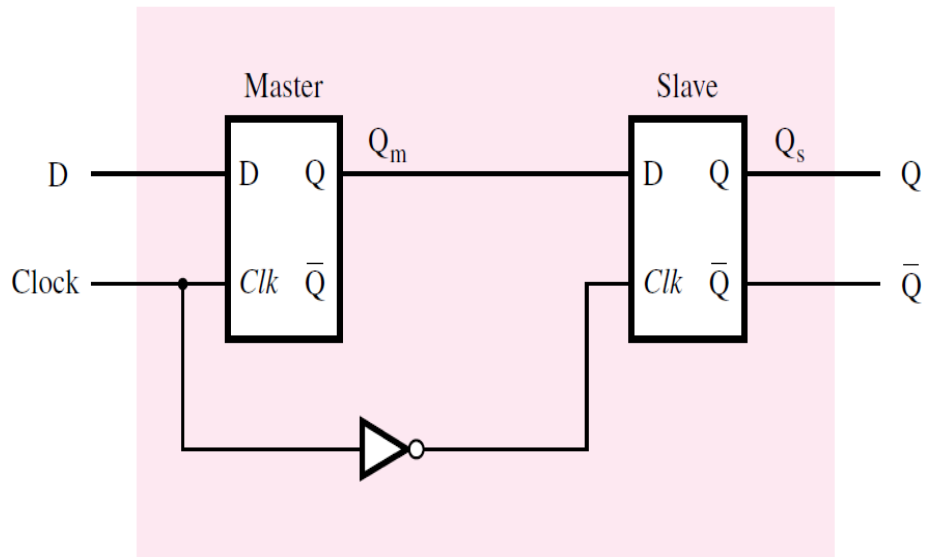
Building D Flip-flops using D Latches

- By cascading a positive level triggered D latch and a negative level triggered D latch we can build a negative edge triggered D flip-flop

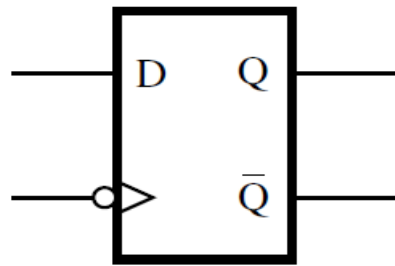
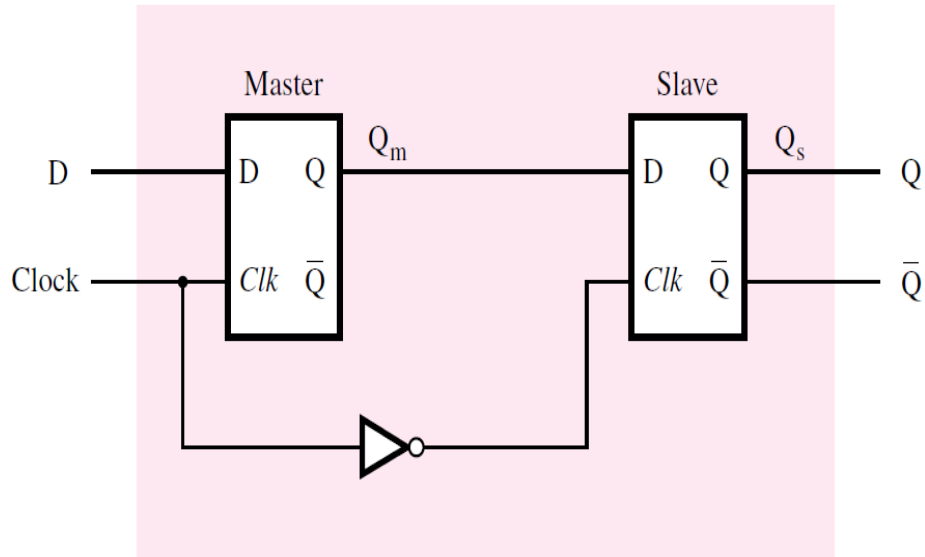


Building D Flip-flops using D Latches

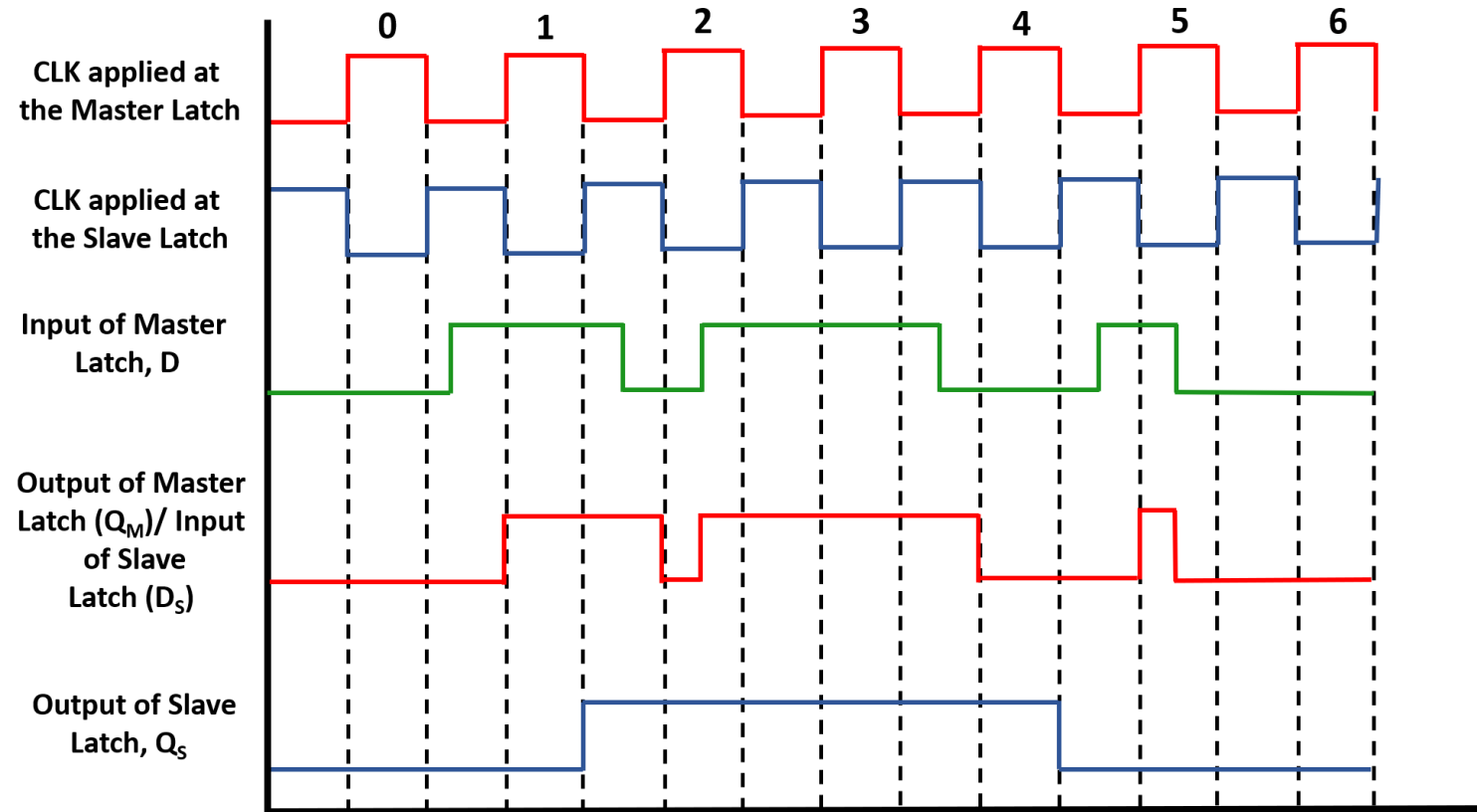
- By cascading a positive level triggered D latch and a negative level triggered D latch we can build a negative edge triggered D flip-flop



Building D Flip-flops using D Latches



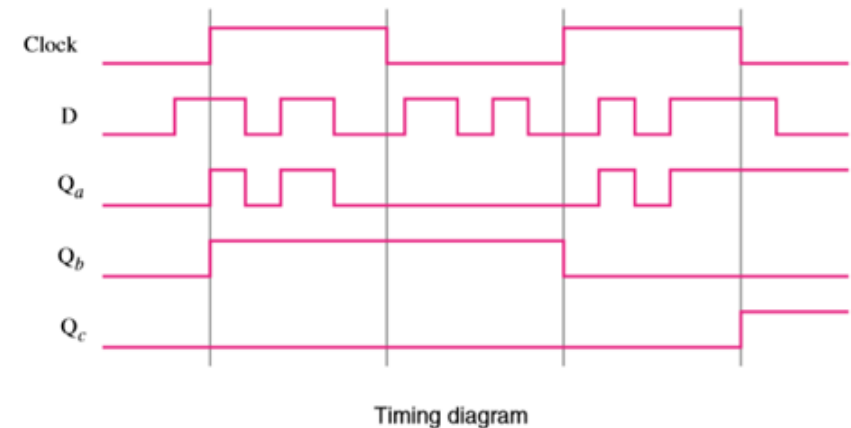
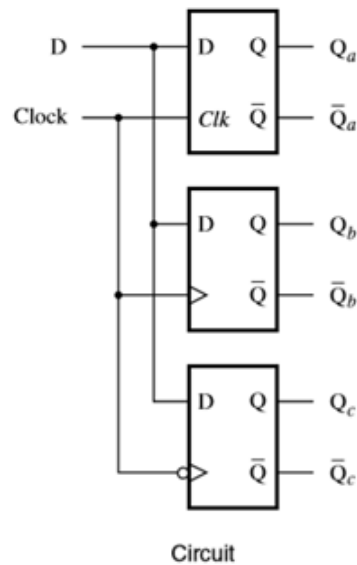
D Flip-flop



- The output of the master-slave D flip-flop is similar to the output of a negative edge-triggered D flip-flop (FF)

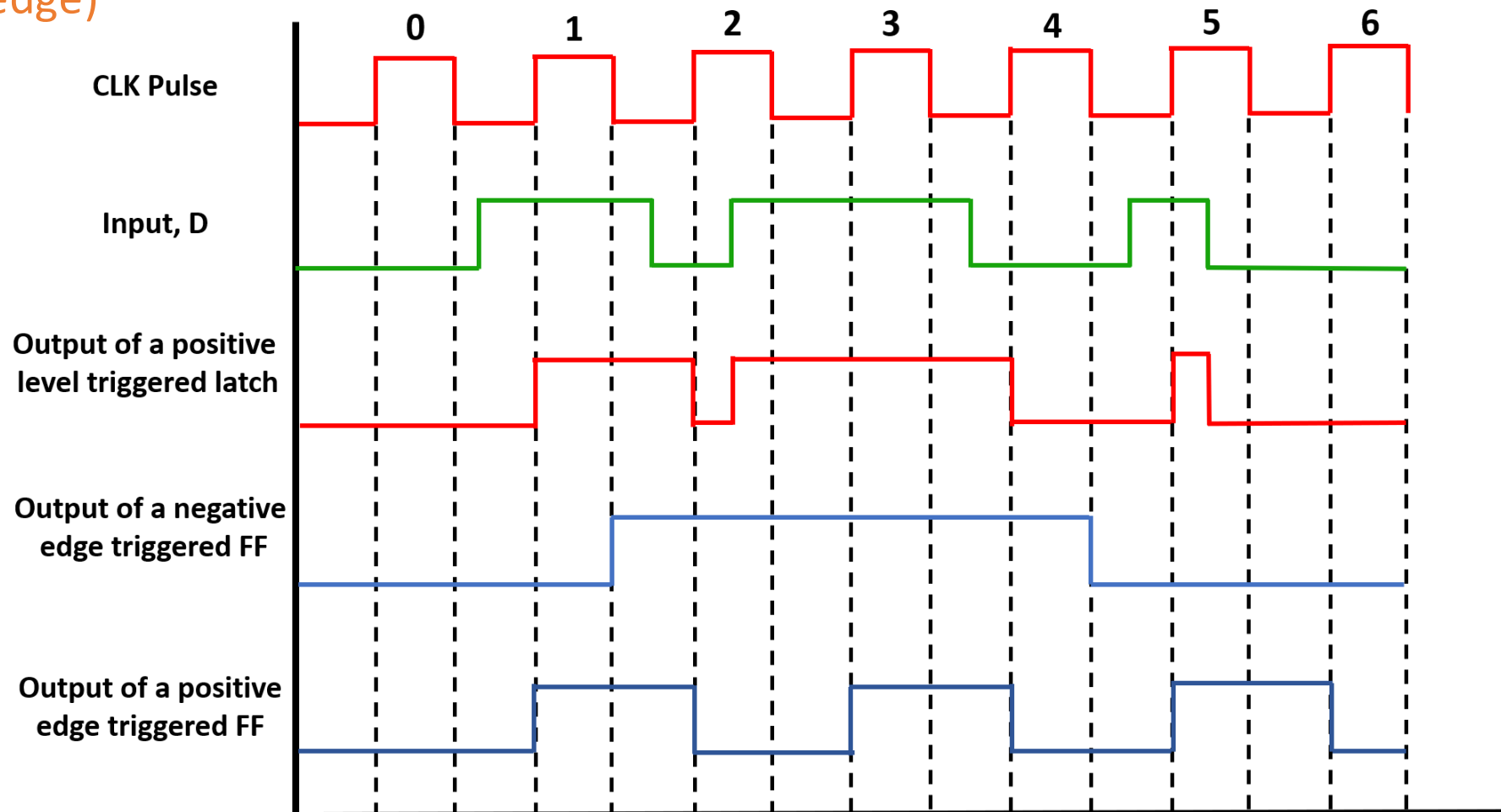
Level triggered vs. Edge triggered

- In level triggered elements
 - output is affected by the clock levels (*high/low*)
- In edge triggered elements
 - output is affected by the clock edges (positive edge/negative edge) (rising edge/falling edge)



Level triggered vs. Edge triggered

- In level triggered elements
 - output is affected by the clock levels (*high/low*)
- In edge triggered elements
 - output is affected by the clock edges (positive edge/negative edge) (rising edge/falling edge)



Slide references

1. <https://instrumentationtools.com/logic-gates/>
2. Stephen Brown & Zvonko Vranesic - Fundamentals of Digital Logic with Verilog Design

Thank you!

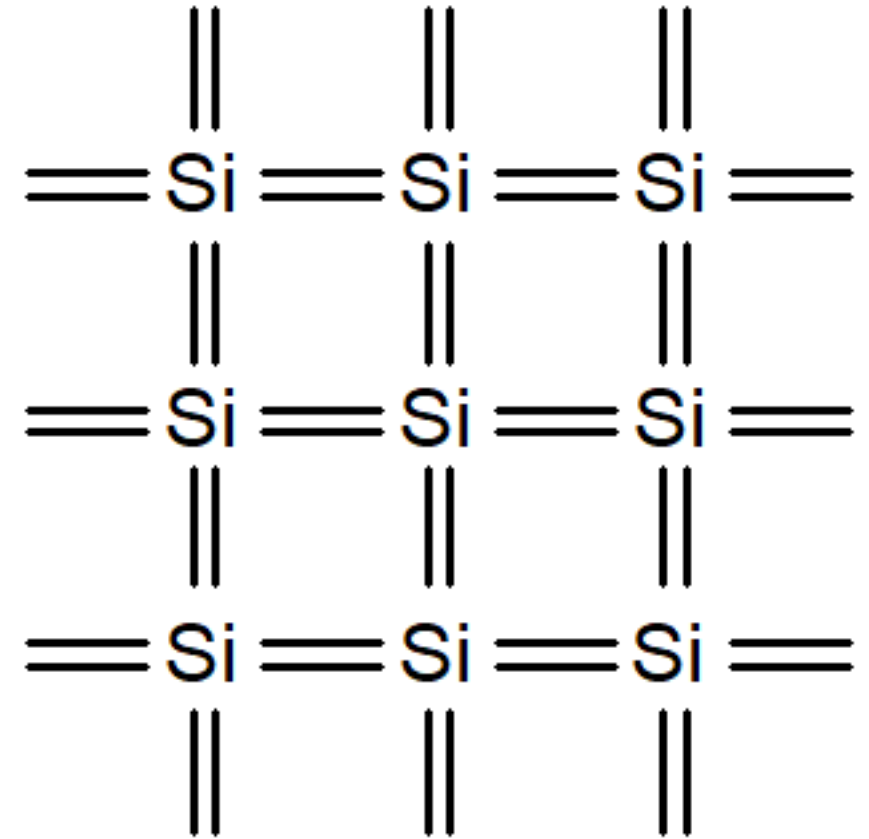
CMOS Logic

Contents

- Silicon lattice structure
- p-type and n-type semiconductors, p-n junction diode
- nMOS and pMOS transistors
- Transistors as switches
- Basic logic gate review
- CMOS gate structure
- Pull-up network (PUN) and pull-down network (PDN)
- Series and parallel PUN & PDN networks
- CMOS implementation of AOI, OAI and other logic gates

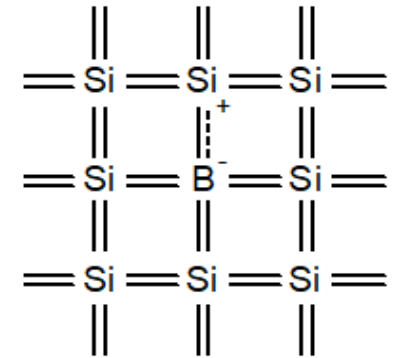
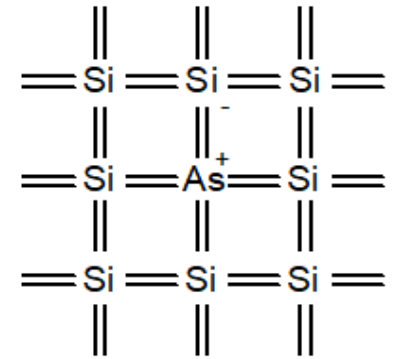
Silicon Lattice

- **Silicon is a semiconductor**
- Transistors are built on a silicon substrate
- Silicon is a **Group IV** material
 - Si (16) $\rightarrow 1s^2 2s^2 2p^6 3s^2 3p^2$
- Forms crystal lattice with covalent bonds to four neighbors
- Pure silicon has no free carriers and conducts poorly
- *How to increase its conductivity?*



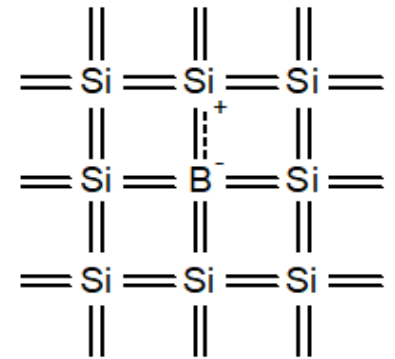
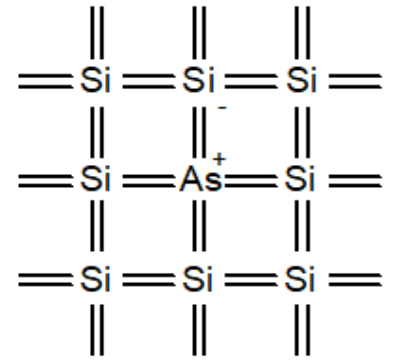
Doped (Impure) Silicon

- *How to increase the conductivity of a pure semiconductor (usually Group IV materials)?*
- Answer: by adding impurity! (dopants)
- Adding dopants increases the conductivity
- **Group V** dopants: provides *free “electron”* to the pure semiconductor (Ex: Arsenic)
 - Called the n-type semiconductor afterwards
- **Group III** dopants: provides *free “holes”* to the pure semiconductor (Ex: Boron)
 - Called the p-type semiconductor afterwards



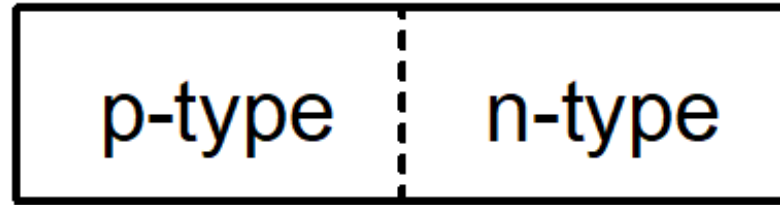
Concept Review

- n-type semiconductor
 - Carrier: electrons
 - *Net electrical charge of the material stays the same (**electrically neutral**)*
 - Difference with pure semiconductor: it has more free electrons, even after forming bonds with the neighboring atoms
 - Free electrons increase conductivity
- p-type semiconductor
 - Carrier: holes (absence of electrons)
 - *Net electrical charge of the material stays the same (**electrically neutral**)*
 - Difference with pure semiconductor: it has empty electron spaces



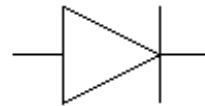
Concept Review

- A junction between p-type and n-type semiconductor forms a diode.
- Current flows only in one direction
- p-type to n-type, ~~n-type to n-type~~



anode

cathode

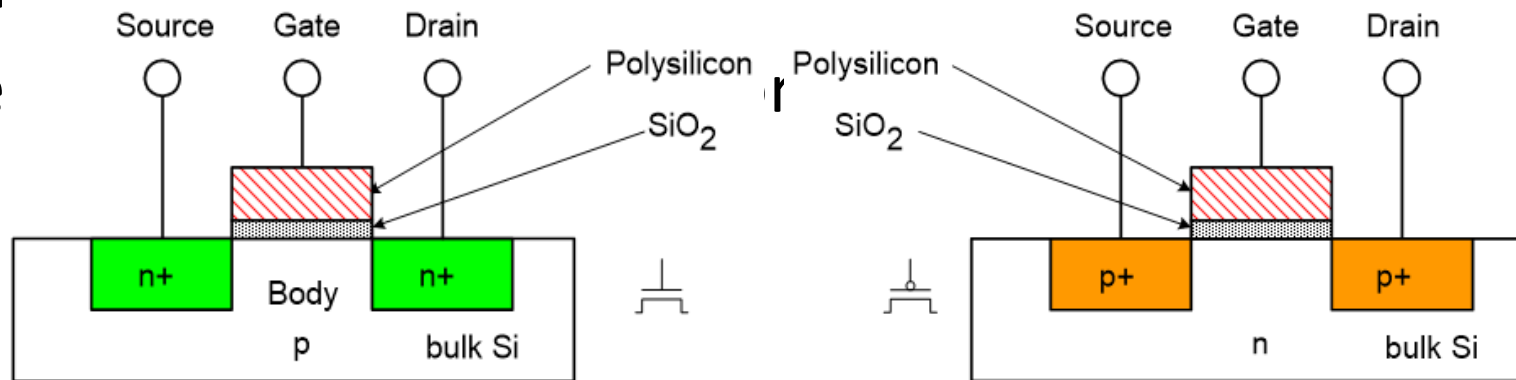


Electrical Voltage $\Leftrightarrow$ Logic Conversion

- **V_{DD} = Logical High (1)**
- **GND = Logical Low (0)**
- $V_{DD} = 5\text{ V}, 3.3\text{ V}, 2.5\text{ V}, 1.8\text{ V}, 1.5\text{ V}, 1.2\text{ V}, 1.0\text{ V}, \dots$ (electrical voltage)
- GND = 0 V (electrical voltage)
- V_{DD} has decreased in modern processes
 - High V_{DD} would damage modern tiny transistors
 - Lower V_{DD} saves power

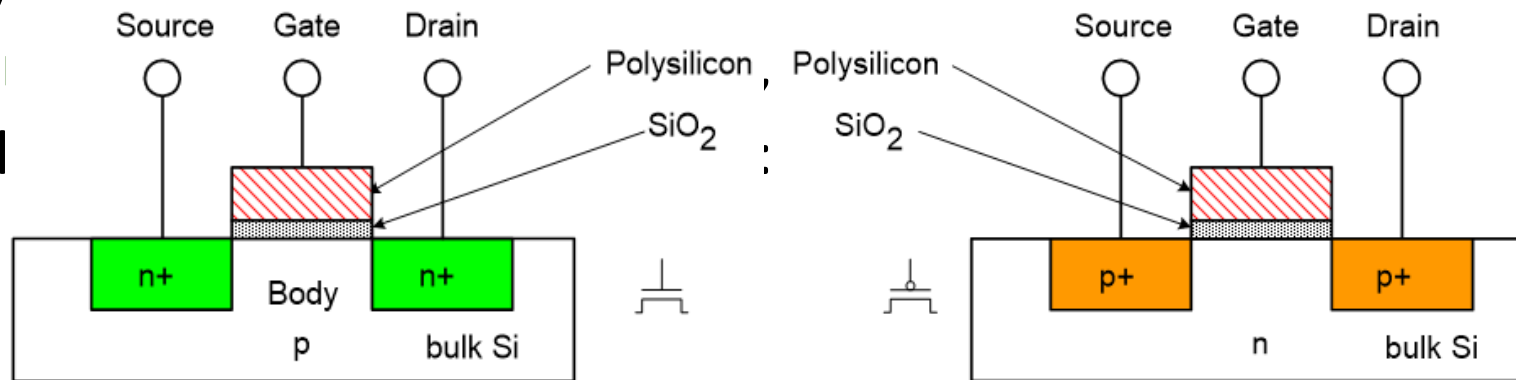
MOS Transistor

- Four terminals: gate, source, drain, body
 - gate, source, body -> conductors, SiO_2 (oxide) -> insulator
 - **source and drain are identical**, can be used interchangeably, no difference in construction
- Body is tied to the source: **functionally a 3 terminal device**
- Depending on the *voltage applied* to the gate terminal, source and drain are either



nMOS Transistor

- **Low voltage (GND)** applied to the gate: TRANSISTOR is **OFF**
 - source and drain are **disconnected** (no channel between them)
 - **no current flows**, transistor is **OFF**
- **High voltage (V_{DD})** applied to the gate: TRANSISTOR is **ON**
 - source and drain are **connected** by an *n-channel*, right under the gate oxide (SiO_2)
 - **current flows**
 - **detail**

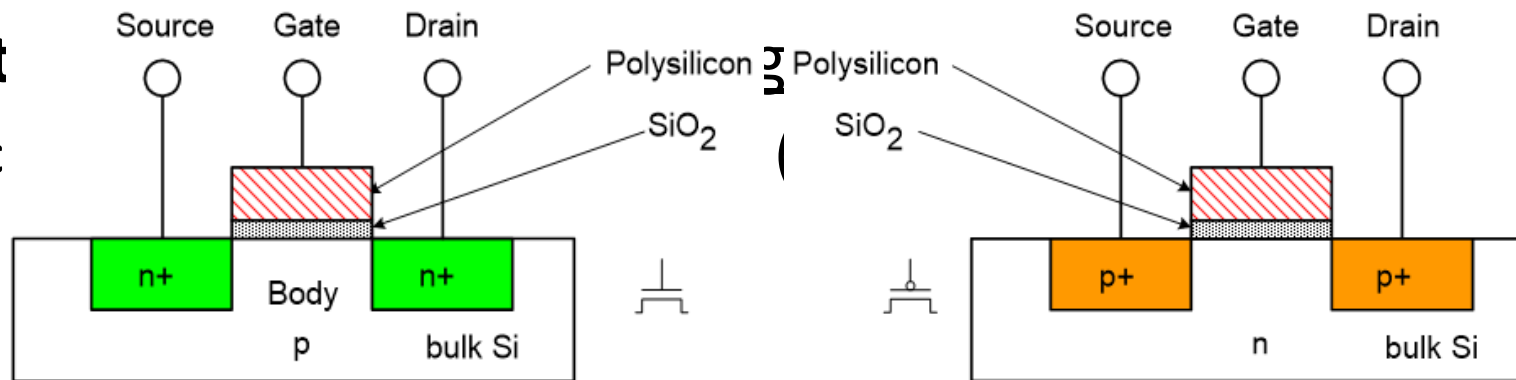


pMOS Transistor

- **Low voltage (GND)** applied to the gate: TRANSISTOR is **ON**
 - source and drain are **connected** by a *p-channel*, right under the gate oxide (SiO_2)
 - **current flows from source to drain (opposite direction of nMOS)**, transistor is **ON**
 - details on the working mechanism: later in this course!

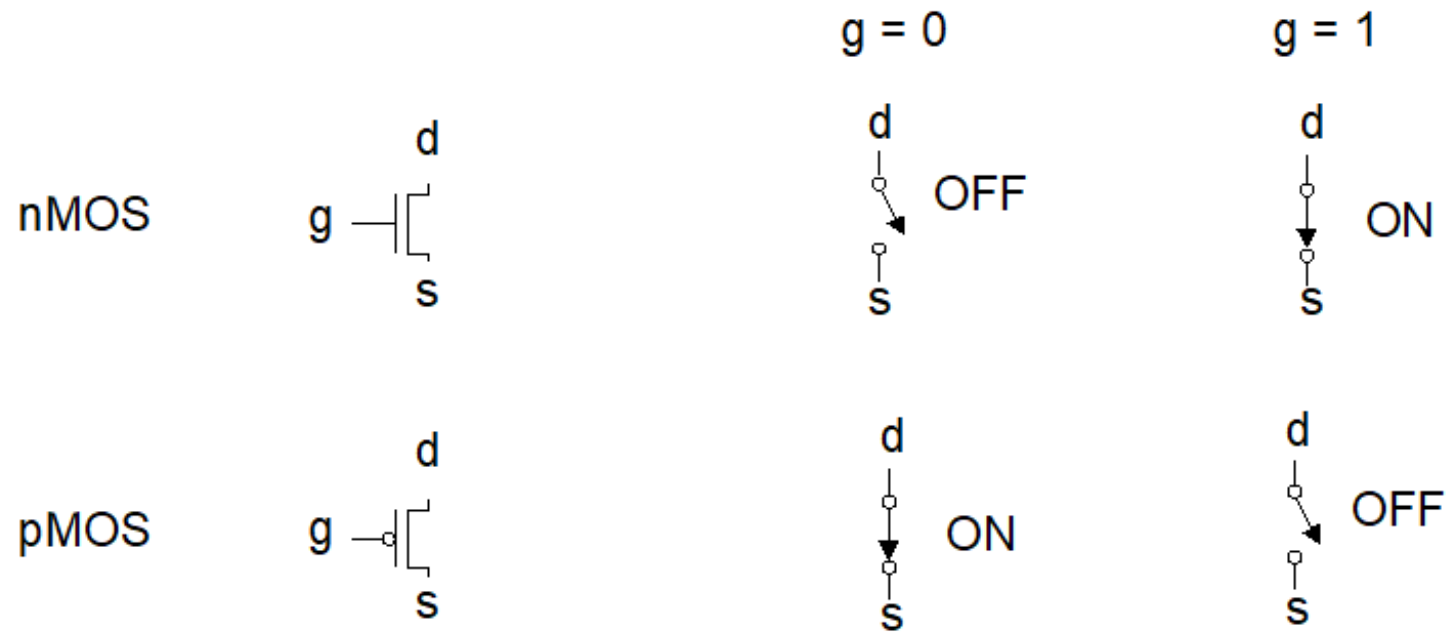
- **High volt**

- sourc
- **no cu**

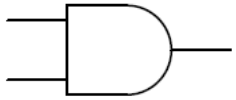


MOS Transistors as Switches

- Electrically controlled switches
- Voltage at gate controls path from source to drain

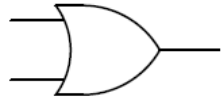


Basic logic gates



AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



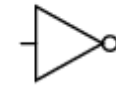
OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



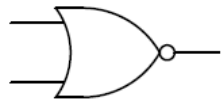
NOT

Input	Output
0	1
1	0



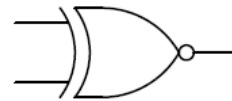
NAND

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0



NOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

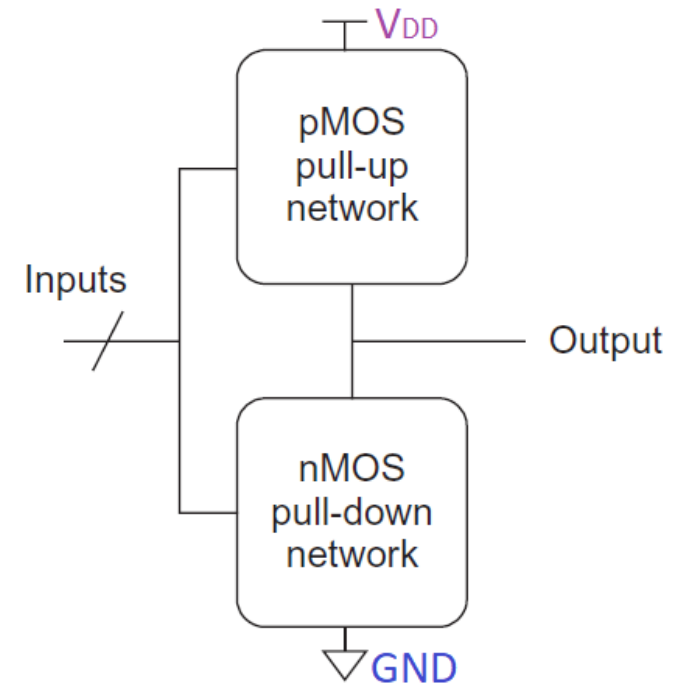


XNOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

CMOS

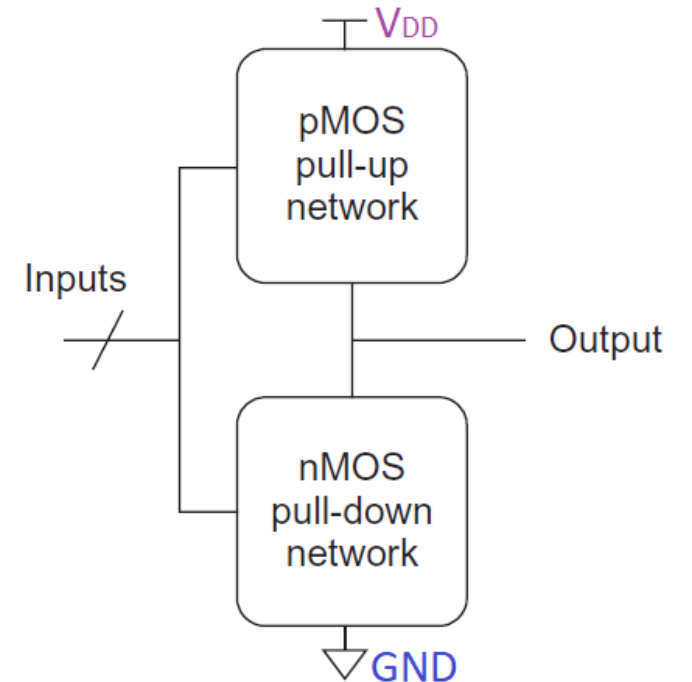
- CMOS can be used to build both simple and compound logic gates
- Consists of 2 networks
 - pMOS pull-up network
 - nMOS pull-down network
- pMOS & nMOS both used in the same structure -> CMOS



CMOS

- Depending on inputs the networks can be ON or OFF
- Network **ON**: Network is *conducting* electricity
- Network **OFF**: Network is *not conducting* electricity

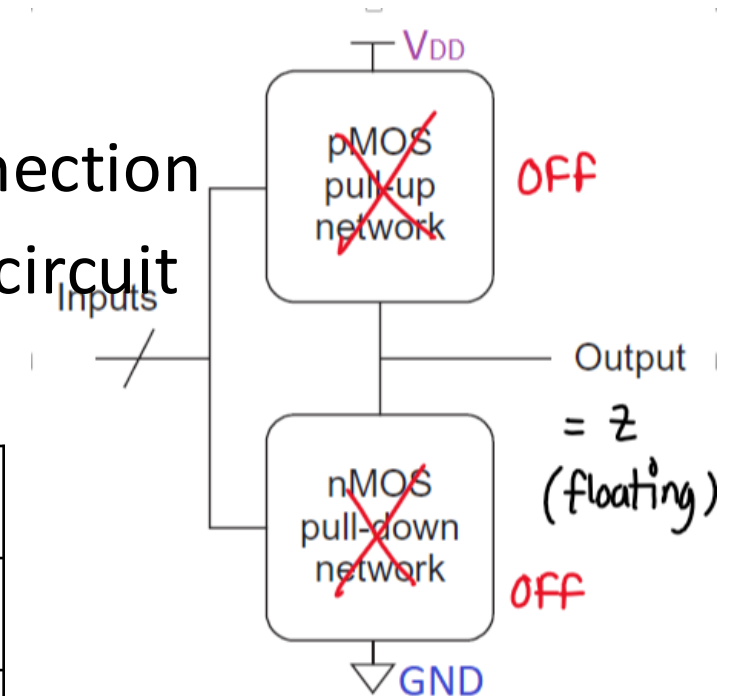
Depending on the inputs, 4 possible combinations of the 2 networks are possible:		
Pull-up OFF	Pull-up ON	
Pull-down OFF	Output = 1 (High)	
Pull-down ON	Output = 0 (Low)	Output = X (indeterminate)



CMOS gate structure

- Pull-up: **OFF**
- Pull-down: **OFF**
- Output is very high impedance/Z/floating/no connection
- Useful when we want to detach some part of our circuit

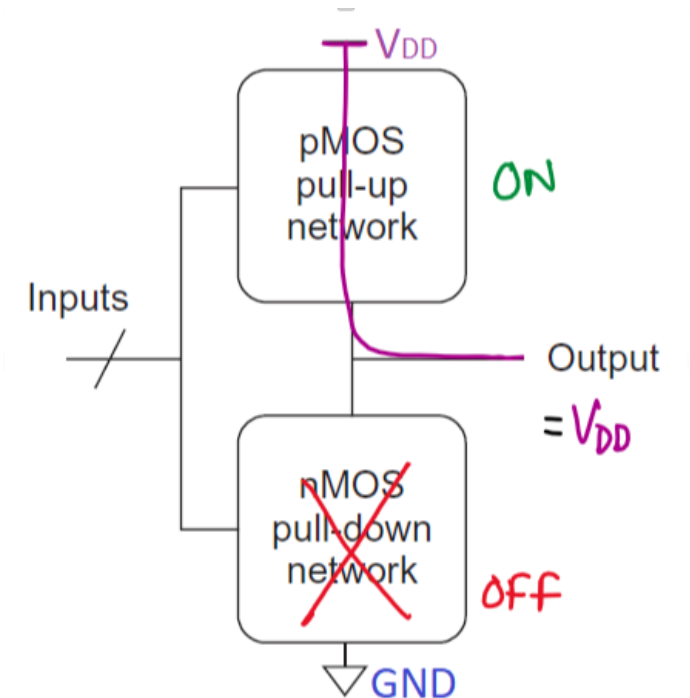
	Pull-up OFF	Pull-up ON
Pull-down OFF	Output = Z (float)	Output = 1 (High)
Pull-down ON	Output = 0 (Low)	Output = X (indeterminate)



CMOS gate structure

- Pull-up: **ON**
- Pull-down: **OFF**
- Output is *pulled-up* to V_{DD}
- Useful when we want to output **logical high (1)**

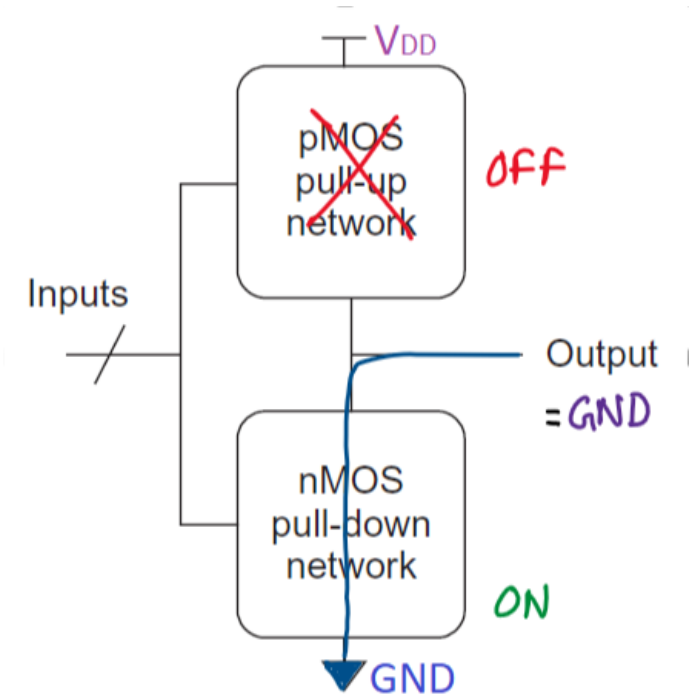
	Pull-up OFF	Pull-up ON
Pull-down OFF	Output = Z (float)	Output = 1 (High)
Pull-down ON	Output = 0 (Low)	Output = X (indeterminate)



CMOS gate structure

- Pull-up: **OFF**
- Pull-down: **ON**
- Output is *pulled-down* to **GND**
- Useful when we want to output **logical low (0)**

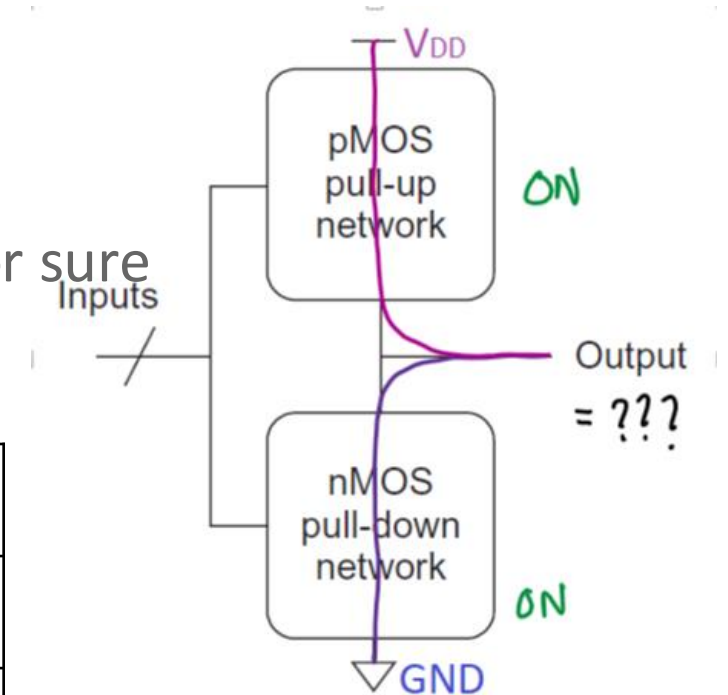
	Pull-up OFF	Pull-up ON
Pull-down OFF	Output = Z (float)	Output = 1 (High)
Pull-down ON	Output = 0 (Low)	Output = X (indeterminate)



CMOS gate structure

- Pull-up: ON
- Pull-down: ON
- Output is indeterminate
 - somewhere between V_{DD} and GND , we don't know for sure
- Usually a faulty condition

	Pull-up OFF	Pull-up ON
Pull-down OFF	Output = Z (float)	Output = 1 (High)
Pull-down ON	Output = 0 (Low)	Output = X (indeterminate)

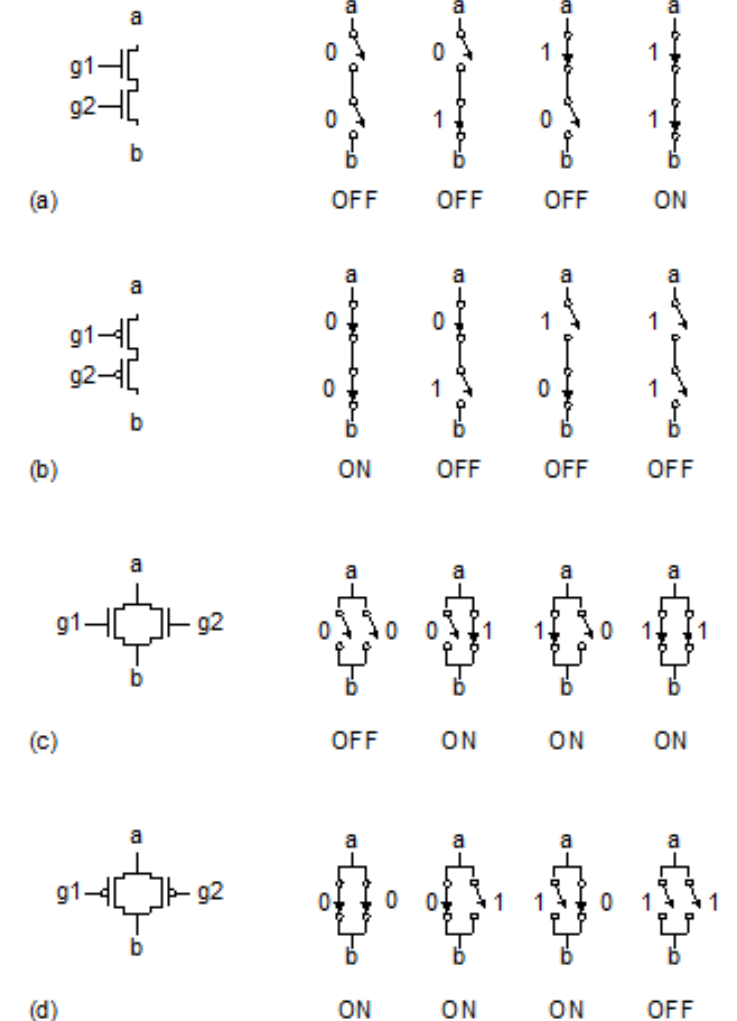


Pull-up & Pull-down Networks

- Simplest pull-up network: a single pMOS
 - Complex pull-up network can be build from different series and parallel combinations of pMOS transistors
-
- Simplest pull-down network: a single nMOS
 - Complex pull-down network can be build from different series and parallel combinations of nMOS transistors

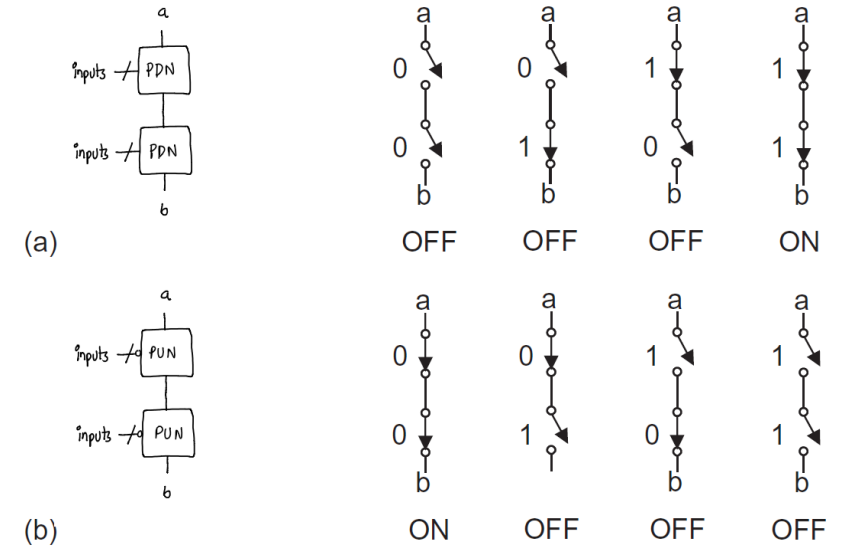
Series and Parallel Networks: *Simple* case

- Pull-up network: a single pMOS
 - High voltage at gate input: **OFF**
 - Low voltage at gate input: **ON**
- Pull-down network: a single nMOS
 - High voltage at gate input: **ON**
 - Low voltage at gate input: **OFF**
- *Series* combination of networks
 - *Every network* must be **ON** to make the **whole network conducting** (figure: a, b)
- *Parallel* combination of networks
 - *At least one network* must be **ON** to make the **whole network conducting** (figure: c, d)



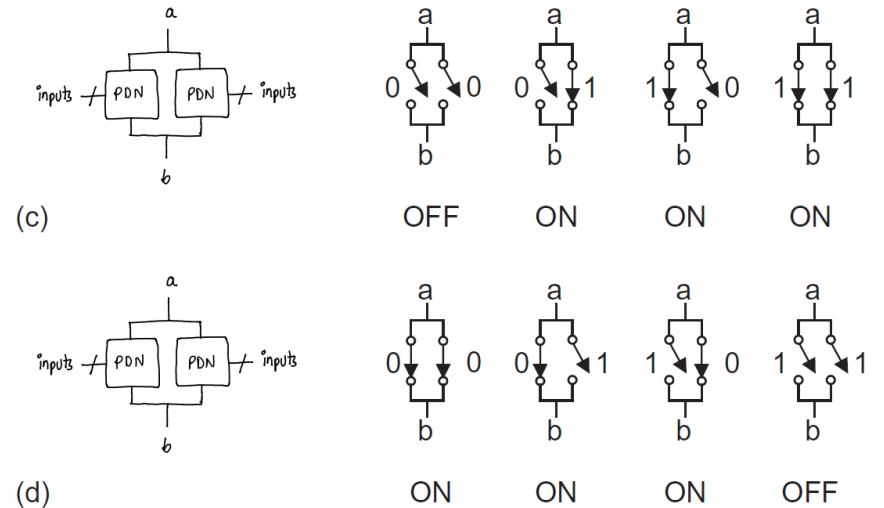
Series and Parallel Networks: *General* case 1

- Pull-up network: multiple pMOS transistors
 - *series* combination
 - Low voltage at all gate inputs: **ON**
 - *parallel* combination
 - Low voltage at minimum one gate input: **ON**
- Pull-down network: multiple nMOS transistors
 - *series* combination
 - High voltage at all gate inputs: **ON**
 - *parallel* combination
 - High voltage at minimum one gate input: **ON**



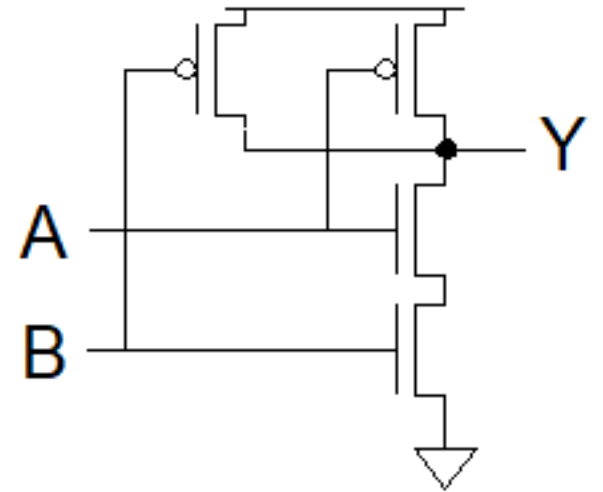
Series and Parallel Networks: *General* case 2

- Pull-up network: multiple pMOS transistors
 - *series* combination
 - Low voltage at all gate inputs: **ON**
 - *parallel* combination
 - Low voltage at minimum one gate input: **ON**
- Pull-down network: multiple nMOS transistors
 - *series* combination
 - High voltage at all gate inputs: **ON**
 - *parallel* combination
 - High voltage at minimum one gate input: **ON**



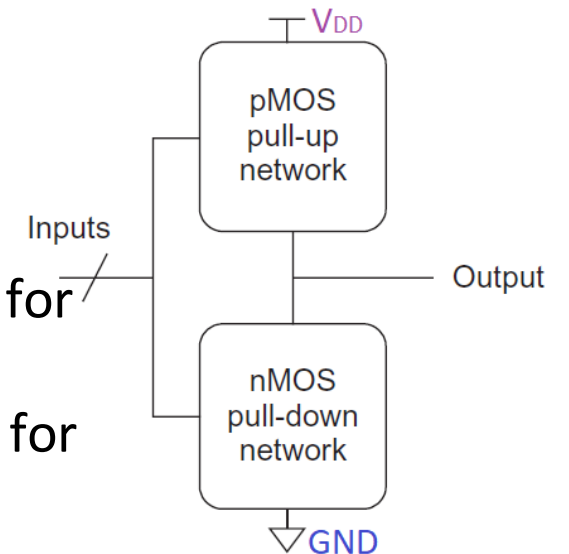
Conduction Complement

- Rule of Conduction Complements
 - Pull-up network is complement of pull-down (and vice versa)
 - Parallel $\rightarrow$ series, series $\rightarrow$ parallel
- Example: CMOS NAND-2 gate
 - Pull-down: 2 nMOS in **series** connection with inputs A & B
 - Pull-up: 2 pMOS in **parallel** (compliment of series) connection with the same inputs A & B
- Conclusion: for any CMOS logic gate, we just need to build only one network (usually the pull-down, because its easier) and the conduction complement rule will take care of the other one!



Building An Inverting CMOS Logic Gate

1. Build the **pull-down network** using nMOS transistors
 - a. series nMOS transistor networks for inputs/signals that are “**AND**”ed
 - b. parallel nMOS transistors for inputs/signals that are “**OR**”ed
2. Build the **pull-up network** using pMOS transistors
 - a. Using the conduction complement rule
 - b. Parallel networks in pull-down network will be redrawn in series for pull-up network
 - c. Series networks in pull-down network will be redrawn in parallel for pull-up network
3. Connect the pull-up network (PUN) and the pull-down network (PDN) in series between V_{DD} and GND

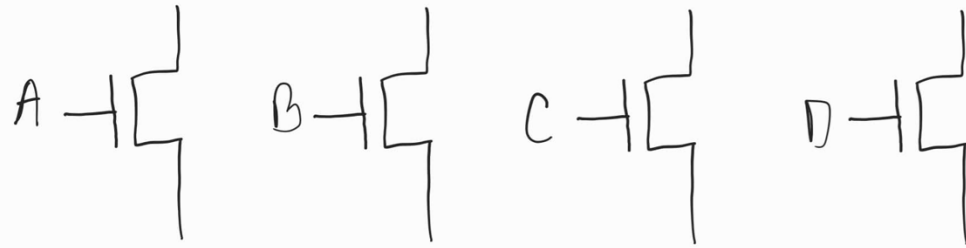


An Example: Building the Pull-down Network

- **series nMOS** transistor networks for inputs/signals that are “**AND**”ed together
- **parallel nMOS** transistors for inputs/signals that are “**OR**”ed together
 - Example: consider the inverting function, $Y = [(A+BC)D]'$
 - Step 1: Individual PD networks for A, B, C, D inputs (simple case)
 - 4 single nMOS transistors
 - Step 2: PD network for **BC** signal
 - B & C ANDed in **BC**: B & C PD networks will be in **series**
 - Step 3: PD network for **(A+BC)** signal
 - A and BC ORed (**A+BC**): A & BC PD networks will be in **parallel**
 - Step 4: PD network for **(A+BC)D** signal
 - (A+BC) and D ANDed in **(A+BC)D**: (A+BC) & D PD networks will be in **series**
 - (The term “signal” is used to describe an intermediate value which are generated from inputs. Here A,B,C,D are inputs but A+BC or BC are referred to as signal)

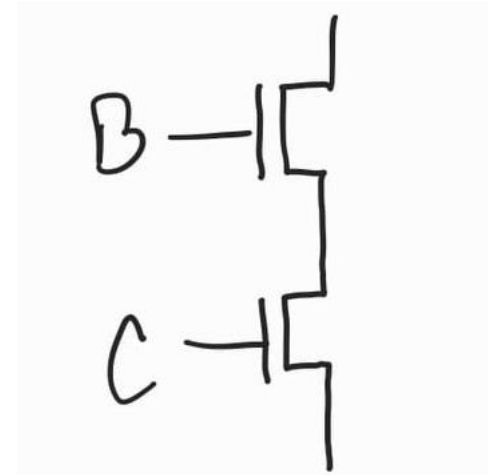
An Example: Building the Pull-down Network

- $Y = [(A+BC)D]'$
 - Step 1: Individual PD networks for A, B, C, D inputs (simple case)
 - 4 single nMOS transistors



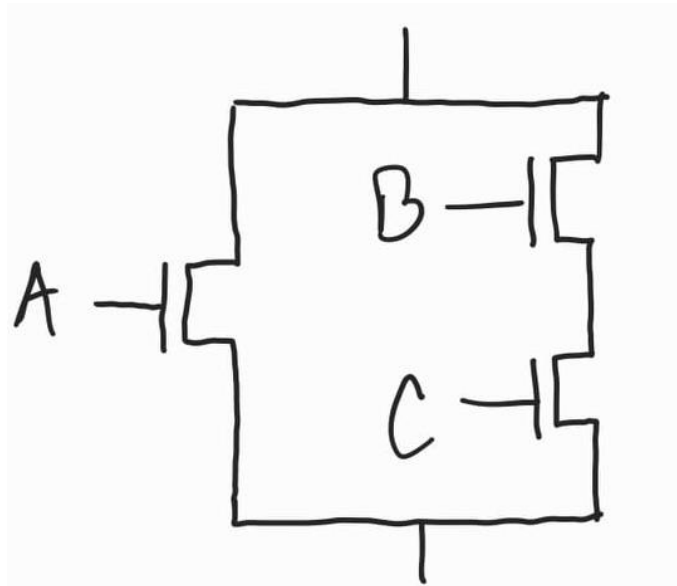
An Example: Building the Pull-down Network

- $Y = [(A+BC)D]'$
 - Step 2: PD network for **BC** signal
 - B & C ANDed in **BC**: B & C PD networks will be in **series**



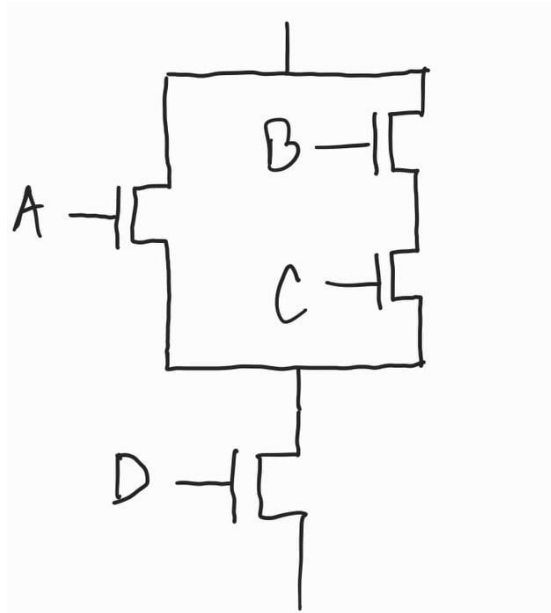
An Example: Building the Pull-down Network

- $Y = [(A+BC)D]'$
 - Step 3: PD network for **(A+BC)** signal
 - A and BC ORed **(A+BC)**: A & BC PD networks will be in **parallel**



An Example: Building the Pull-down Network

- $Y = [(A+BC)D]'$
 - Step 4: PD network for **$(A+BC)D$** signal
 - $(A+BC)$ and D ANDed in **$(A+BC)D$** : $(A+BC)$ & D PD networks will be in **series**

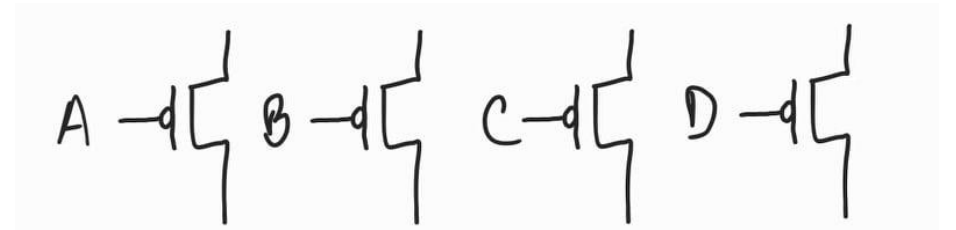
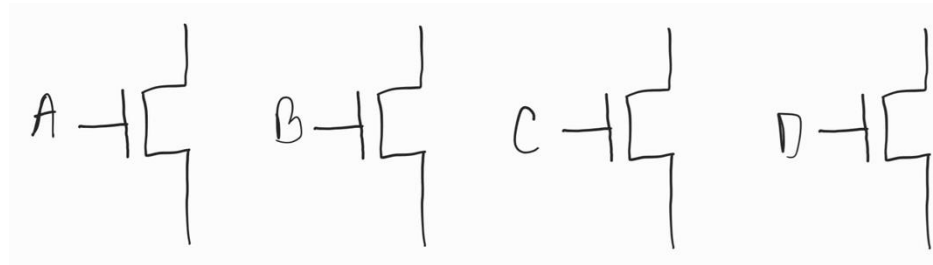


Building the Pull-up Network

- Just follow the CONDUCTION COMPLEMENT rule!!
- Use pMOS transistors instead of nMOS
- Parallel networks in pull-down network will be redrawn in series for pull-up network
- Series networks in pull-down network will be redrawn in parallel for pull-up network

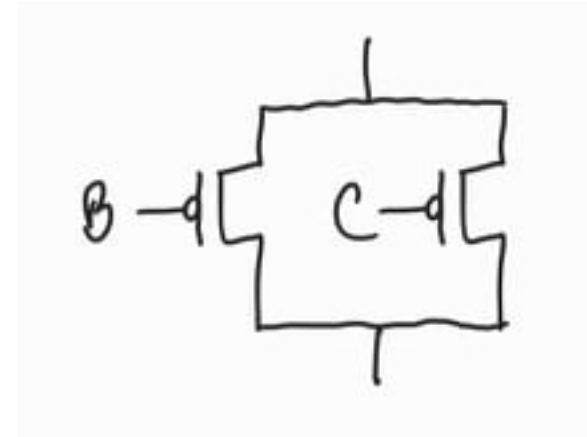
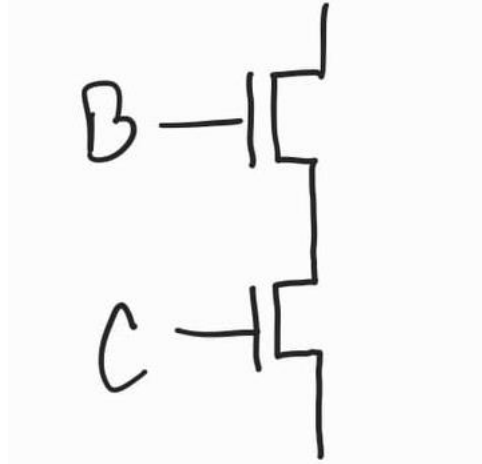
An Example: Building the Pull-up Network

- $Y = [(A+BC)D]'$
 - Step 1: Individual PU networks for A, B, C, D inputs (simple case)
 - 4 single pMOS transistors



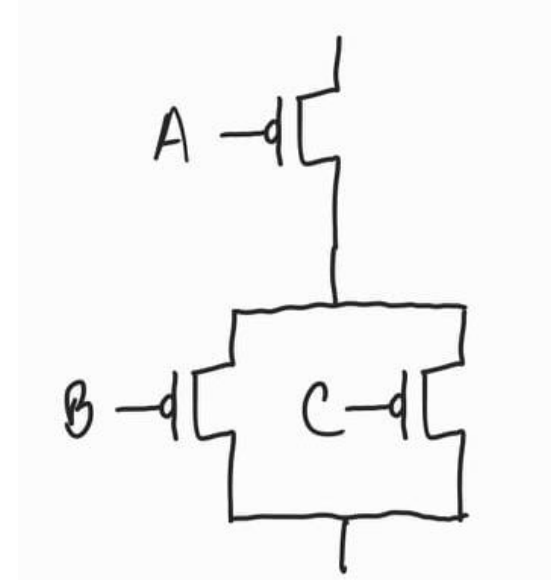
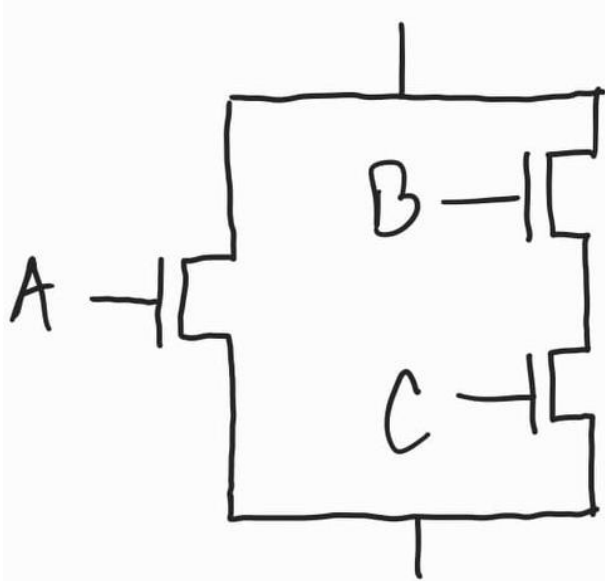
An Example: Building the Pull-up Network

- $Y = [(A+BC)D]'$
 - Step 2: PU network for **BC** signal
 - B & C PD networks was in series
 - B & C PU networks will be in **parallel**



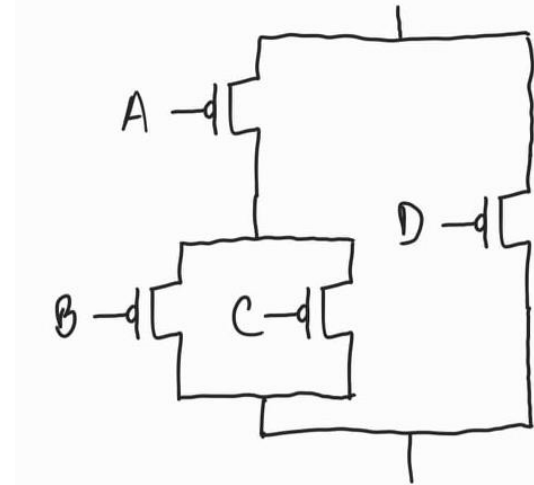
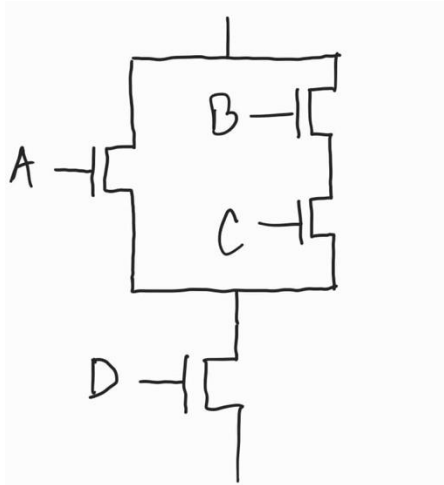
An Example: Building the Pull-up Network

- $Y = [(A+BC)D]'$
 - Step 3: PU network for **(A+BC)** signal
 - A & BC PD networks was in parallel
 - A & BC PU networks will be in **series**



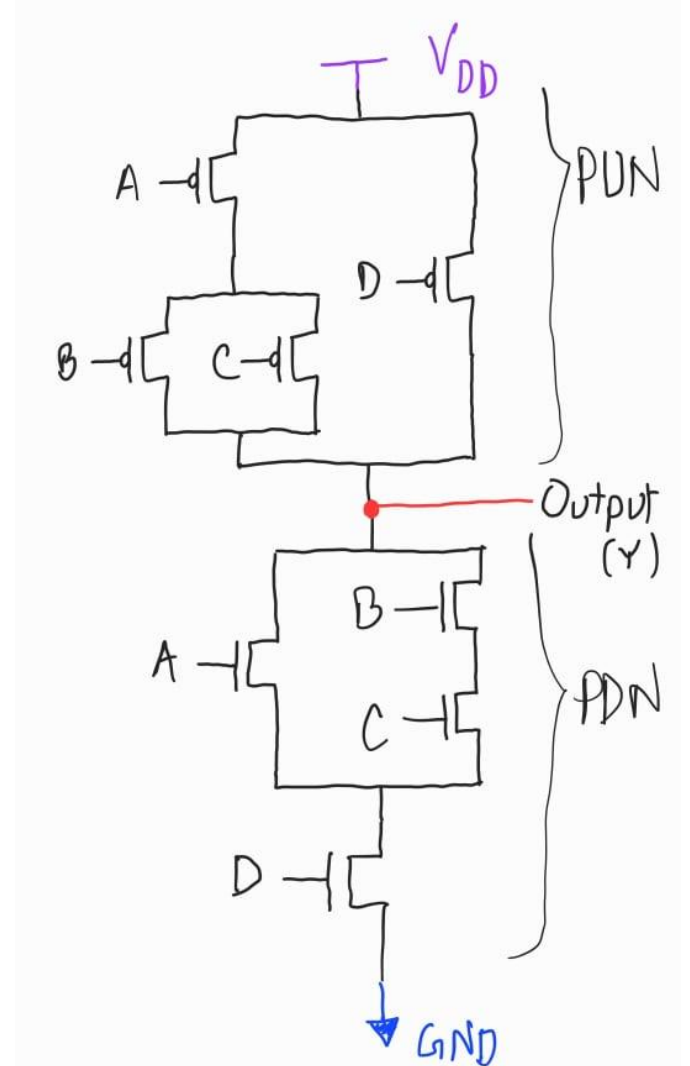
An Example: Building the Pull-up Network

- $Y = [(A+BC)D]'$
 - Step 4: PU network for $(A+BC)D$ signal
 - $(A+BC)$ & D PD networks was in series
 - $(A+BC)$ & D PU networks will be in parallel



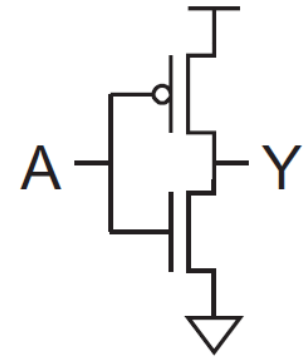
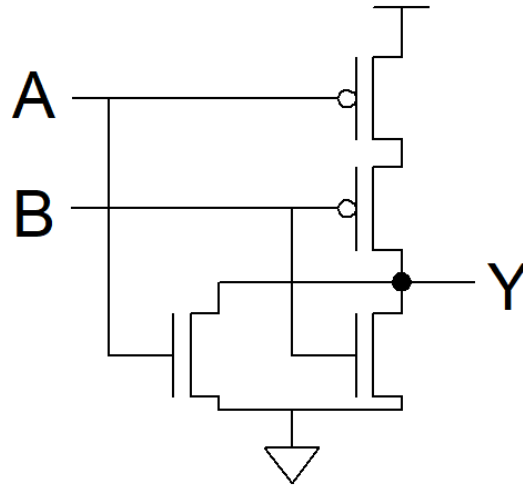
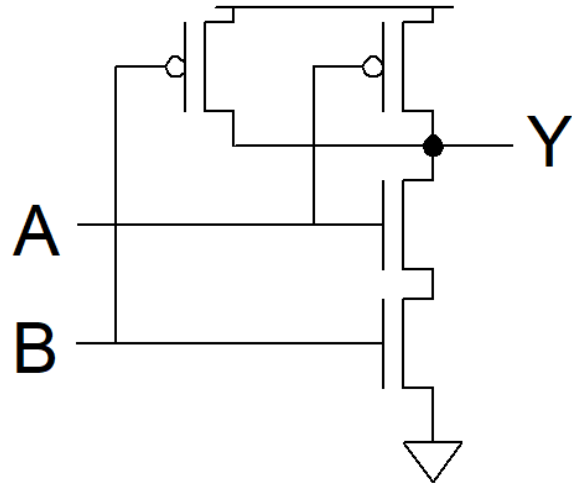
An Example: Putting it all together

- $Y = [(A+BC)D]'$
- Connect the pull-up network (PUN) and the pull-down network (PDN) in series between V_{DD} and GND
- The Output of the inverting function (Y) is taken from the common node of PUN and PDN
- This is the CMOS logic gate for the given inverting logic function Y



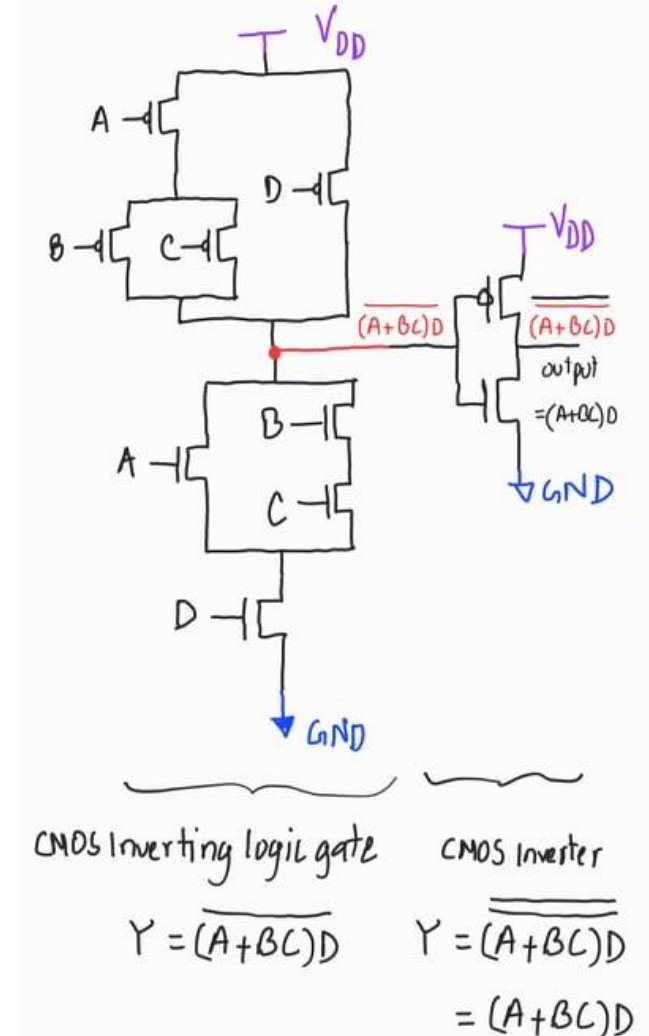
Example: CMOS NAND-2, NOR-2 and Inverter Gates

- NAND-2 [$Y=(AB)'$] NOR-2 [$Y=(A+B)'$] Inverter [$Y=A'$] logic functions can also be implemented in CMOS like this (try yourself first):



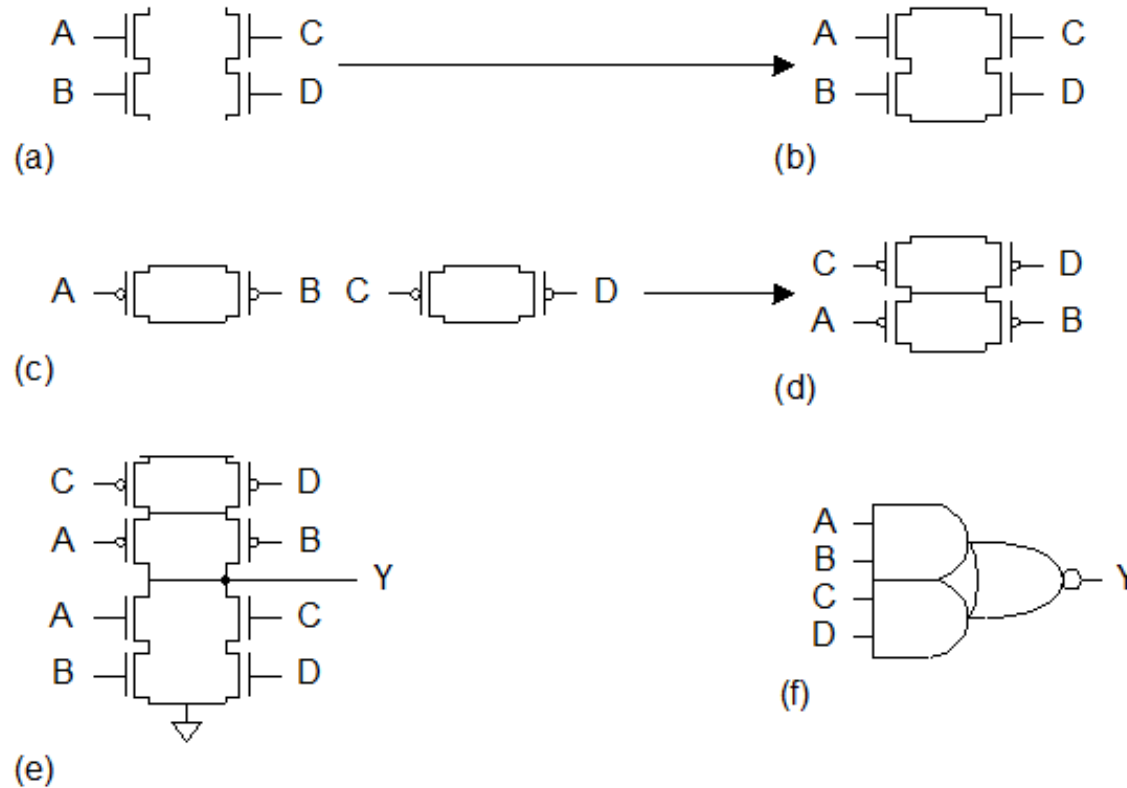
Implementing non-inverting logics in CMOS

- The techniques described so far can only be applied to build any **inverting** logic function
- There are multiple ways to implement **non-inverting** logic functions using CMOS
- The easiest way is to use an additional inverter at the output of the *inverting* function
- Example: $Y = (A+BC)D$ can be build like this
 - Build a CMOS logic gate for $Y = [(A+BC)D]'$
 - Connect the output of the previous gate to the input of an CMOS inverter
 - The output of the inverter will be $Y = (A+BC)D$



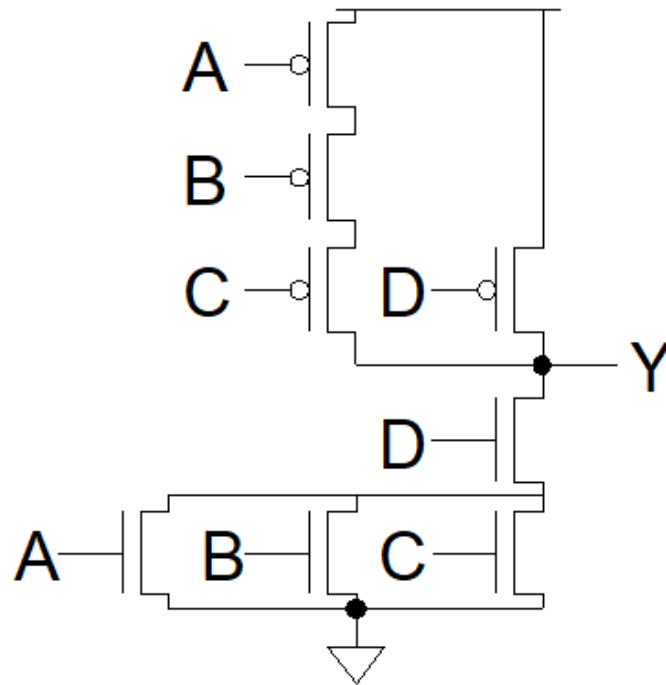
Example: CMOS AOI22 Gate

- $Y = (AB+CD)'$ also called the AND-OR-INVERT-2-2 or AOI22 GATE



Final Example: CMOS OAI31 Gate

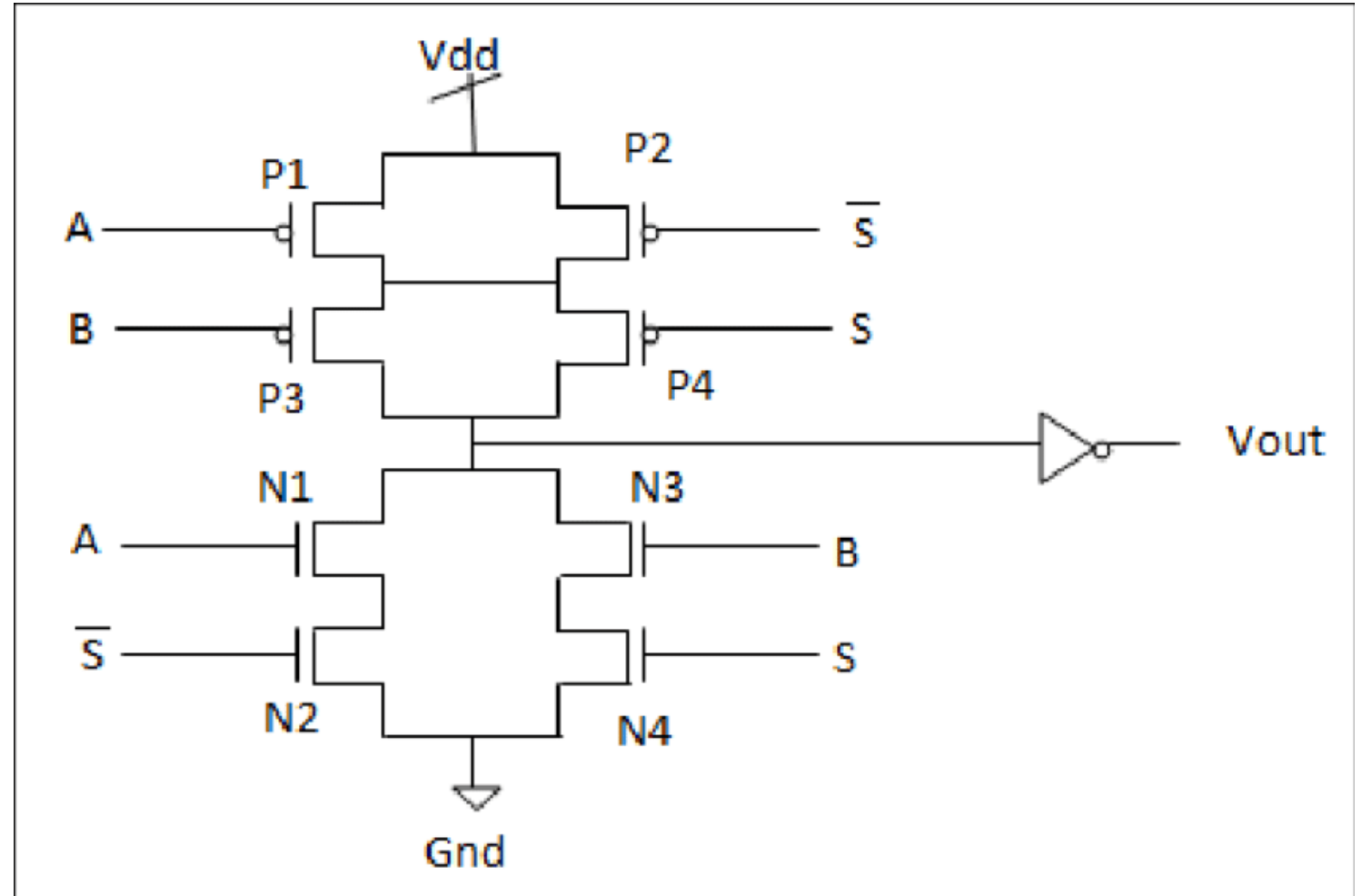
- $Y = [(A+B+C)D]'$ also called the OR-AND-INVERT-3-1 or OAI31 GATE



Final Example: CMOS Implementation of 2x1 MUX using

$$Y = S'A + SB$$

How many transistors required?



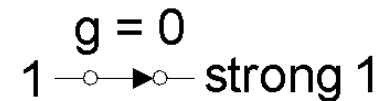
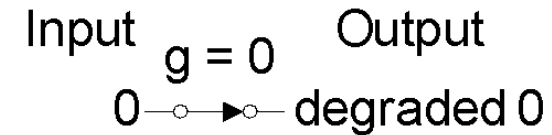
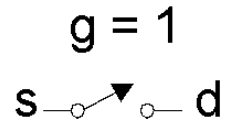
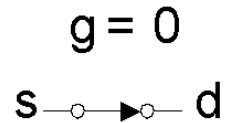
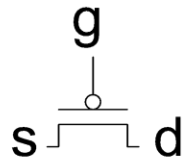
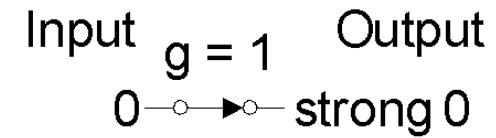
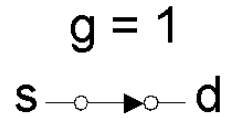
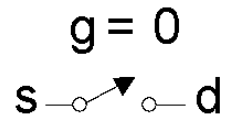
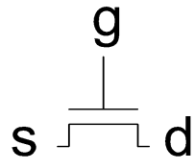
Thank you!

Signal Strength

- ❑ *Strength* of signal
 - How close it approximates ideal voltage source
- ❑ V_{DD} and GND rails are strongest 1 and 0
- ❑ nMOS pass strong 0
 - But degraded or weak 1
- ❑ pMOS pass strong 1
 - But degraded or weak 0
- ❑ Thus **nMOS are best for pull-down network**
- ❑ And, **pMOS are best for pull-up network**

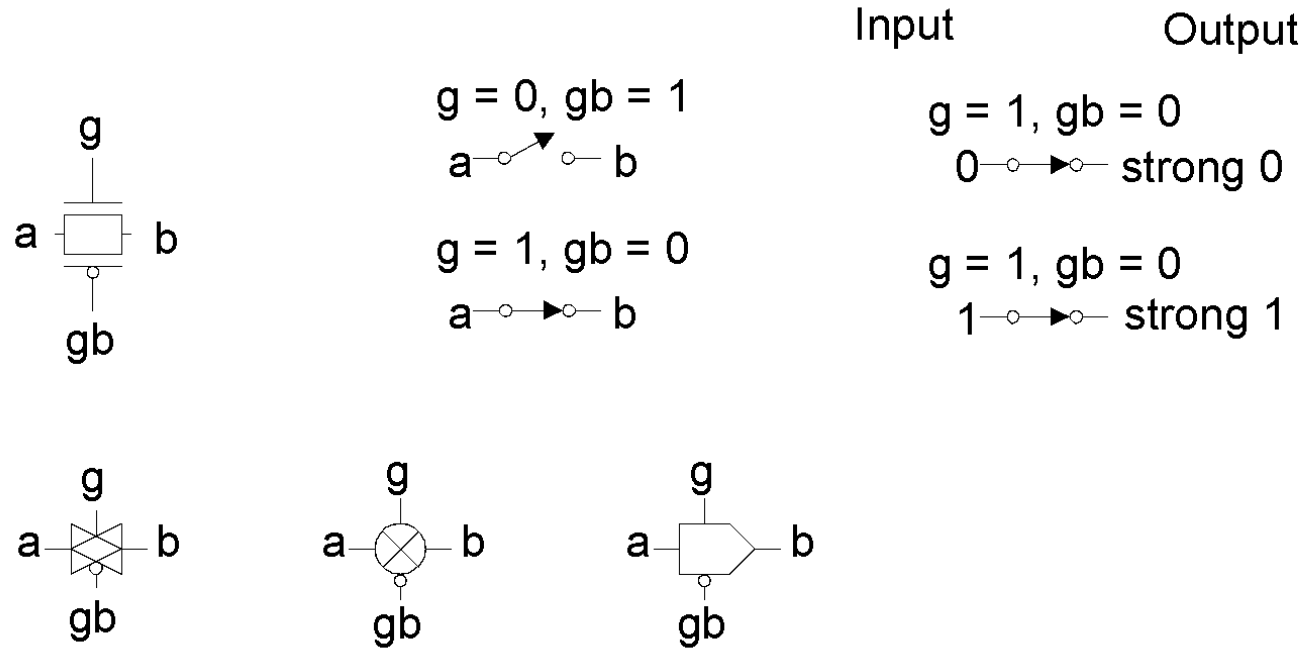
Pass Transistors

- Transistors can be used as switches



Transmission Gates

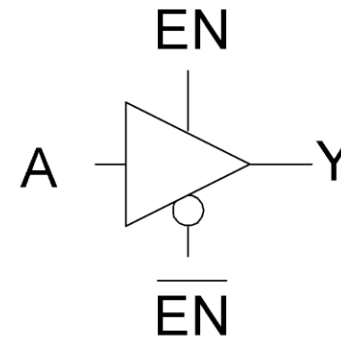
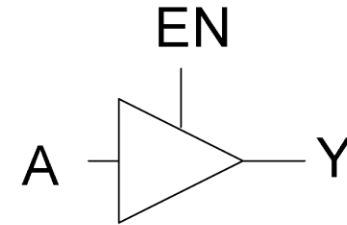
- ❑ Pass transistors produce degraded outputs
- ❑ *Transmission gates* pass both 0 and 1 well



Tristates

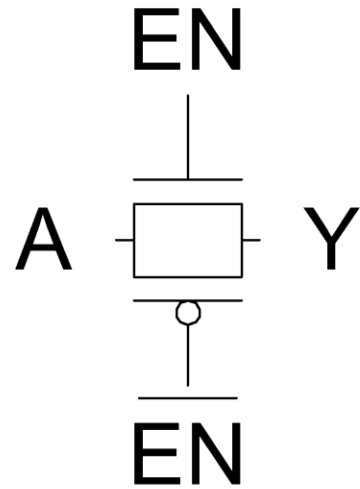
- ❑ *Tristate buffer* produces Z when not enabled

EN	A	Y
0	0	
0	1	
1	0	
1	1	



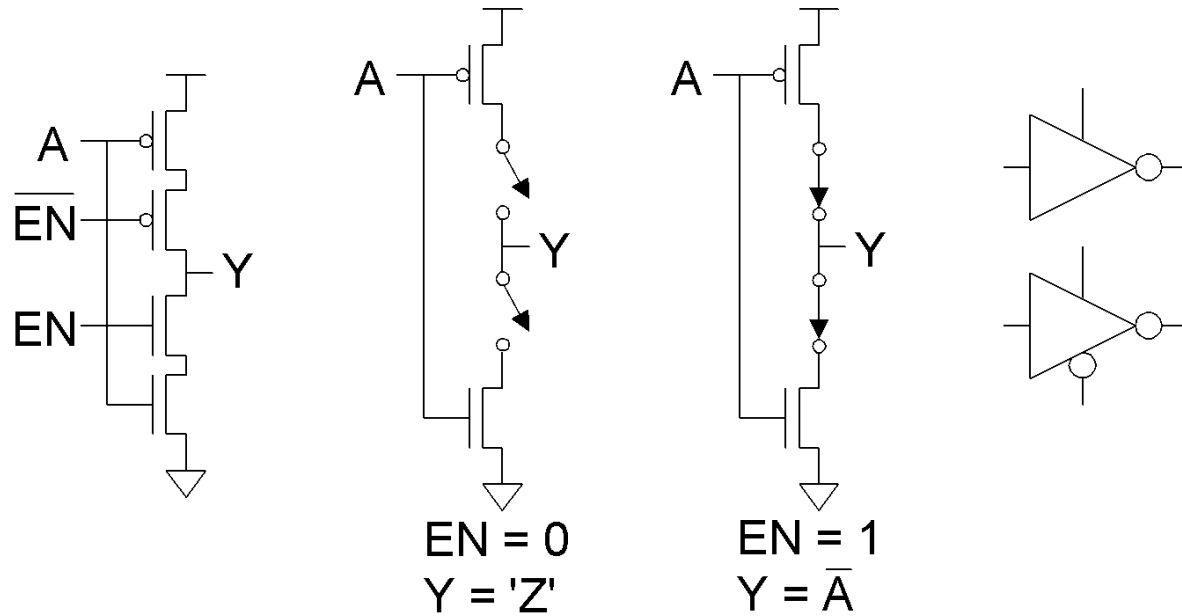
Nonrestoring Tristate

- ❑ Transmission gate acts as tristate buffer
 - Only two transistors
 - But *nonrestoring*
 - Noise on A is passed on to Y



Tristate Inverter

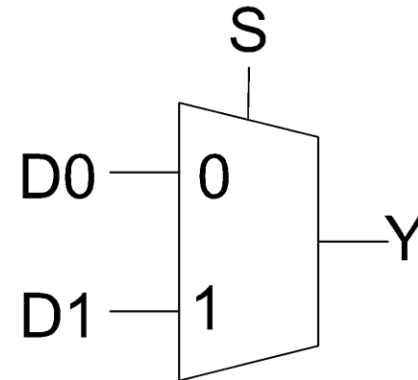
- ❑ Tristate inverter produces restored output
 - Violates conduction complement rule
 - Because we want a Z output



Multiplexers

- ❑ 2:1 multiplexer chooses between two inputs

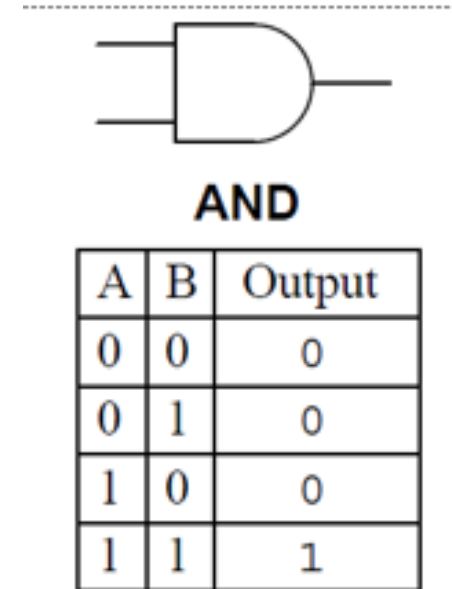
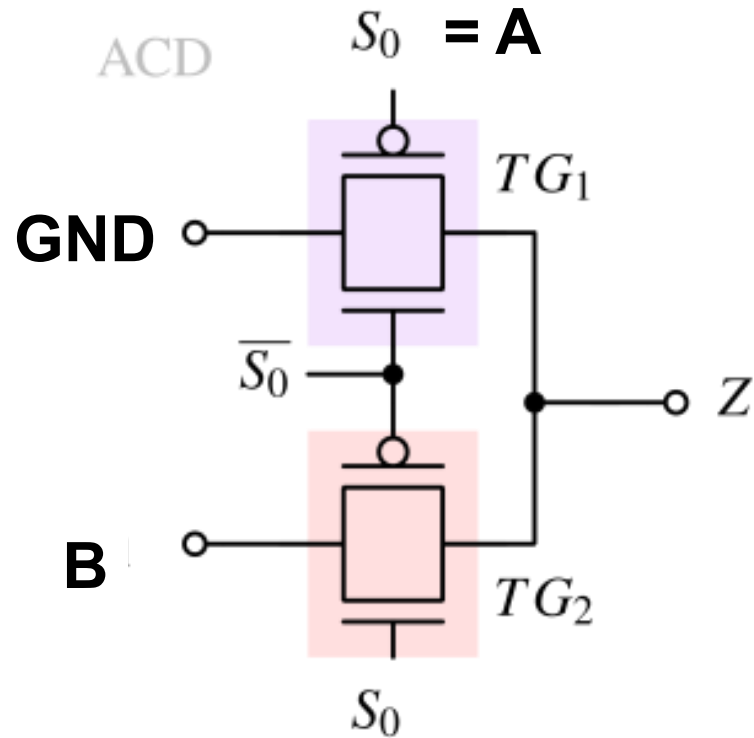
S	D1	D0	Y
0	X	0	
0	X	1	
1	0	X	
1	1	X	



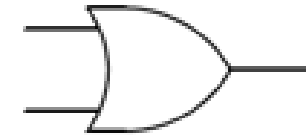
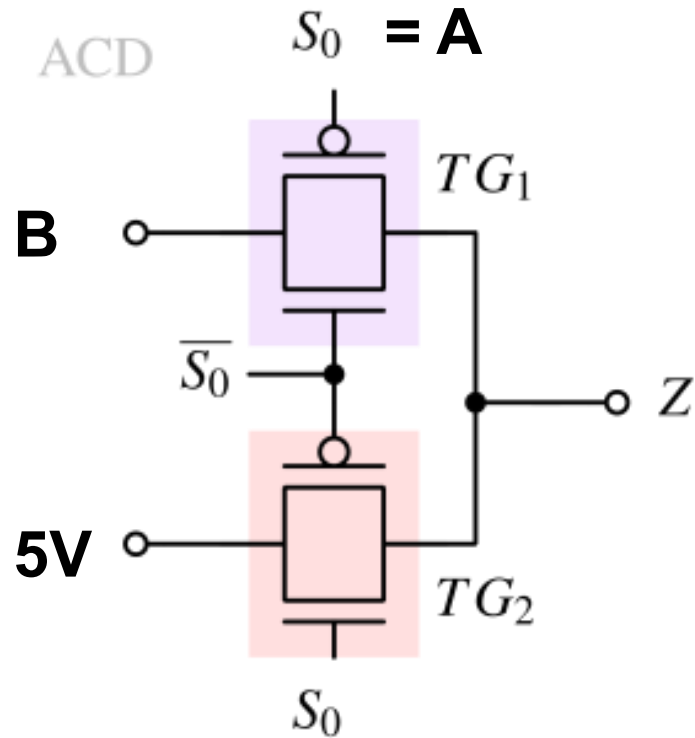
Gate-Level Mux Design

- ❑ $Y = SD_1 + \bar{S}D_0$ (too many transistors)
- ❑ How many transistors are needed?

Logic Gate using TG



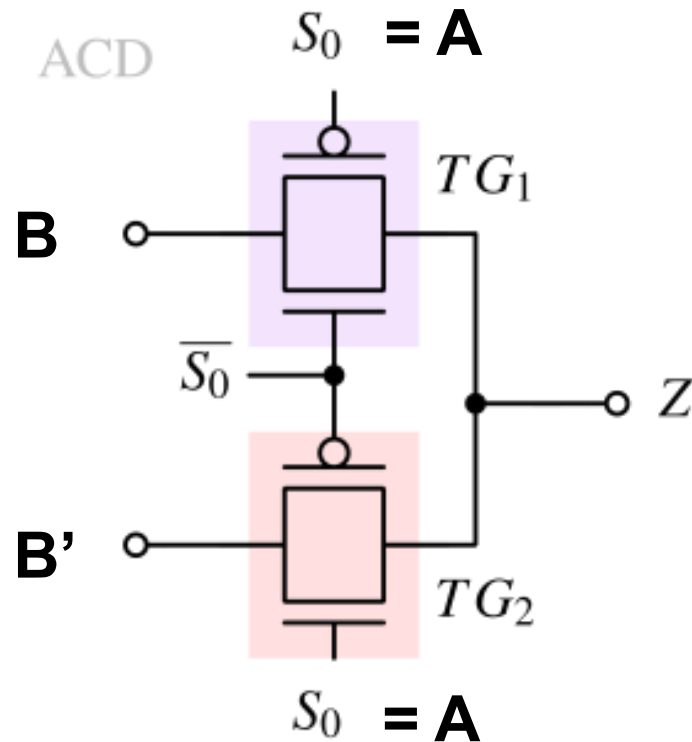
Logic Gate using TG



OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

Logic Gate using TG

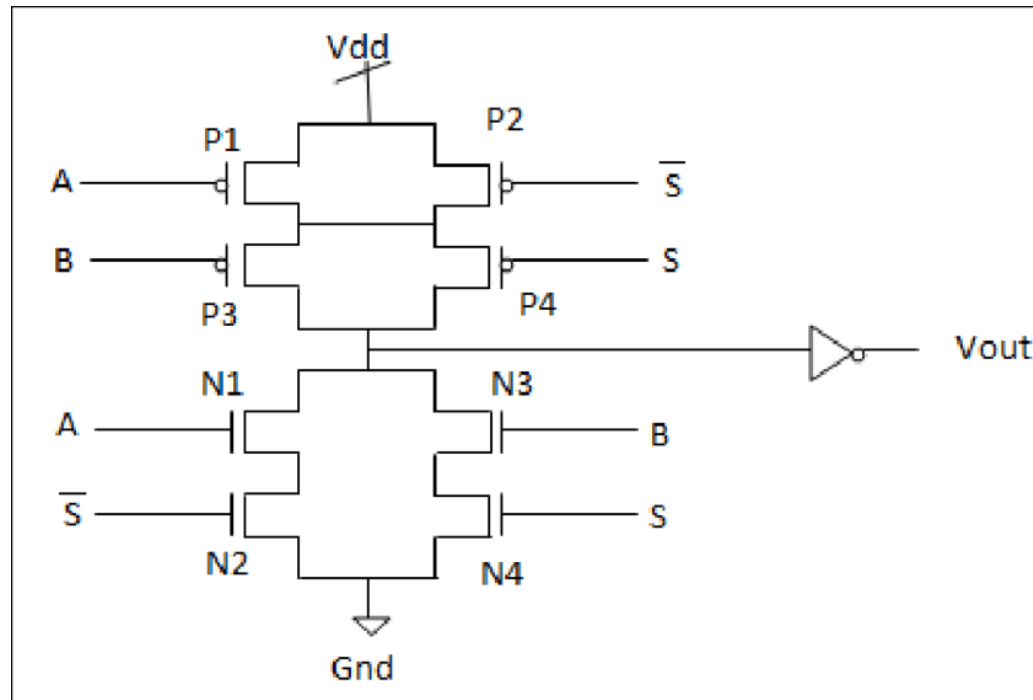
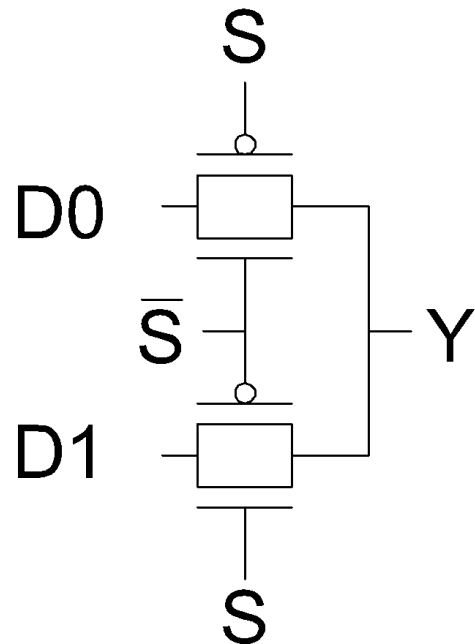


XOR

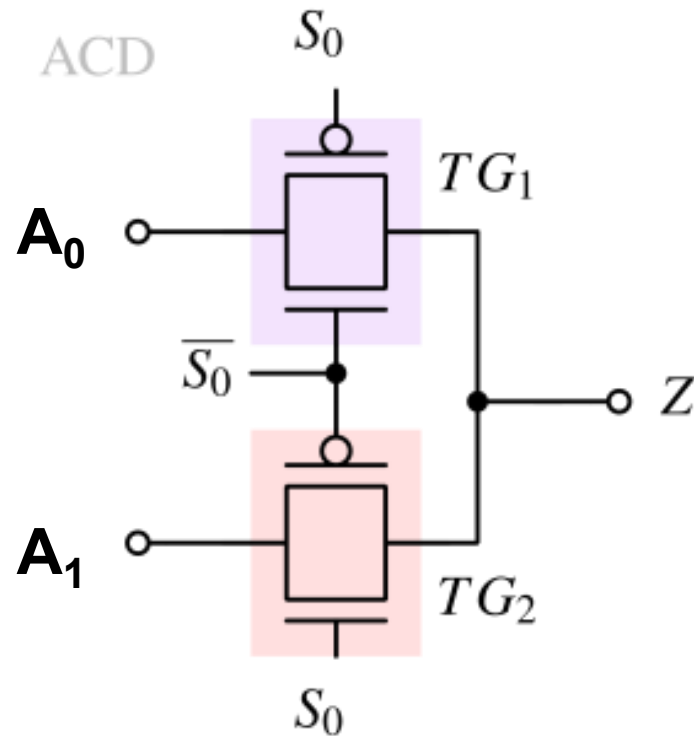
A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Transmission Gate Mux

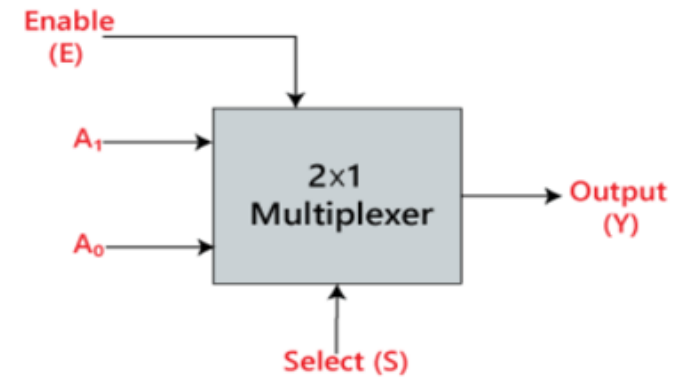
- ❑ Nonrestoring mux uses two transmission gates
 - Only 4 transistors



2:1 MUX using TG



Block Diagram:

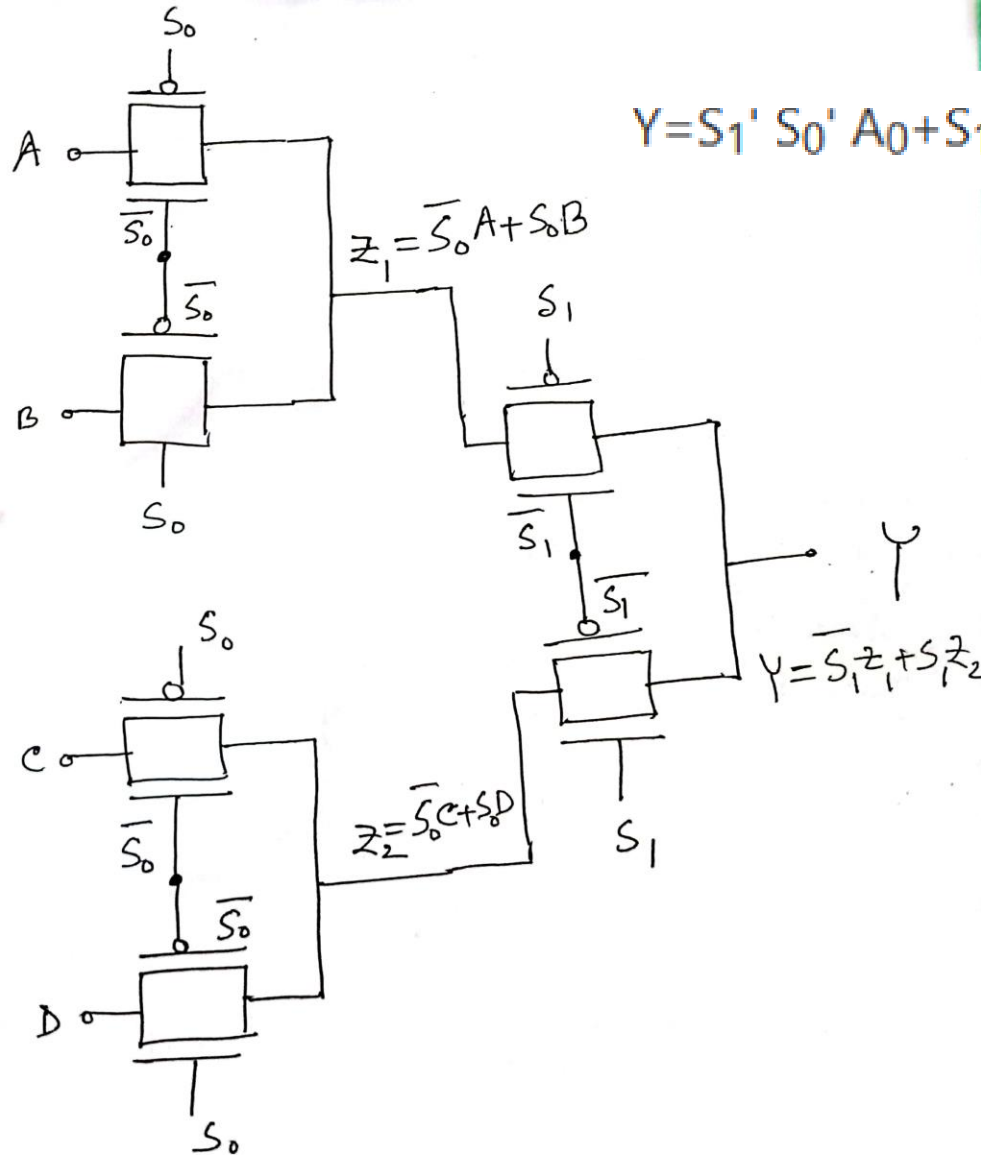


INPUTS	Output
S_0	Y
0	A_0
1	A_1

The logical expression of the term Y is as follows:

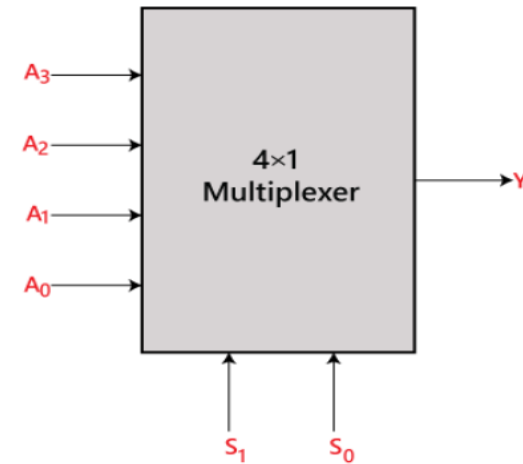
$$Y = S_0' \cdot A_0 + S_0 \cdot A_1$$

4:1 MUX using TG



$$Y = S_1' S_0' A_0 + S_1' S_0 A_1 + S_1 S_0' A_2 + S_1 S_0 A_3$$

Block Diagram:

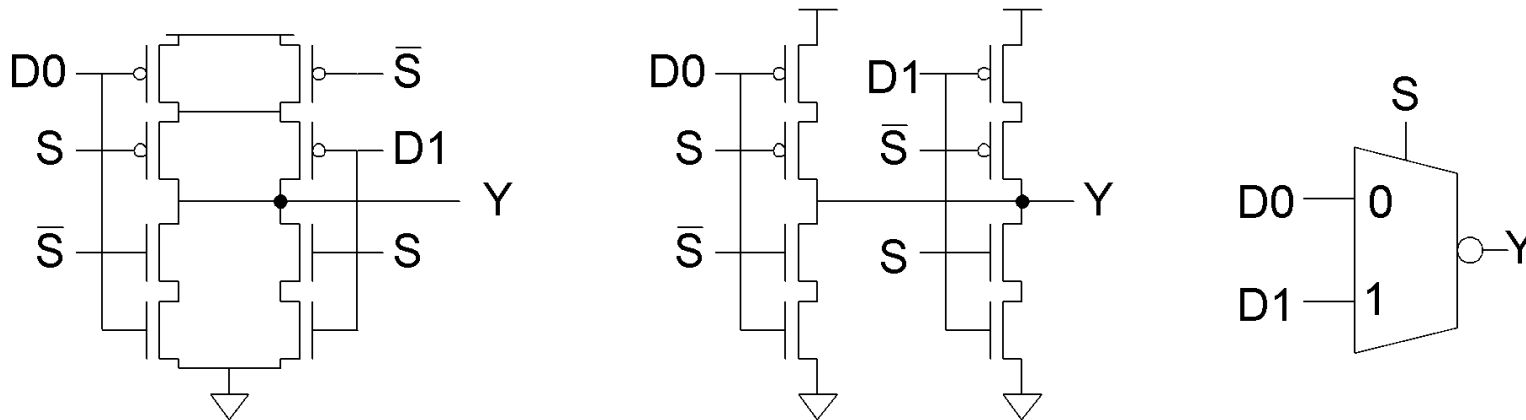


Truth Table:

INPUTS		Output
S_1	S_0	Y
0	0	A_0
0	1	A_1
1	0	A_2
1	1	A_3

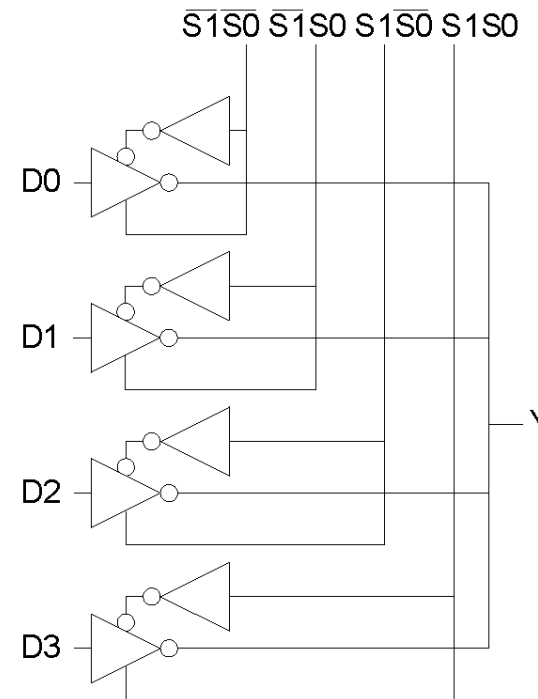
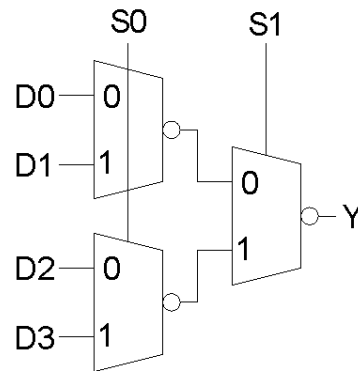
Inverting Mux

- ❑ Inverting multiplexer
 - Use compound AOI22
 - Or pair of tristate inverters
 - Essentially the same thing
- ❑ Noninverting multiplexer adds an inverter



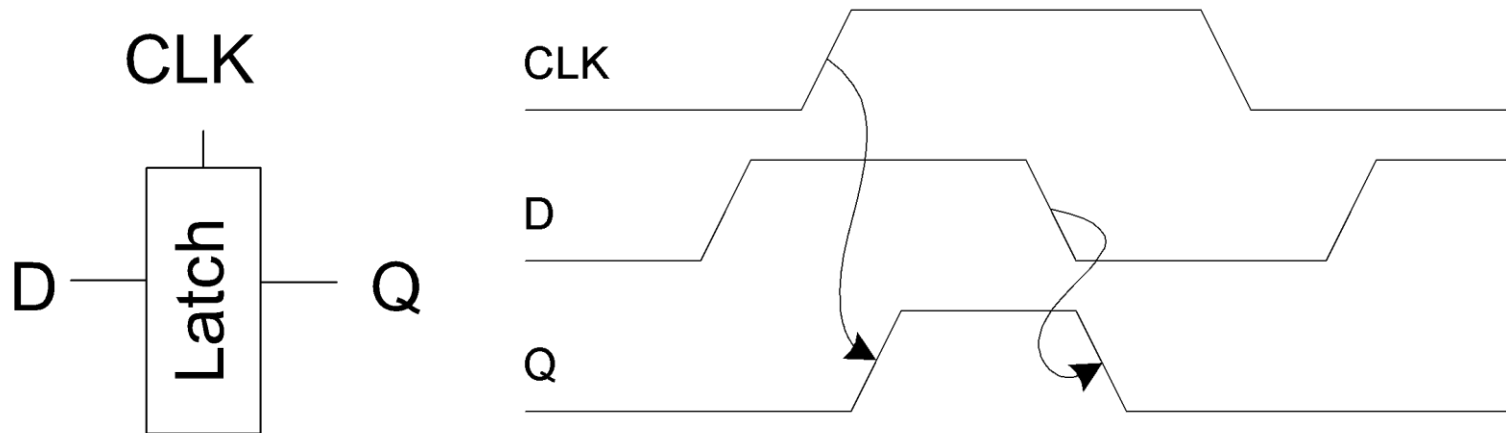
4:1 Multiplexer

- ❑ 4:1 mux chooses one of 4 inputs using two selects
 - Two levels of 2:1 muxes
 - Or four tristates



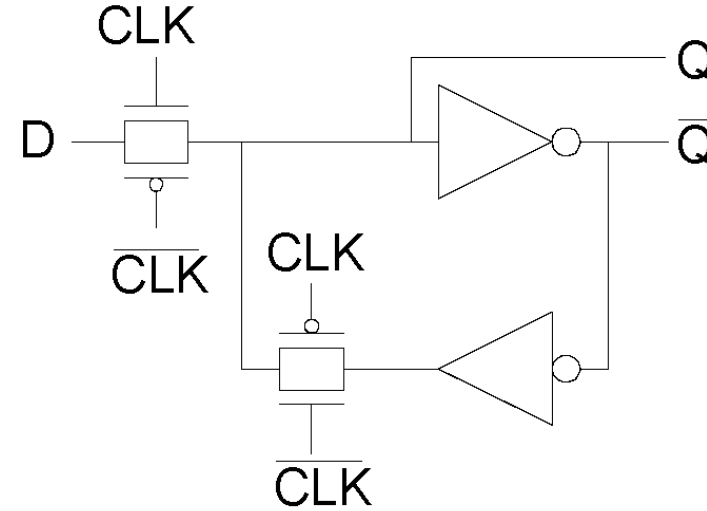
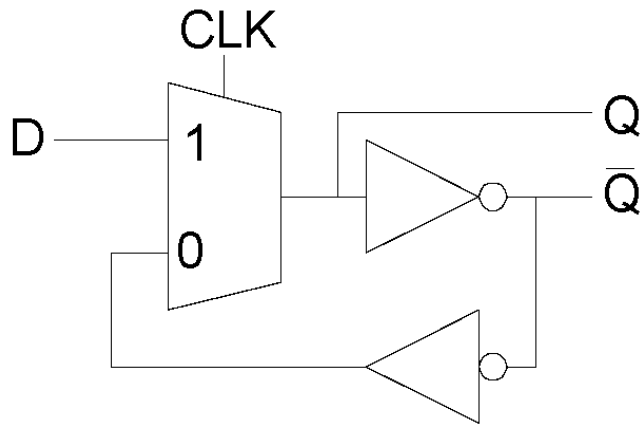
D Latch

- ❑ When $CLK = 1$, latch is *transparent*
 - D flows through to Q like a buffer
- ❑ When $CLK = 0$, the latch is *opaque*
 - Q holds its old value independent of D
- ❑ a.k.a. *transparent latch* or *level-sensitive latch*

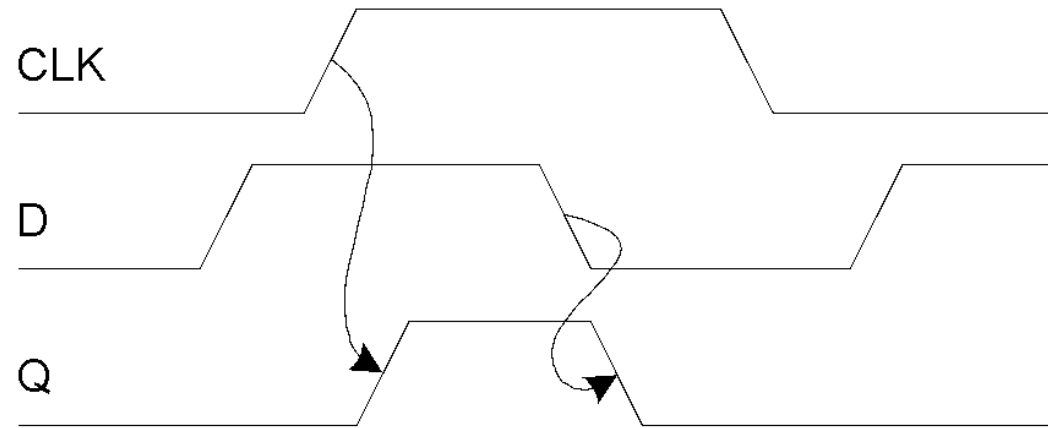
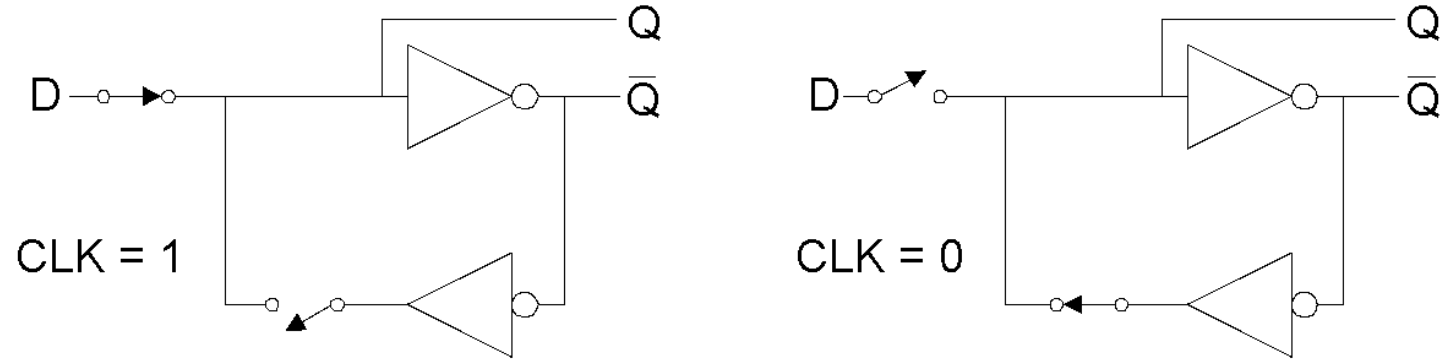


D Latch Design

- ❑ Multiplexer chooses D or old Q

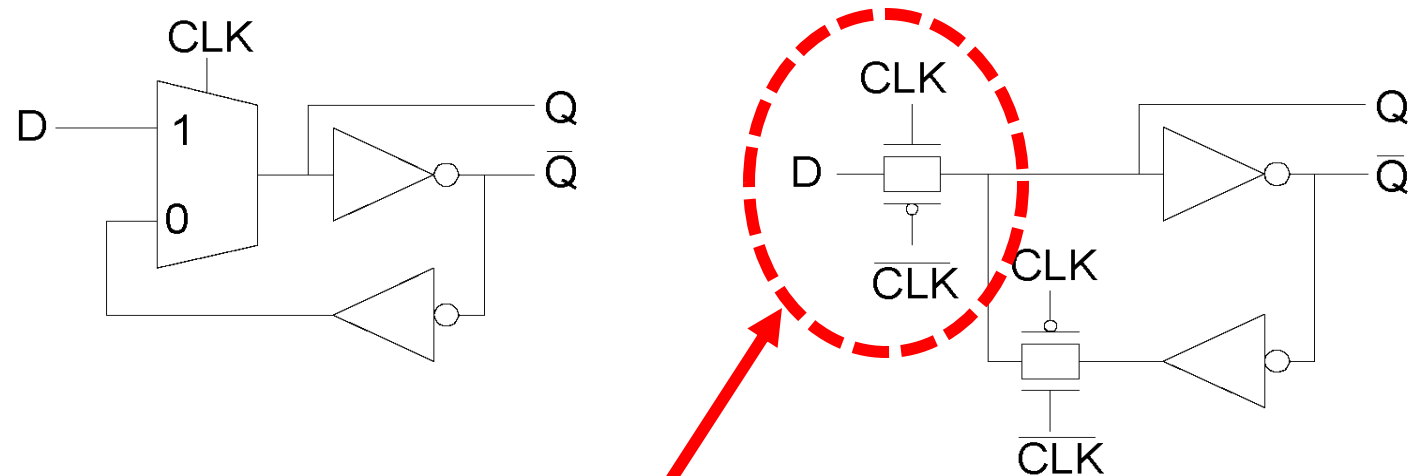


D Latch Operation



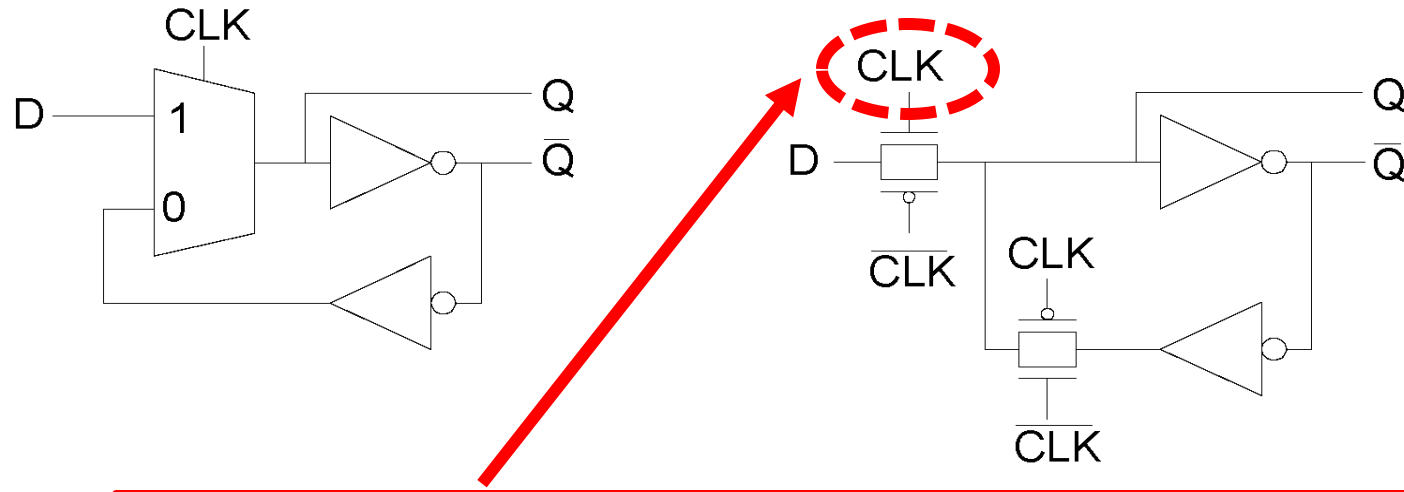
How to identify the Latch

Find out the TG which holds the input of the latch. Now look at the gate of the NMOS of that TG. If gate of the NMOS of that TG have CLK as it's input. Then the latch is a positive latch. Otherwise it's a negative latch.



This is the TG that holds the input D

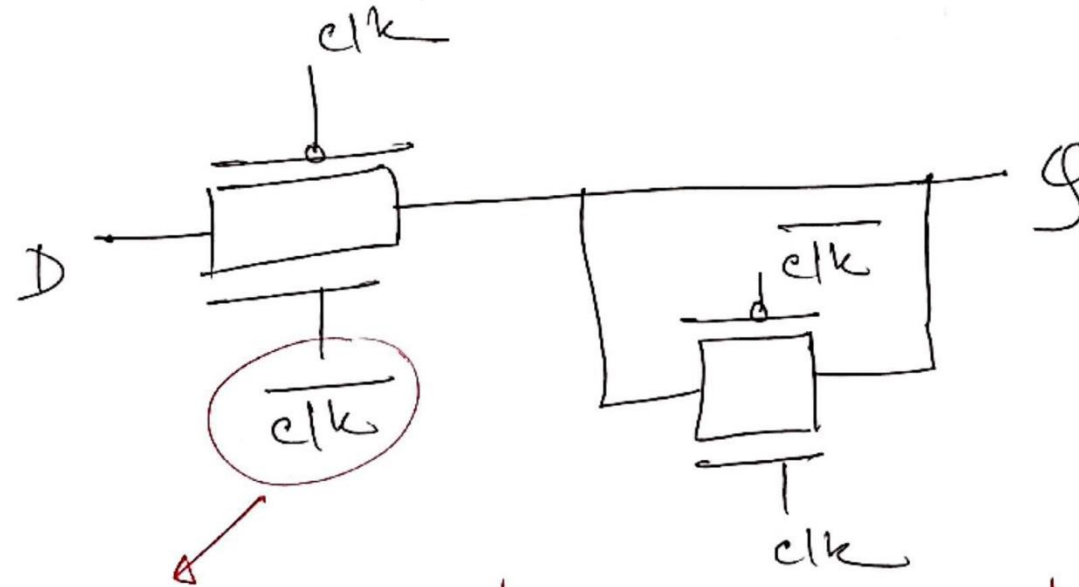
How to identify the Latch



Look at the input of the NMOS gate of the TG that holds input of the latch

As the gate of this NMOS have CLK, it's a positive latch

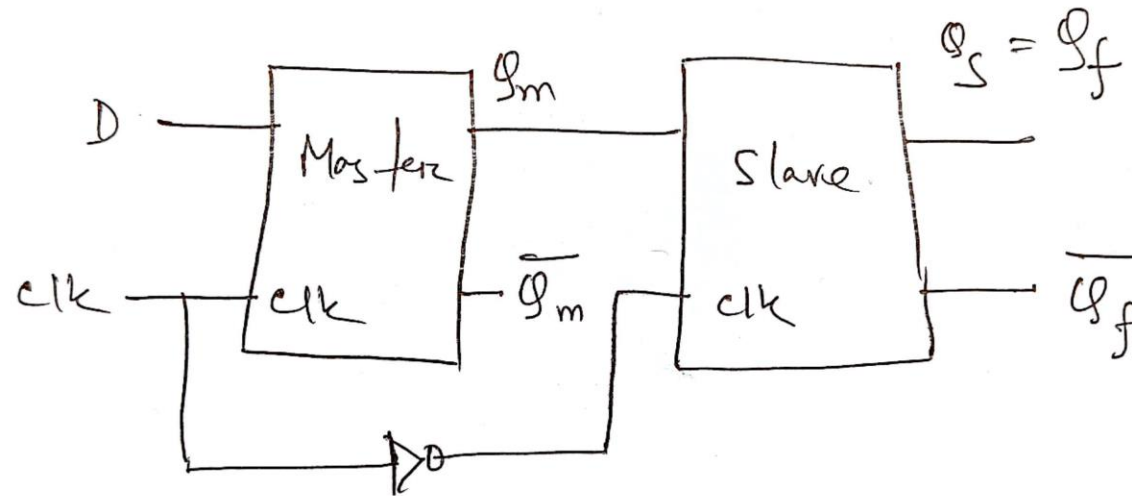
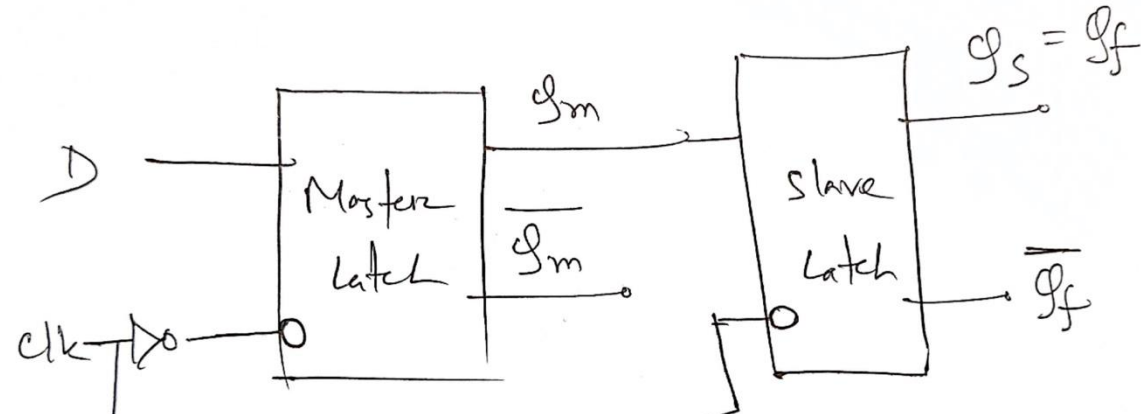
How to identify the Latch



This clock is ~~negative~~ inverted. That's why,
it's a negative latch.

D Flip-flop Operation

Negative Edge Triggered D-FF:



D Flip-flop Operation

Negative Edge Triggered D-FF:

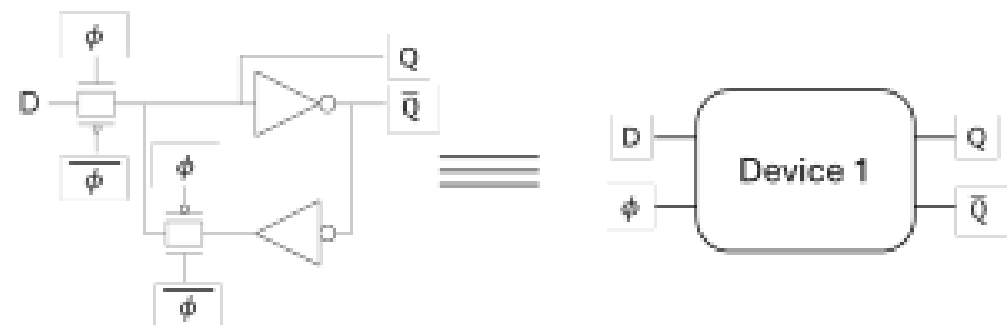
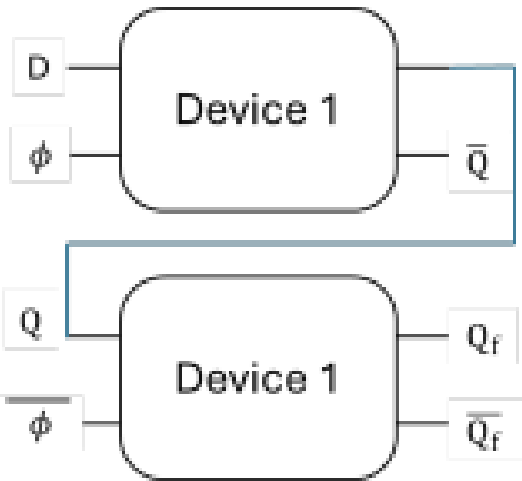


Figure 1: Circuit on the left is represented by the block - Device 1



(d) $Q=D \rightarrow$ Positive level triggered Latch

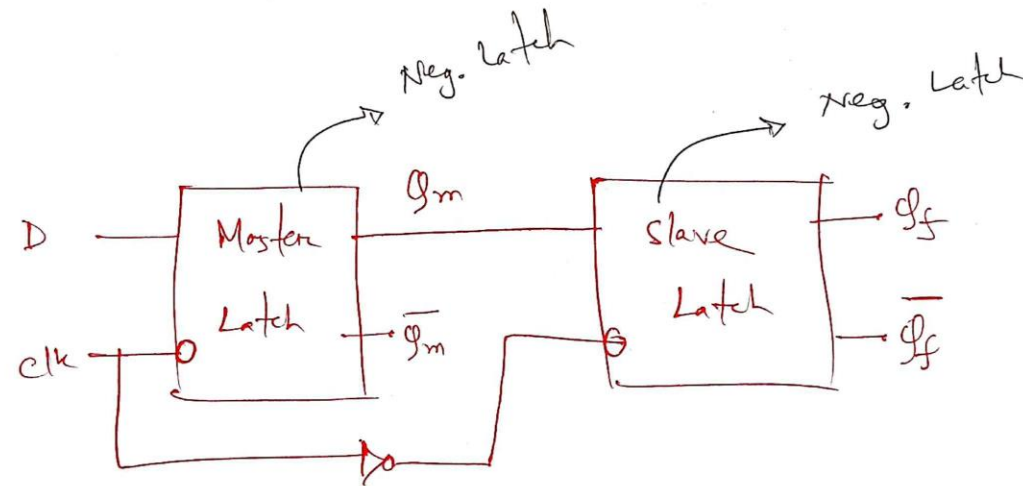
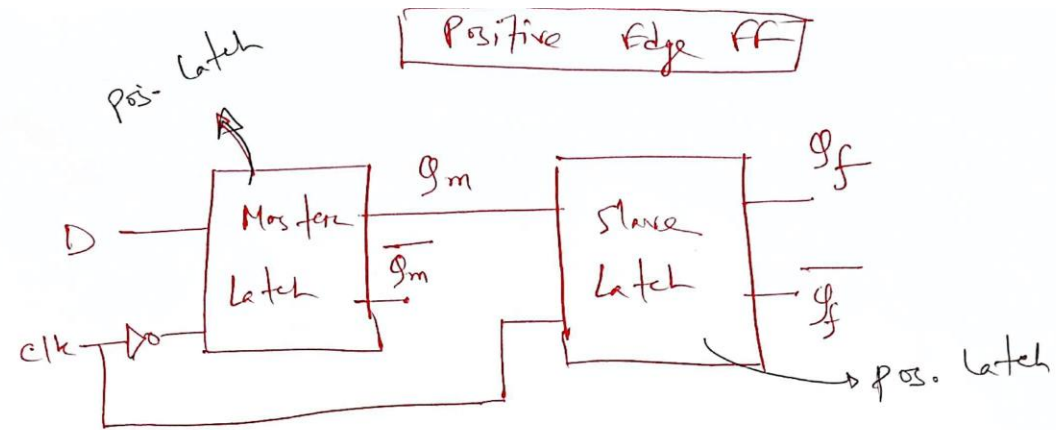
(e)

Node ↓ / Time →	5 ns	10 ns	15 ns	20 ns	25 ns
ϕ	0	1	0	1	0
D	0	1	1	1	0
Q	X/1	1	1	1	1
Q_f	1	1	1	1	1

Negative-edge-triggered flip-flop.

Figure 2: A circuit made up of two Device 1 blocks from Fig. 1

Positive Edge Triggered D-FF:



Positive Edge Triggered D-FF:

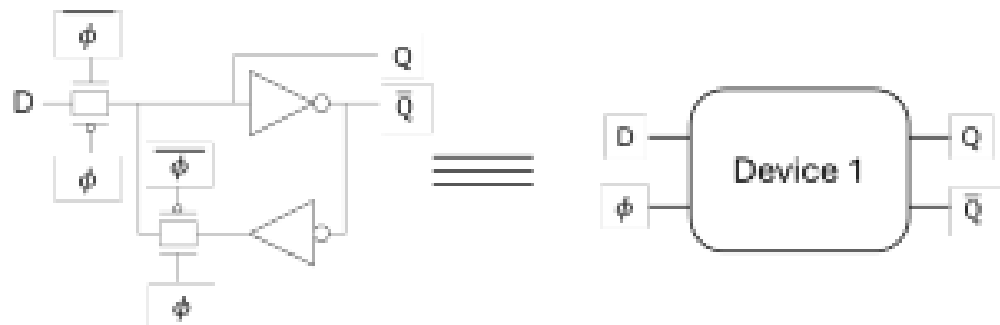
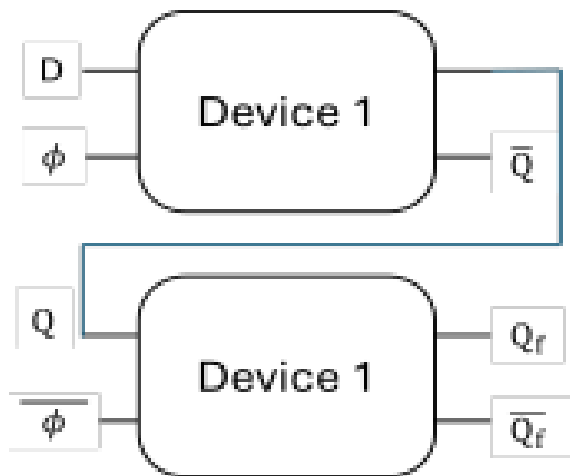


Figure 1: Circuit on the left is represented by the block - **Device 1**



(d)

$Q=D \rightarrow$ *Negative level triggered Latch*

(e)

Node ↓ / Time →	5 ns	10 ns	15 ns	20 ns	25 ns
ϕ	0	1	0	1	0
D	0	1	1	1	0
Q	0	0	1	1	0
Q_f	1	0	0	1	1

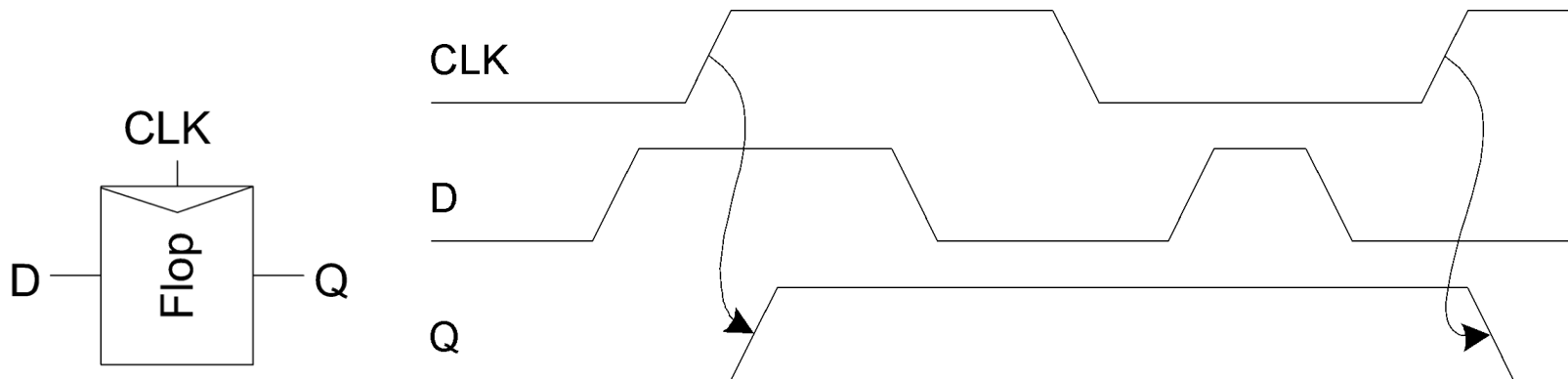
Positive-edge triggered flip-flop.

Positive-edge triggered flip-flop.

Figure 2: A circuit made up of two **Device 1** blocks from Fig. 1

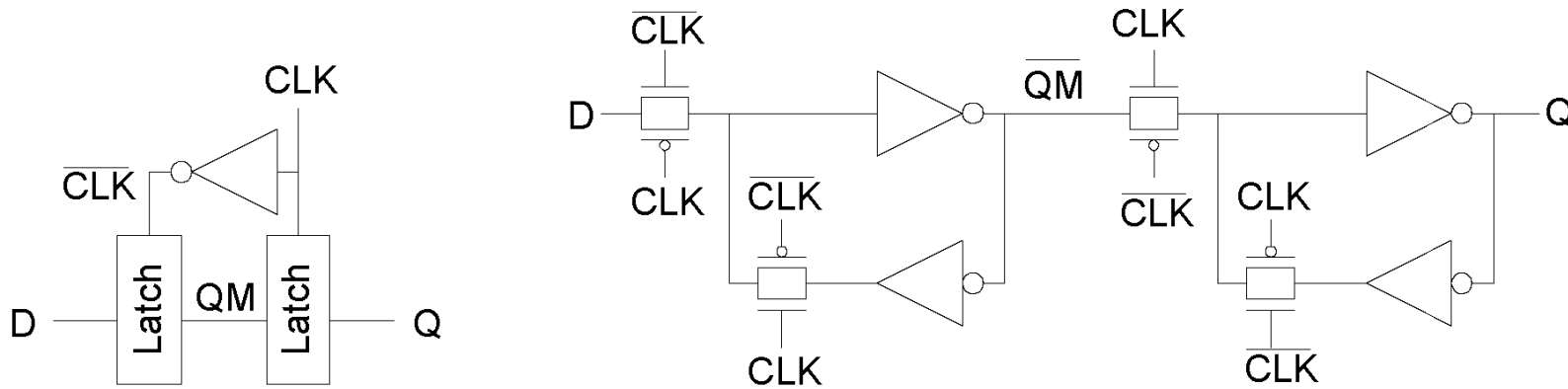
D Flip-flop

- ❑ When CLK rises, D is copied to Q
- ❑ At all other times, Q holds its value
- ❑ a.k.a. *positive edge-triggered flip-flop, master-slave flip-flop*

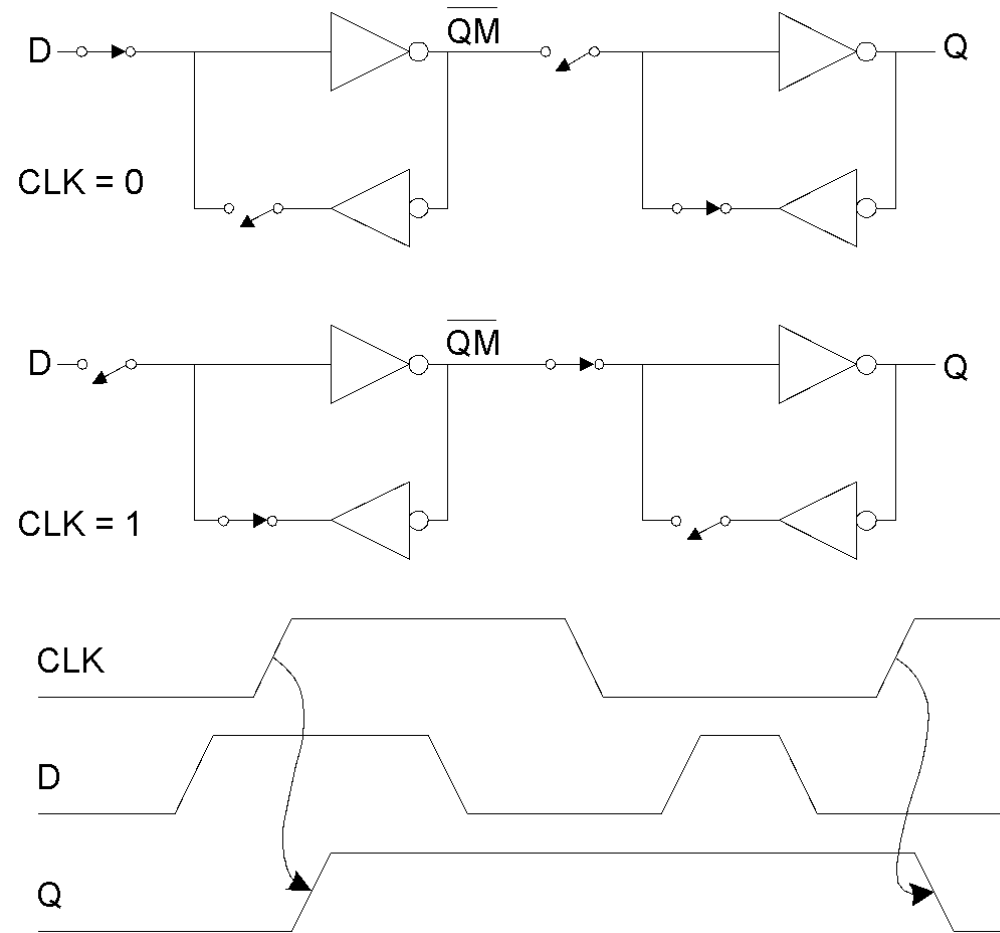


D Flip-flop Design

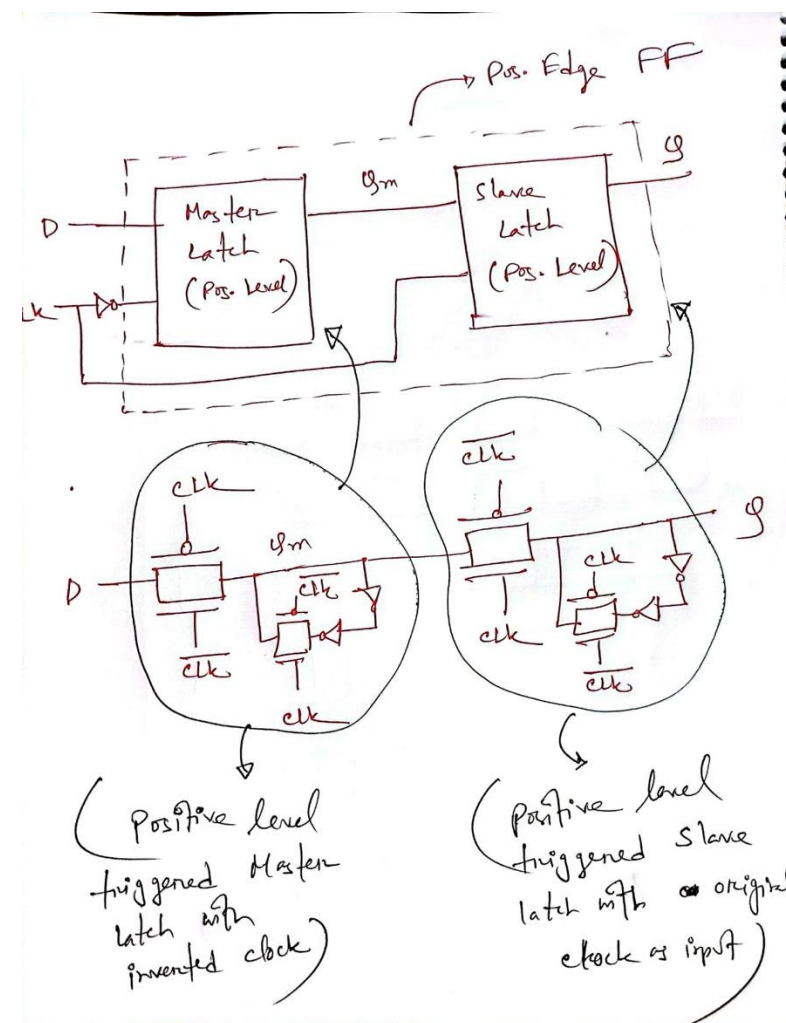
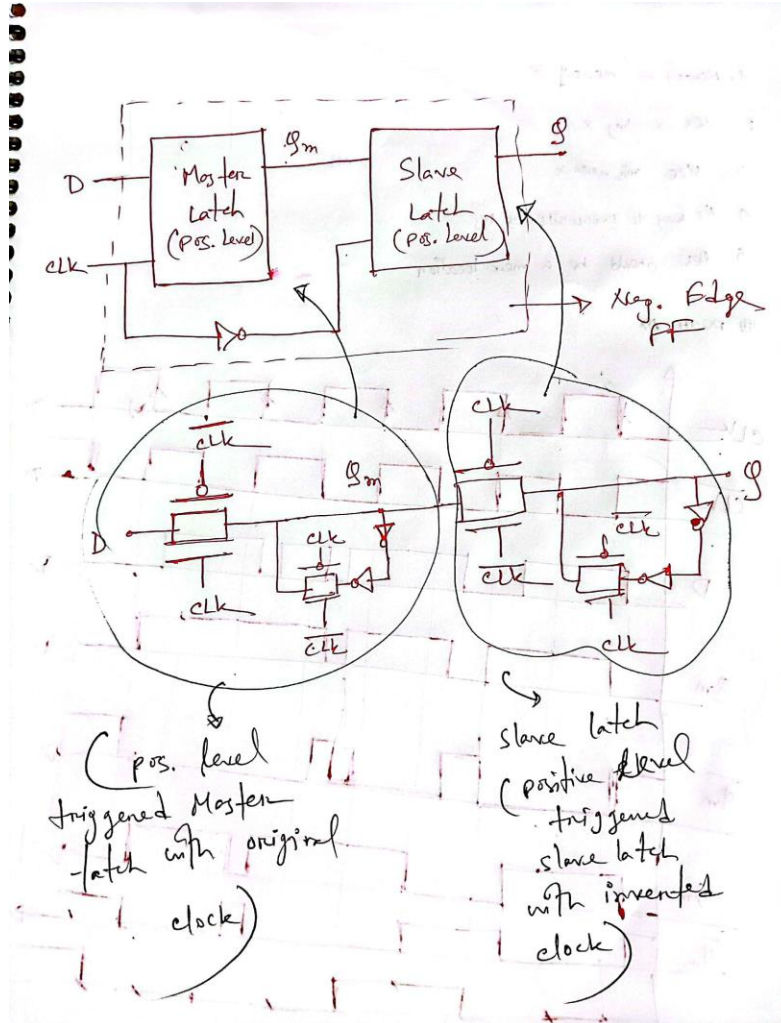
- Built from master and slave D latches



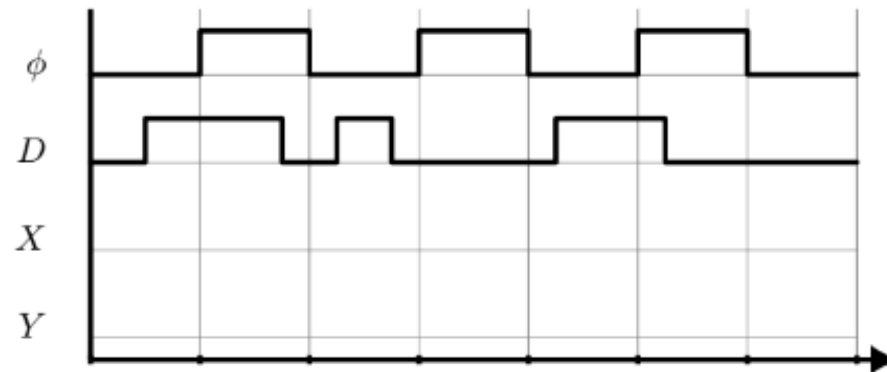
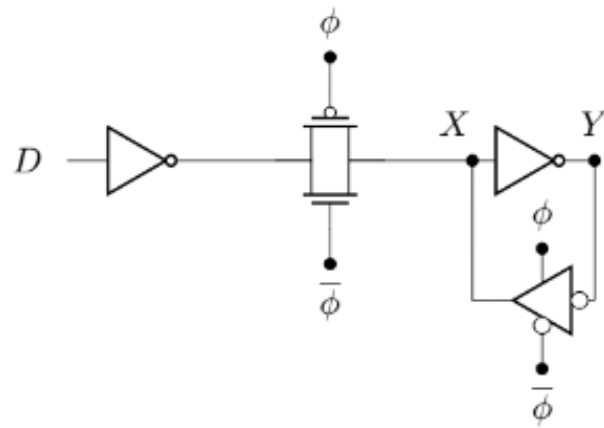
D Flip-flop Operation

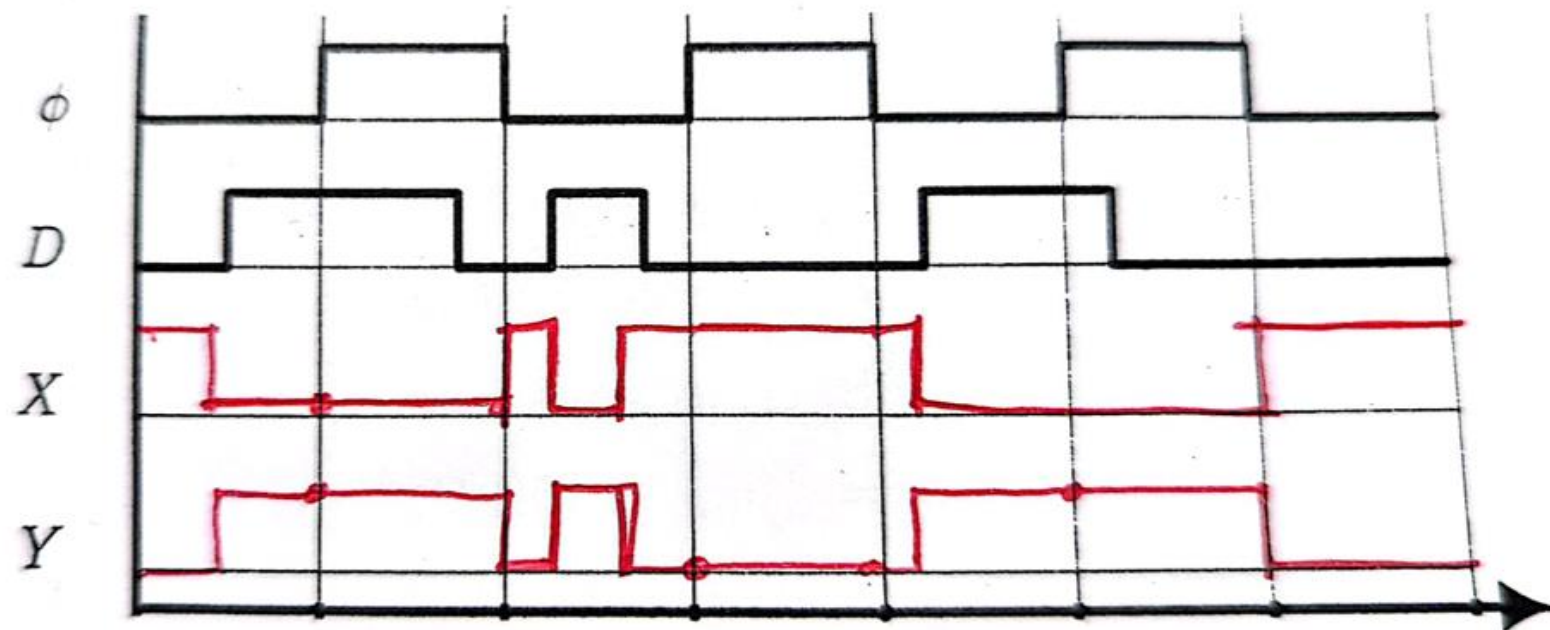


D Flip-flop Operation



[3 marks] **Complete** the timing diagram shown below on the right, in the question paper, based on the circuit diagram presented below on the left. Ensure that your completed timing diagram accurately reflects the logic behavior of the circuit, taking into account the states of ϕ and D to determine the corresponding outputs X and Y .





Pass Transistor DC Characteristics

Recall from our previous lectures that

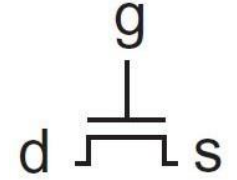
- nMOS transistors pass '0's well but '1's poorly
- pMOS transistors pass '1's well but '0's poorly

We are now better prepared to define how “poorly”

nMOS Pass Transistor DC Characteristics

To keep an nMOS transistor ON, we need

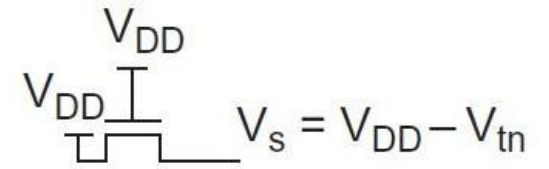
- $V_{gs} \geq V_{tn}$
- $\Rightarrow V_g - V_s \geq V_{tn}$
- $\Rightarrow V_s \leq V_g - V_{tn}$
- $\Rightarrow V_{s(max)} = V_g - V_{tn}$



nMOS Pass Transistor DC Characteristics

To keep an nMOS transistor ON, we need

- $V_{s(\max)} = V_g - V_{tn}$



nMOS transistors passing '1' ($V_{in} = V_{DD}$; $V_{out} = ?$)

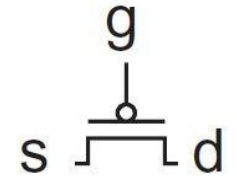
- Gate is connected to V_{DD} : $V_g = V_{DD}$
- Current flowing from drain to source
- $V_{in} = V_d = V_{DD}$; $V_{out} = V_s = ?$
- But we know $V_{s(\max)} = V_g - V_{tn} \Rightarrow V_{out(\max)} = V_{DD} - V_{tn}$

nMOS transistors attempting to pass a '1' never pass above $V_{DD} - V_{tn}$

pMOS Pass Transistor DC Characteristics

To keep a pMOS transistor ON, we need

- $V_{sg} \geq |V_{tp}|$
- $\Rightarrow V_s - V_g \geq |V_{tp}|$
- $\Rightarrow V_s \geq V_g + |V_{tp}|$
- $\Rightarrow V_{s(\min)} = V_g + |V_{tp}|$



pMOS Pass Transistor DC Characteristics

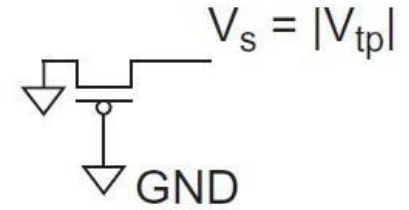
To keep a pMOS transistor ON, we need

- $V_{s(\min)} = V_g + |V_{tp}|$

pMOS transistors passing '0' ($V_{in} = 0$; $V_{out} = ?$)

- Gate is connected to GND: $V_g = 0$
- Current flowing from source to drain
- $V_{in} = V_d = 0$; $V_{out} = V_s = ?$
- But we know $V_{s(\min)} = V_g + |V_{tp}| \Rightarrow V_{out(\min)} = |V_{tp}|$

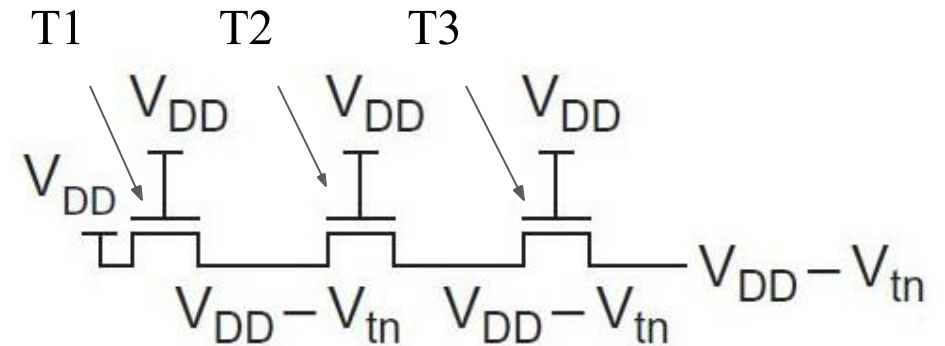
pMOS transistors attempting to pass a '0' never pass lower than $|V_{tp}|$



nMOS Pass Transistor DC Characteristics

Series nMOS transistors passing '1' (V_{DD})

- If all of the transistors have gates tied to V_{DD}
- $V_{g(T1)} = V_{g(T2)} = V_{g(T3)} = V_{DD}$
- $V_{in(T1)} = V_{DD}$; $V_{out(T3)} = ?$



1. Output of Transistor T1: $V_{out(max), T1} = V_{g(T1)} - V_{tn(T1)} = V_{DD} - V_{tn}$
2. Output of Transistor T2: $V_{out(max), T2} = V_{g(T2)} - V_{tn(T2)} = V_{DD} - V_{tn}$
3. Output of Transistor T3: $V_{out(max), T3} = V_{g(T3)} - V_{tn(T3)} = V_{DD} - V_{tn}$

nMOS Pass Transistor DC Characteristics

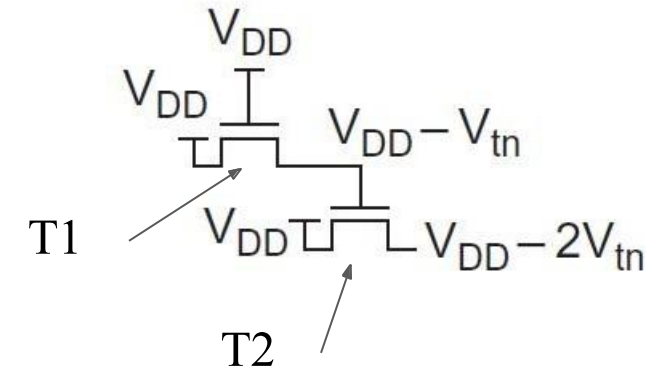
nMOS transistor driven by degraded output

- If T1 transistor has gate tied to V_{DD}
- $V_{g(T1)} = V_{DD}$; $V_{in} = V_{DD}$; $V_{out} = ?$

Output of Transistor T1: $V_{out(max), T1} = V_{g(T1)} - V_{tn(T1)} = V_{DD} - V_{tn}$

- Output of transistor T1 is now driving the gate of transistor T2
- T2 transistor has gate tied to $V_{DD} - V_{tn}$
- $V_{g(T2)} = V_{DD} - V_{tn}$; $V_{in} = V_{DD}$; $V_{out} = ?$

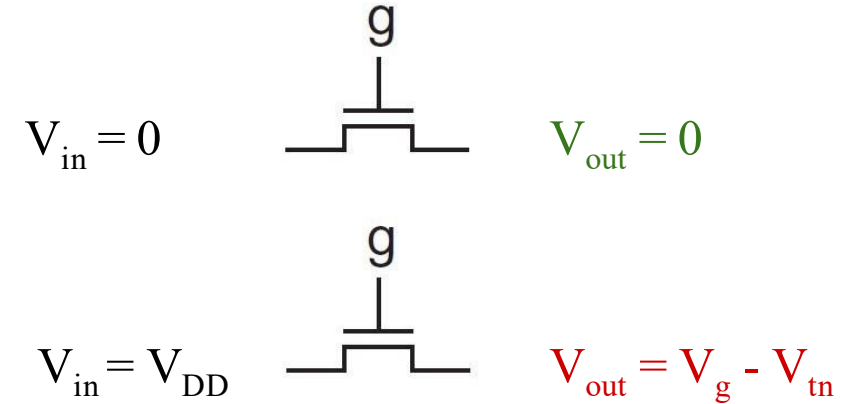
Output of Transistor T2: $V_{out(max), T2} = V_{g(T2)} - V_{tn(T2)} = (V_{DD} - V_{tn}) - V_{tn} = V_{DD} - 2V_{tn}$



Pass Transistor DC Characteristics: Summary

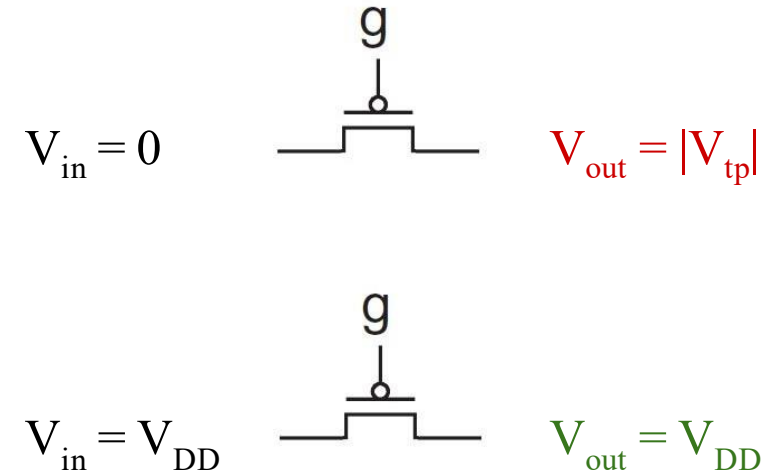
nMOS pass transistor

- $V_{in} = \text{GND} \Rightarrow V_{out} = \text{GND}$ (strong 0)
- $V_{in} = V_{DD} \Rightarrow V_{out(\max)} = V_g - V_{tn}$ (degraded 1)

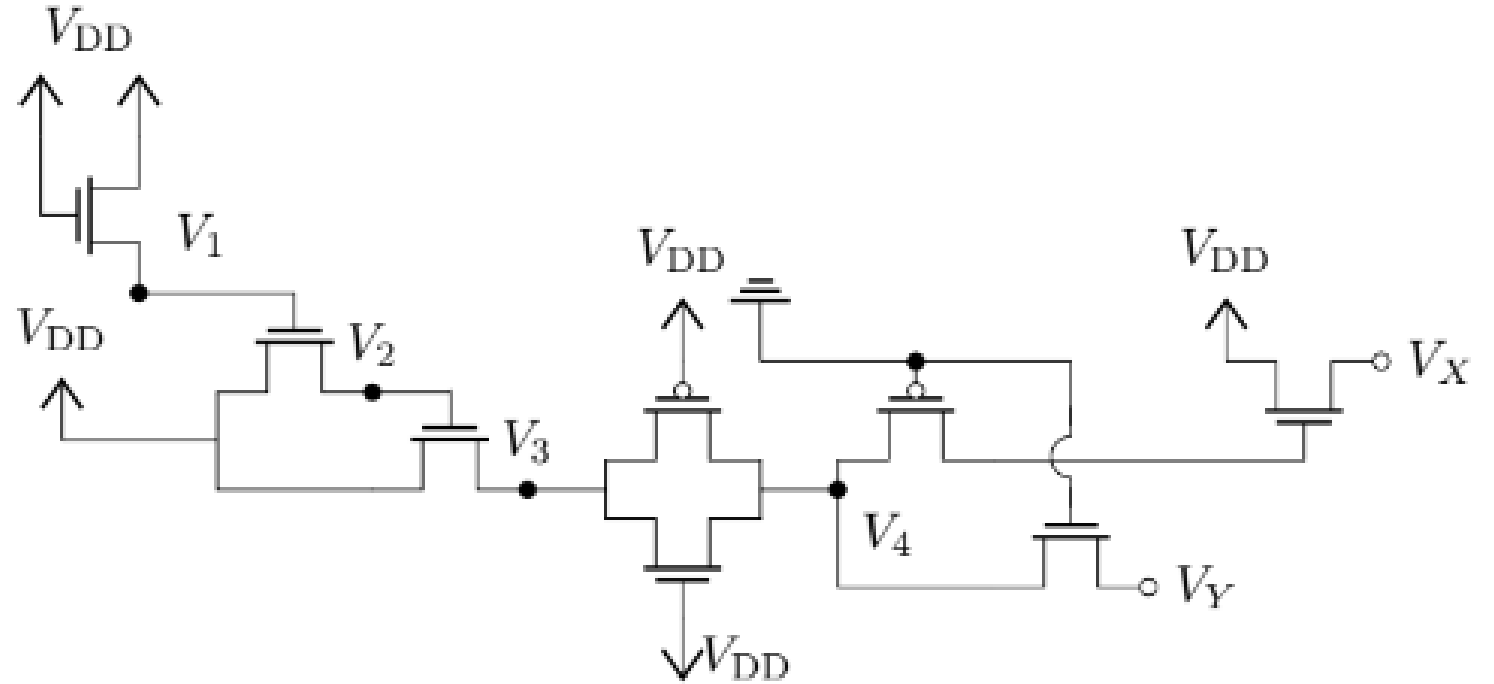


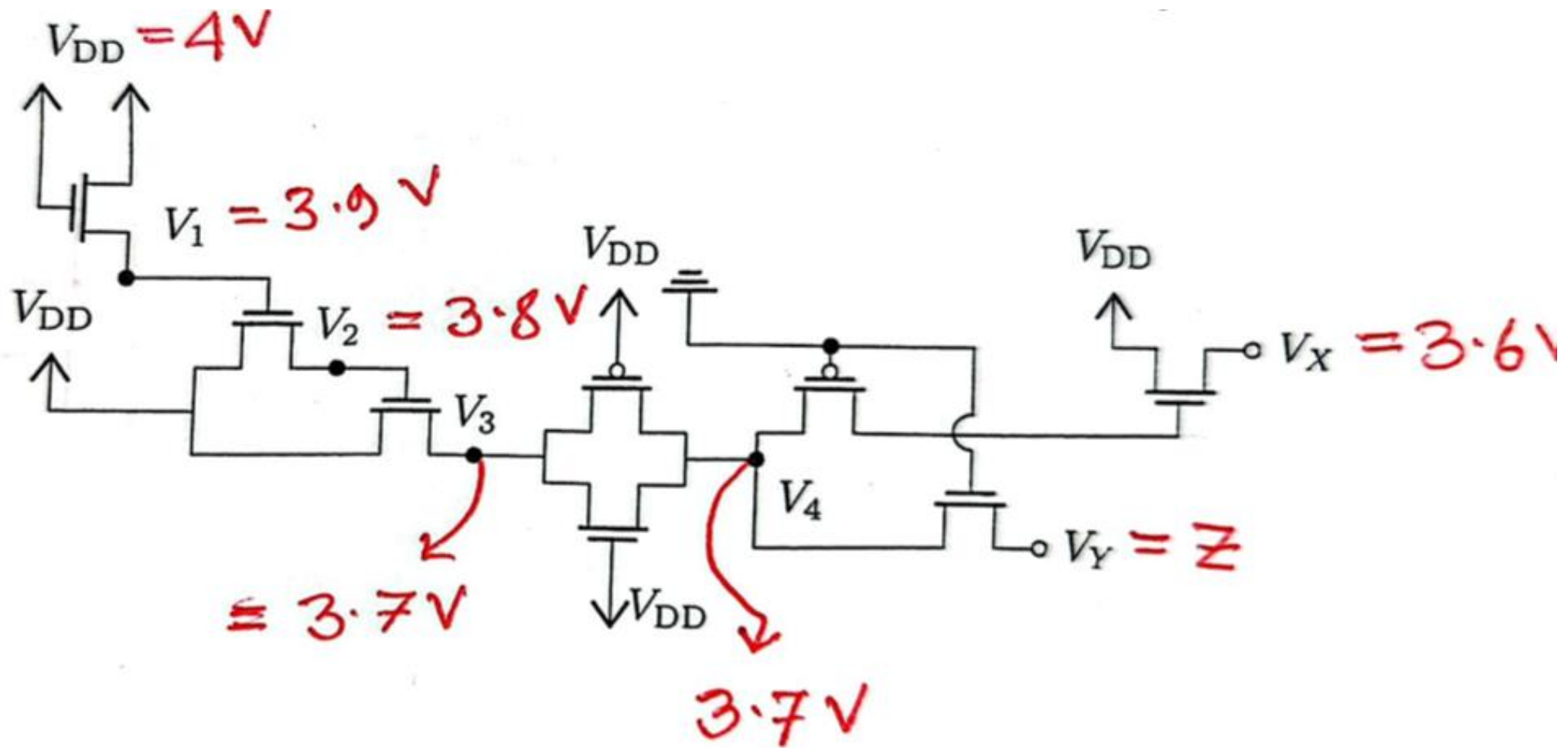
pMOS pass transistor

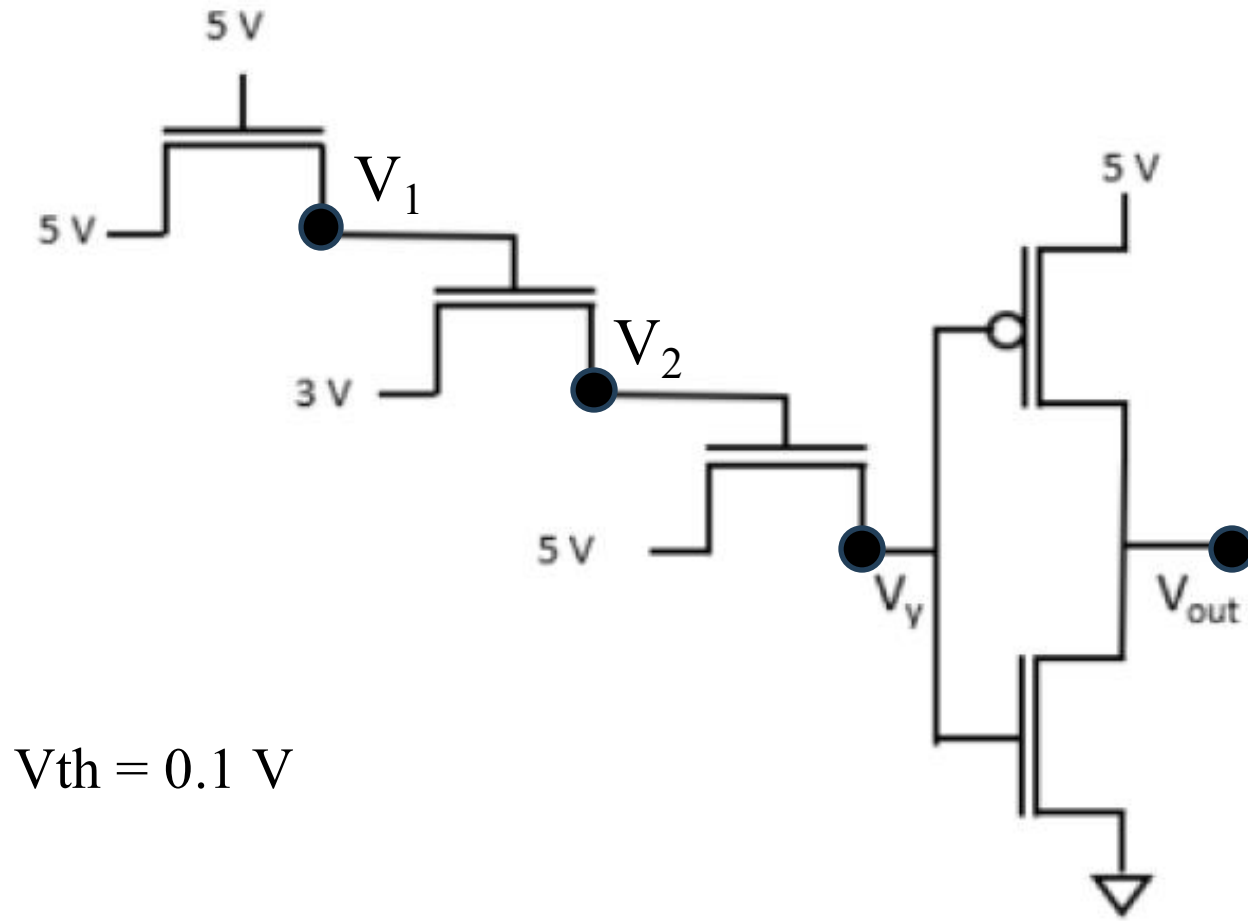
- $V_{in} = \text{GND} \Rightarrow V_{out(\min)} = |V_{tp}|$ (degraded 0)
- $V_{in} = V_{DD} \Rightarrow V_{out} = V_{DD}$ (strong 1)



[5 marks] In the adjacent figure, the supply voltage (V_{DD}) is 4 V. The threshold voltages for the pMOS and nMOS transistors are both set at $V_{tn} = |V_{tp}| = 0.1$ V. **Determine** the voltages V_1 , V_2 , and V_4 as well as the output voltages V_X and V_Y .







$V_{th} = 0.1 \text{ V}$

Figure 4: Circuit Diagram for Q.3

Solve:

$$V_1 = 4.9 \text{ V}$$

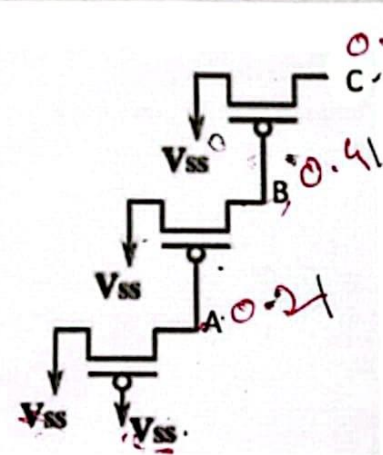
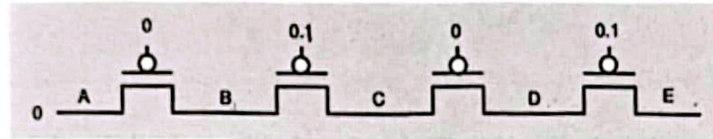
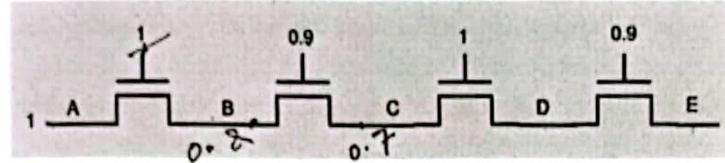
$$V_2 = 3 \text{ V}$$

$$V_Y = 2.9 \text{ V}$$

$$V_{out} = 0 \text{ V}$$

Question 2: [10 Marks]

1. From the following figures, find the voltages at each of the nodes, A, B, C, D, E. Assume $V_{dd} = 3\text{ V}$, $V_{ss} = 0\text{ V}$, $V_{tn} = 0.2\text{ V}$ and $|V_{tp}| = 0.2\text{ V}$.



$0.6 \text{ nMOS} \Rightarrow V_{GS} > V_{tn}$
 or, $V_{GS} - V_{tn} > V_{DS}$
 or, $V_{out} < V_{GS} - V_{tn}$
 $(V_{out})_{max} = V_{GS} - V_{tn}$
 $\text{PMOS} \Rightarrow V_{SG} \geq |V_{tp}|$
 or, $V_{out} - V_{GS} \geq |V_{tp}|$
 or, $V_{out} \geq V_{GS} + |V_{tp}|$

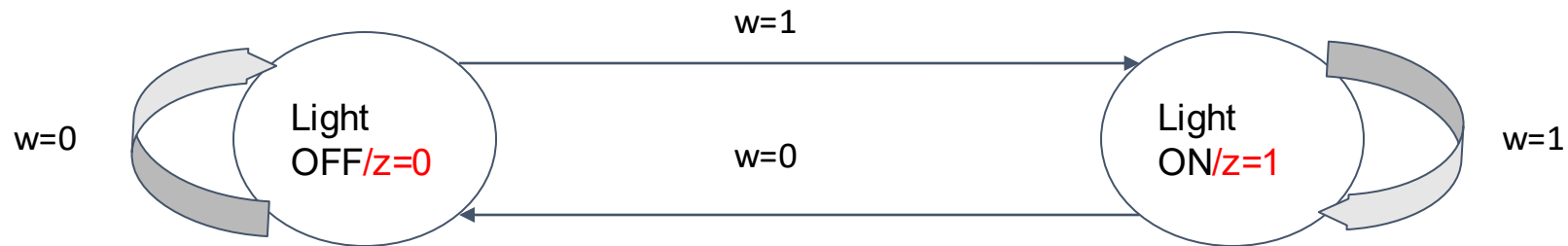
	V_A	V_B	V_C	V_D	V_E
Circuit 1	1V 1V	0.8V	0.7V	0.7V	0.7V
Circuit 2	0V	0.2V	0.3V	0.3V	0.3V
Circuit 3	0.2V	0.4V	0.6V		

CSE460: VLSI Design

Lecture 8: Finite State Machines (Part 1)

A very simple controller[a light switch!!]

Light OFF $\rightarrow$ Light OFF [same state] [switch OFF, $w=0$]
Light OFF $\rightarrow$ Light ON [change of state] [switch ON, $w=1$]
Light ON $\rightarrow$ Light ON [same state] [Switch ON, $w=1$]
Light ON $\rightarrow$ Light OFF [change of state] [switch OFF, $w=0$]



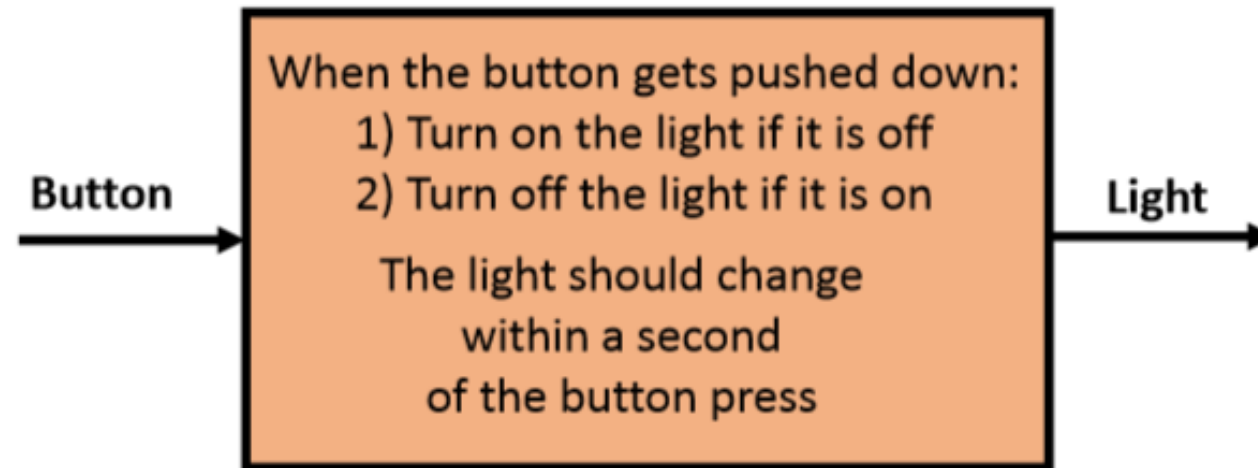
WHAT HAPPENS TO THE OUTPUT??

Light OFF $\rightarrow z=0$

Light ON $\rightarrow z=1$

Let's change the problem!!

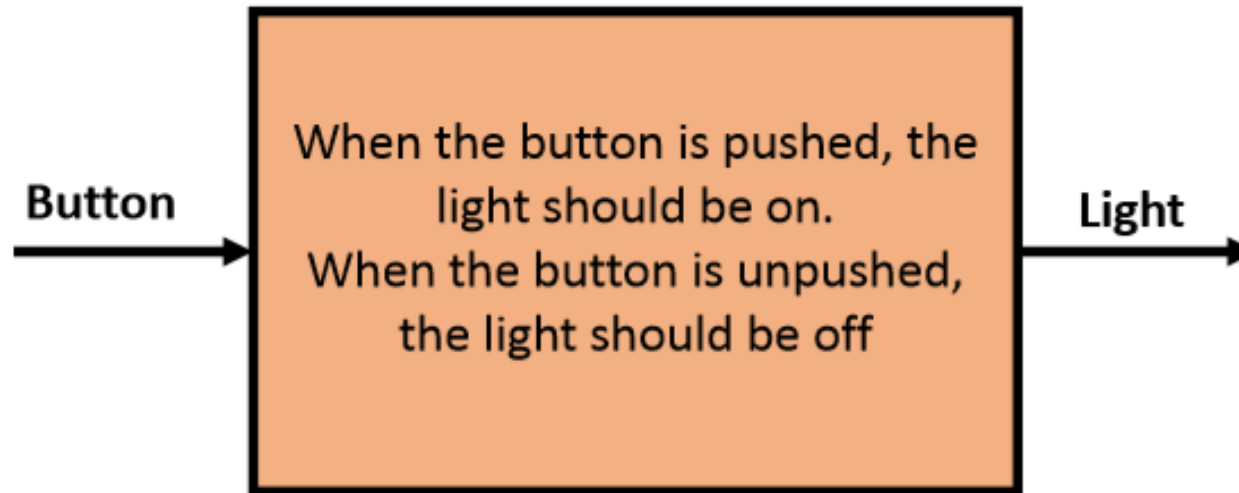
- What if you were given the following design specification:



What makes this circuit so different from those we've discussed before?

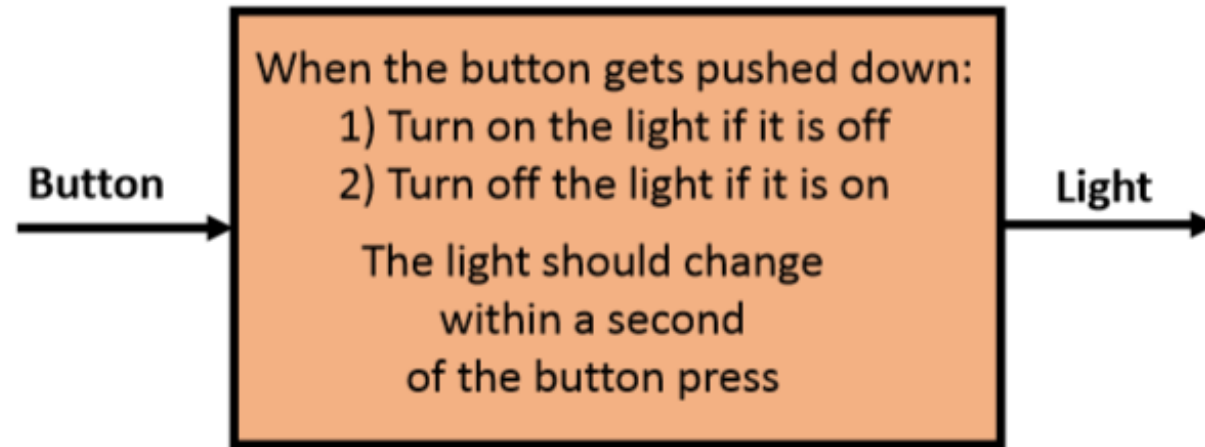
Something We Can Build (So Far)

- What if you were given the following design specification:



What makes this circuit so different from those we've discussed before?

Specifications:

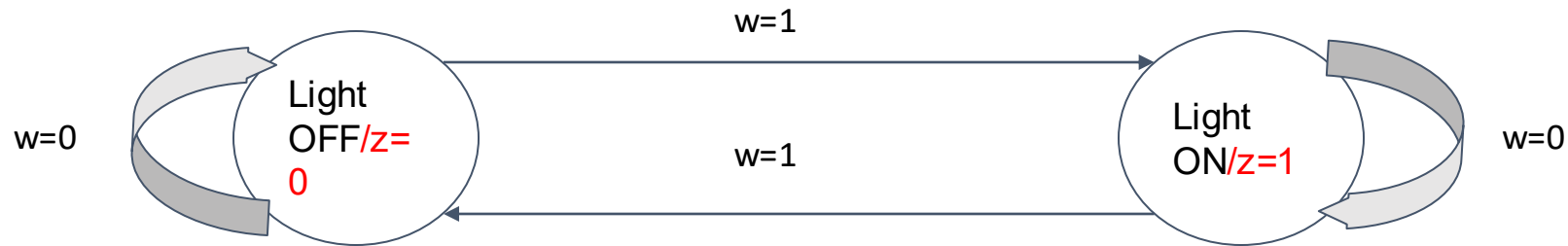


1. “State” –i.e. **the circuit has memory** (become “state-ful”).
2. The output was changed by an input “event” (pushing a button) rather than an input “value” (output may depend on previous inputs).

Illustration of the previous example

Light OFF $\rightarrow$ Light OFF [same state] [Button unpushed, $w=0$]
Light ON $\rightarrow$ Light ON [same state] [Button unpushed, $w=0$]

Light OFF $\rightarrow$ Light ON [change of state] [Button pushed, $w=1$]
Light ON $\rightarrow$ Light OFF [change of state] [Button pushed, $w=1$]



WHAT HAPPENS TO THE OUTPUT??

Light OFF $\rightarrow z=0$

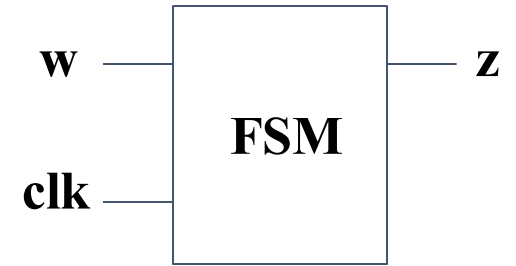
Light ON $\rightarrow z=1$

Example 1

An automatically-controlled vehicle is designed to run at some predetermined speed. However, due to some operational conditions the speed may exceed the desirable limit, in which case the vehicle has to be slowed down. To determine when such action is needed, the speed is measured at regular intervals.

- Let a binary signal w indicate whether the speed exceeds the required limit, such that $w = 0$ means that the speed is within acceptable range and $w = 1$ indicates excessive speed.
- The desired control strategy is that if $w = 1$ during two or more consecutive measurements, a control signal z must be 1 to cause the vehicle to slow down. Thus, $z = 0$ allows the current speed to be maintained, while $z = 1$ reduces the speed. Let a signal Clock define the required timing intervals, such that the speed is measured once during each clock cycle.

Example 1



From previous example 1, we wish to design the circuit that meets the following specification:

1. The circuit has one input **w**, and one output **z**.
2. All changes in the circuit occur on the **positive edge** of a clock signal.
3. The output **z** is equal to 1 if during two immediately preceding clock cycle the input **w** was equal to **1**. Otherwise the value of **z** is equal to **0**.

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	1	0

Figure 3 : Sequences of input and output signals.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence)

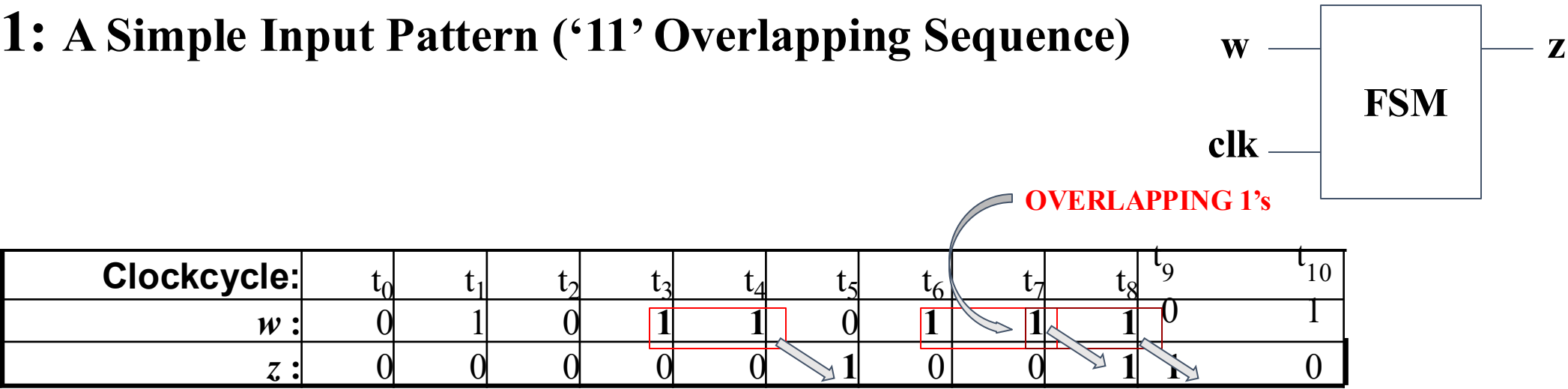
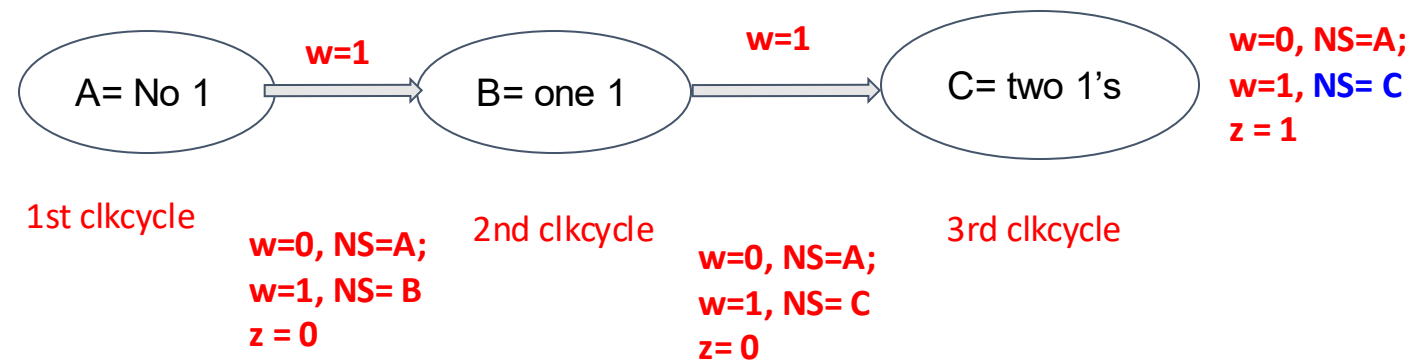


Figure 4 : Sequences of input and output signals.

The circuit detects two consecutive ‘1’. So, it must keep in memory how many ‘1’ it received in its input. So, a state diagram is introduced.

- A state- no ‘1’ in input,
- B state- one ‘1’ in input and
- C state- two ‘1’ in input.



Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit - Moore Type

State Diagram

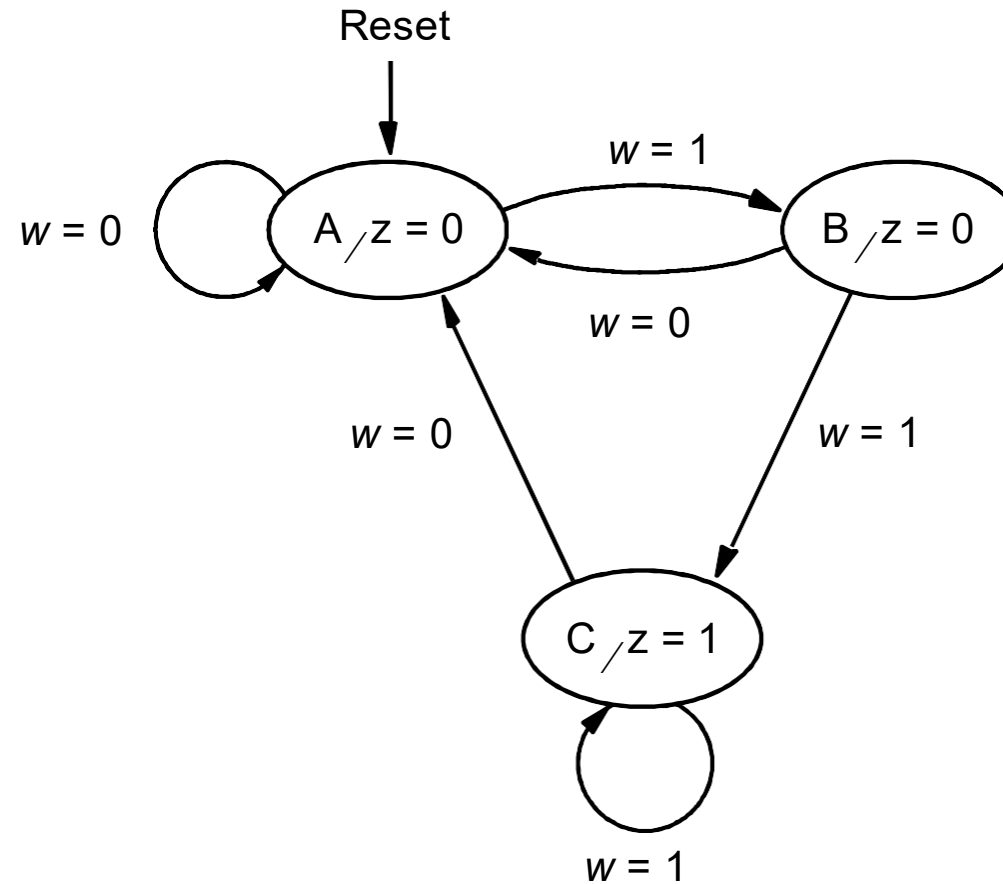


Figure 5: State diagram of a simple sequential circuit.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [State Table]

State Table

Present state	Next state					Output z
	$w=0$		$w=1$			
A	A		B			0
B	A		C			0
C	A		C			1

Note:
n bits can
represent
 2^n states.

Figure 6: State table for the sequential circuit in Figure 5.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [State Assigned Table]

	Present state	Next state		Output z
		$w=0$	$w=1$	
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

We need to find expressions (K-map) for

- next state (Y_2, Y_1)
- output (z).

Note that,

$$\triangleright Y_1 = f(w, y_1, y_2)$$

$$\triangleright Y_2 = f(w, y_1, y_2)$$

$$\triangleright z = f(y_1, y_2)$$

Figure 7: State-assigned table for the sequential circuit in Figure 7.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [K-Map]

		y_2y_1			
		00	01	11	10
w	0	0	0	d	0
	1	1	0	d	0

		y_2y_1			
		00	01	11	10
w	0	0	0	d	0
	1	0	1	d	1

y_2	y_1	
	0	1
0	0	0
1	1	d

Ignoring don't cares

$$Y_1 = wy_1y_2$$

$$Y_2 = wy_1y_2 + wy_1y_2$$

$$z = y_1y_2$$

Using don't cares

$$Y_1 = wy_1y_2$$

$$Y_2 = wy_1 + wy_2$$

$$= w(y_1 + y_2)$$

$$z = y_2$$

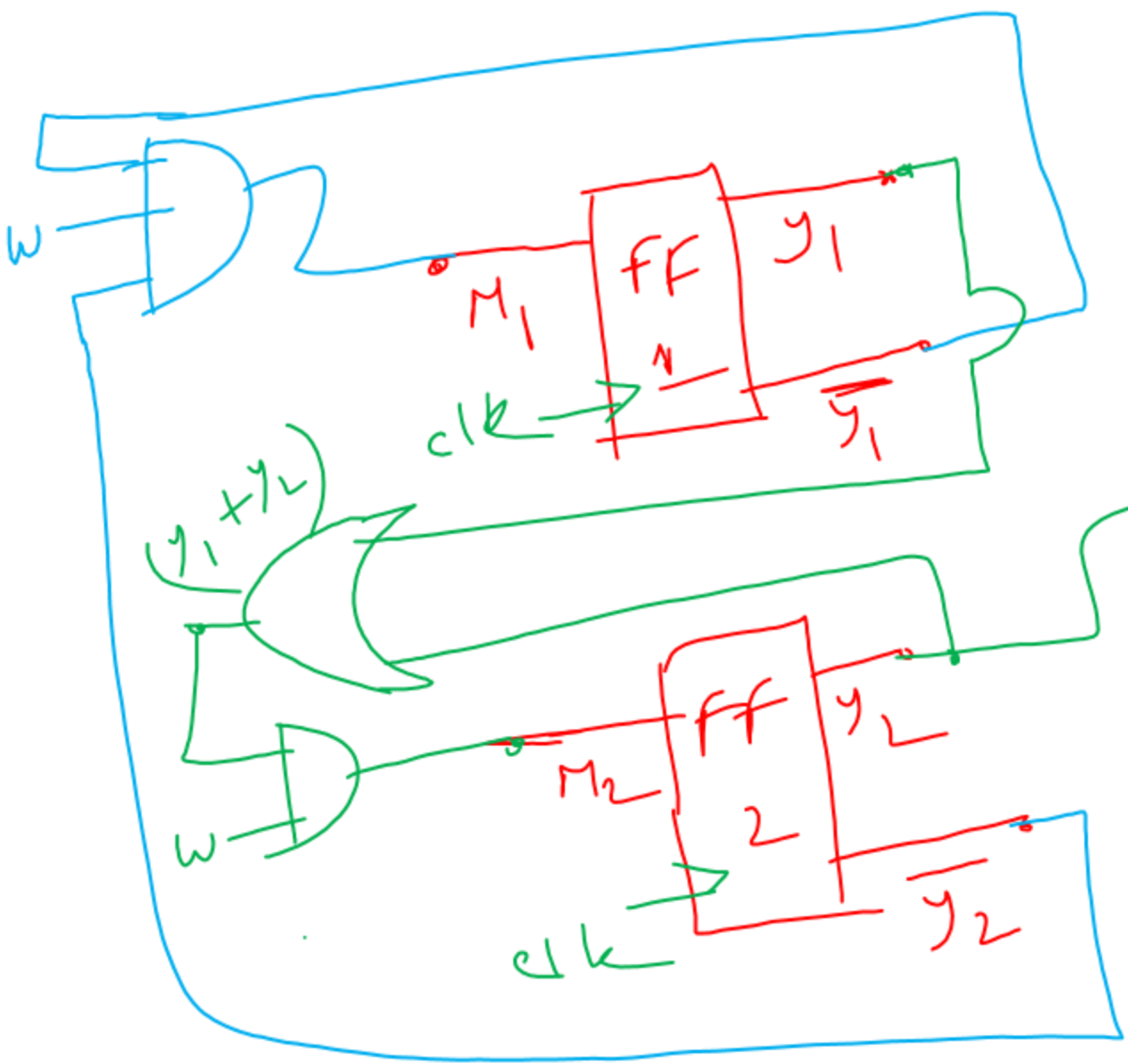
Figure 8: Derivation of logic expressions for the sequential circuit in Figure 7.

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [Circuit]

Note that,

- # flipflops = # bits to represent current state (#y)
- Outputs of flipflops correspond to current state bits (y_1, y_2)
- Inputs of flipflops correspond to next state bits (Y_1, Y_2)

In this problem, we need **two flipflops** as we have used **two bits** (y_1, y_2) to assign all the present states (A, B, C).



Current: $y_2 y_1$

state

0 0 5 7 A
z

Next: $M_2 M_1$
state

$$M_1 = w \overline{y_1} \overline{y_2}$$

$$M_2 = w(y_1 + y_2)$$

$$z = y_2$$

Example 1: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit [Circuit]

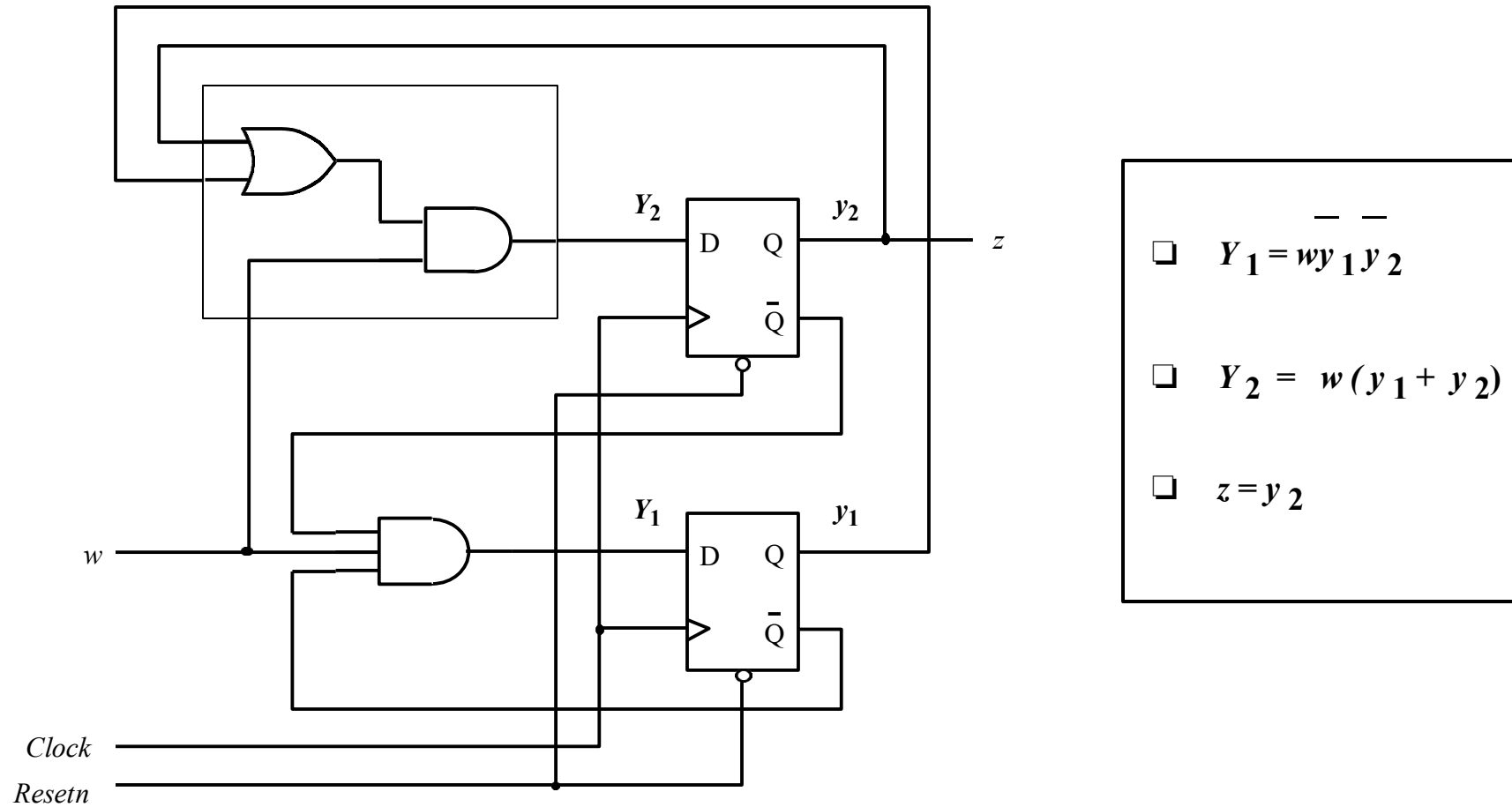
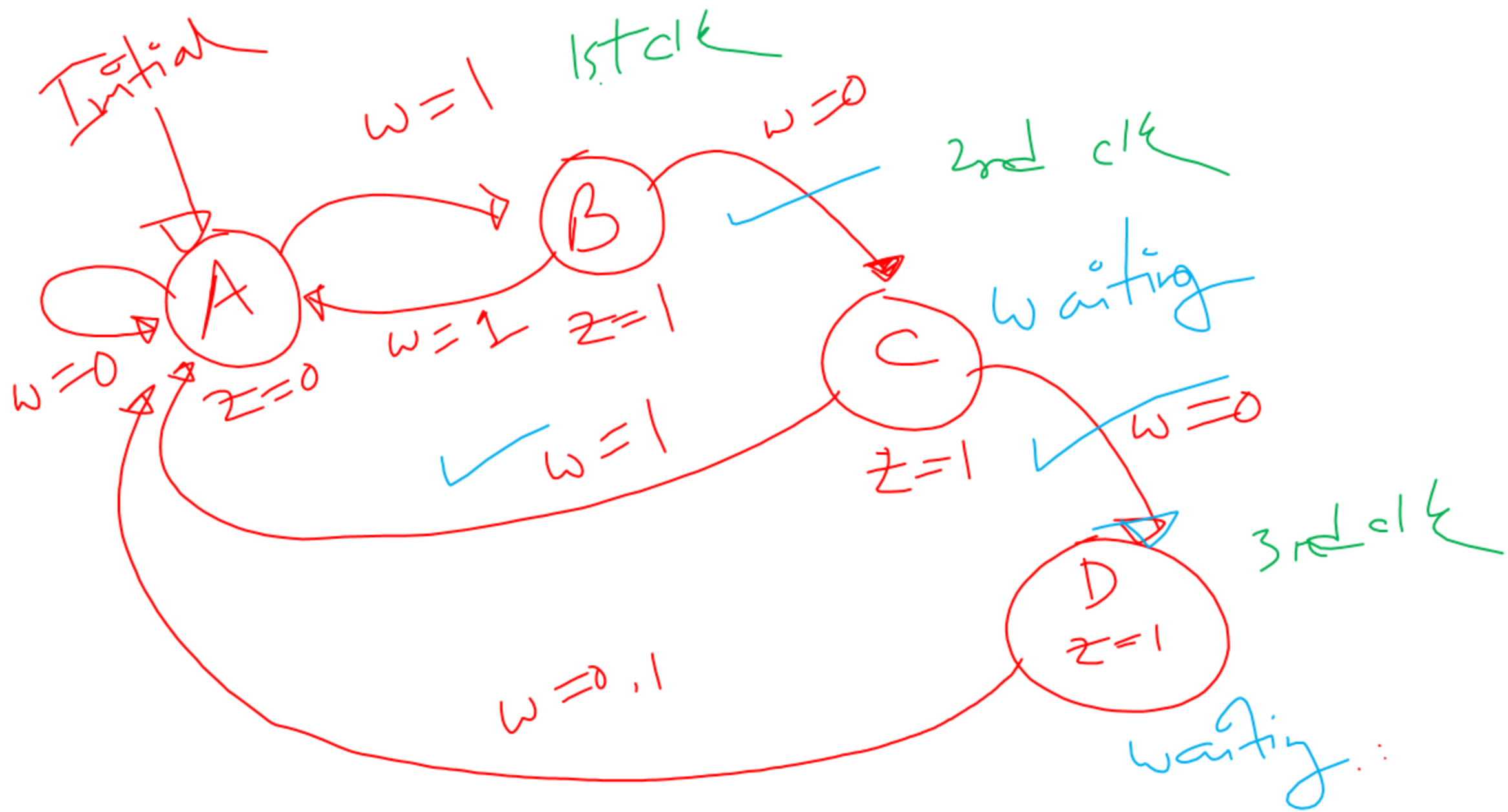


Figure 9. Final implementation of the sequential circuit derived in Figure 8.

Q4. [CO2] - Marks 10

You are tasked with designing a CD/DVD drive's control circuitry using a Moore-type finite-state machine. The drive has one input push button w , which generates $w = 1$ when pushed, and $w = 0$ when un-pushed. A $w = 1$ input causes the tray to open if it was already closed, or causes the tray to close if it was already open. The control circuit has only one output signal, z , which determines if the CD/DVD drive should be open ($z = 1$) or closed ($z = 0$) with the following instructions:

- If the tray is closed, pushing the button (setting $w = 1$) should generate the signal ($z = 1$) to open the tray
- If the tray has been open for less than 3 clock cycles, pushing the button (setting $w = 1$) should generate the signal ($z = 0$) to close the tray
- The tray should automatically close after being open for 3 clock cycles
- For simplicity, assume that the time required for opening and closing the tray is negligible compared to the system clock's time period



Q6. [CO1] - Marks 10

A car was in a stand still condition. To *start* the car, a start/stop button, w should be pressed. An output, $z = 1$ confirms that the car is *moving* (*accelerating*) or *running* (*optimum constant speed*).

- If $w = 0$, the car should remain *still* initially. For, $w = 1$, the car should start *moving* (if it was *still*) or stop (if it has just started *moving*, but not *running* yet).
- Once moving, the car should reach the optimal speed within two consecutive clock cycles without pressing the start/stop button, w .
- After two cycles, it will automatically gain the optimal speed to keep *running*. The car should maintain its speed independent of the value of w while *running*.
- The break, b can only be pressed for an emergency stop while *running*. If the car is *still* or *moving*, break, b can be ignored.

[Hint: The system has two inputs: w , b and one output: z]

Summary

Steps->

- State diagram
- State table
- State assigned table
- K-map (next states & outputs)
- Circuit

Example 1(cont.): A Simple Input Pattern ('11' Non-Overlapping Sequence) (Detection Circuit - Moore Type)

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	0	0

NON-OVERLAPPING 1's

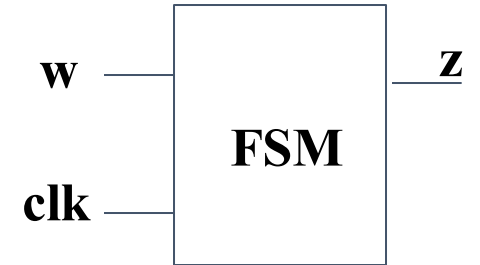
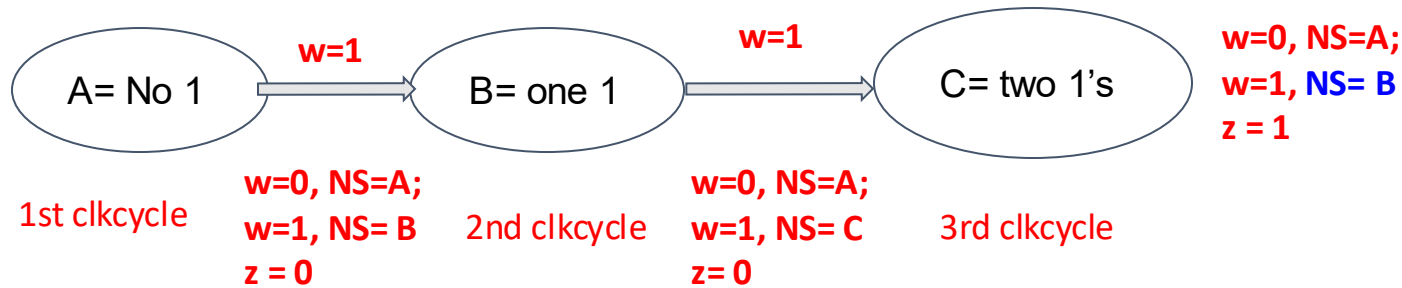
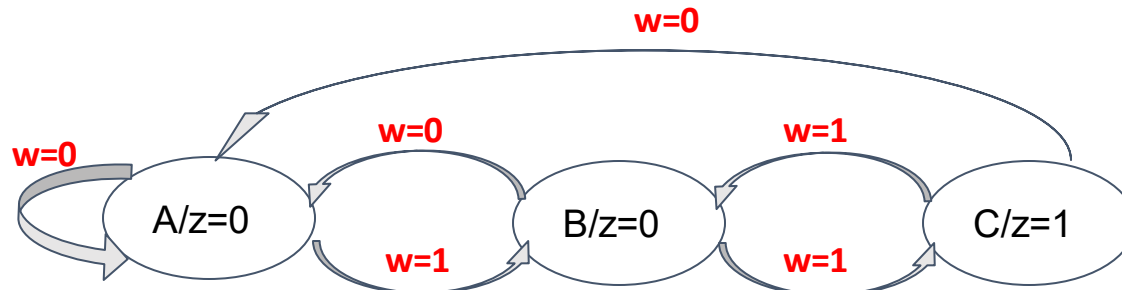


Figure 4 : Sequences of input and output signals.



STATE DIAGRAM



General form of a sequential circuit

- W represents input/s
- Z represents output/s
- Flip-flops are used to save the “present” state of the circuit, Q
- Also called Finite State Machine (FSM) or simply, machine
- 2 types: depending on the connection drawn in pink on the circuit

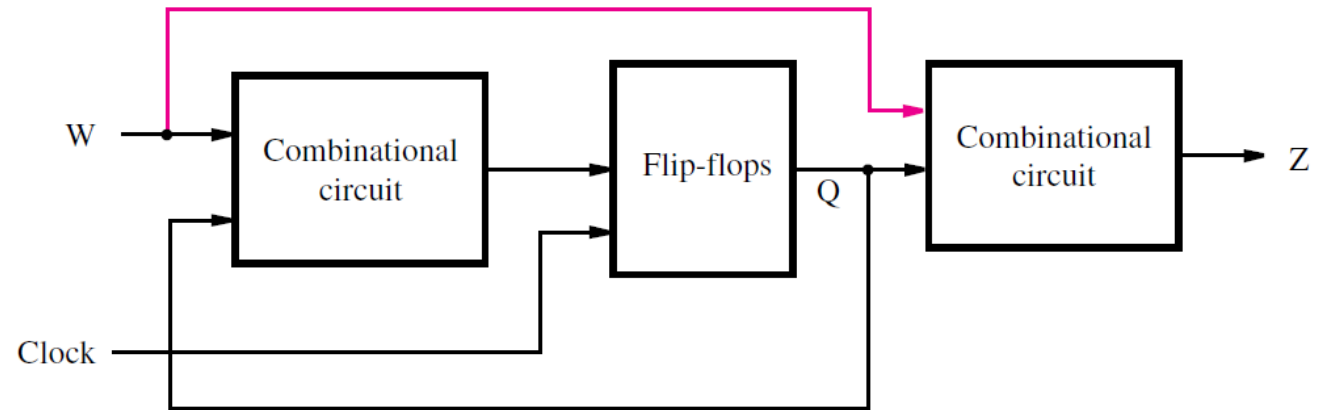
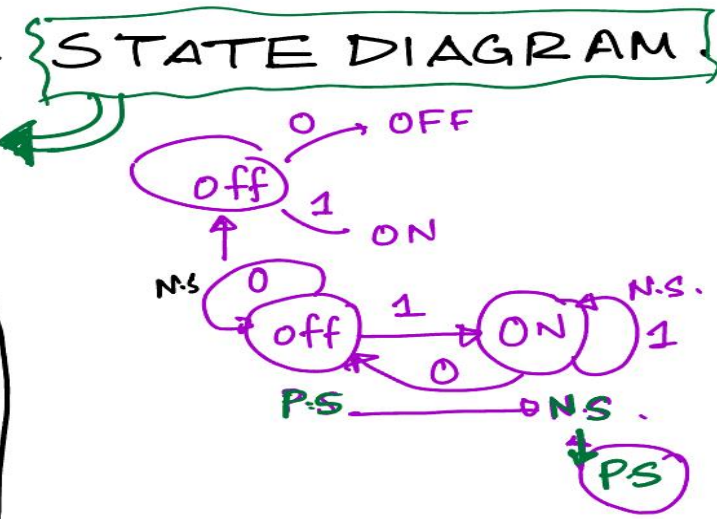
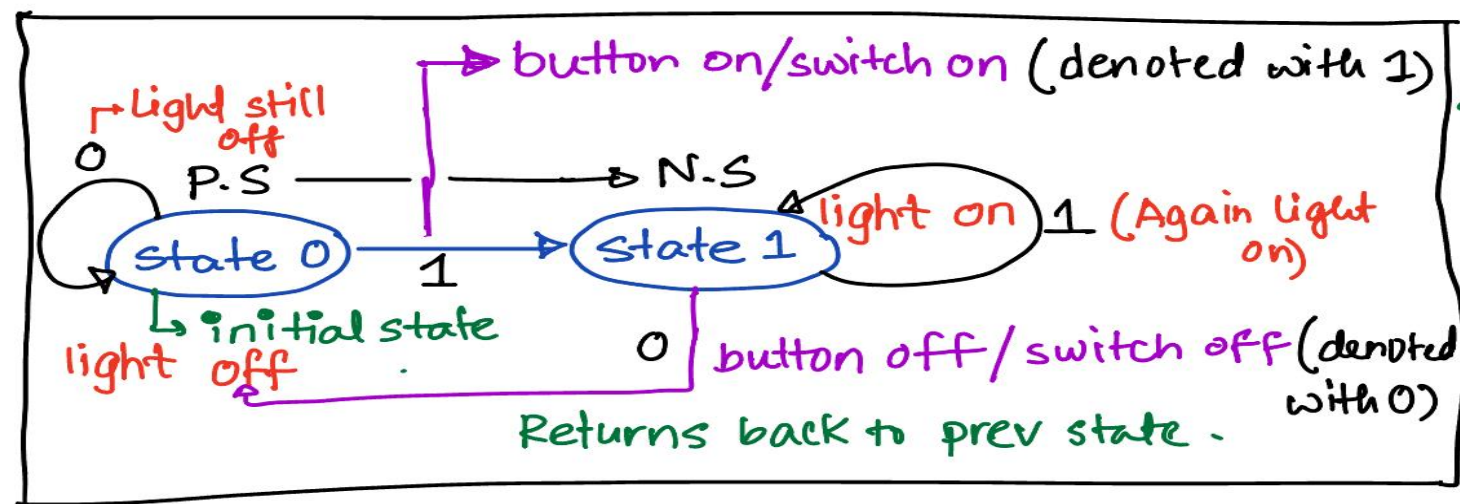
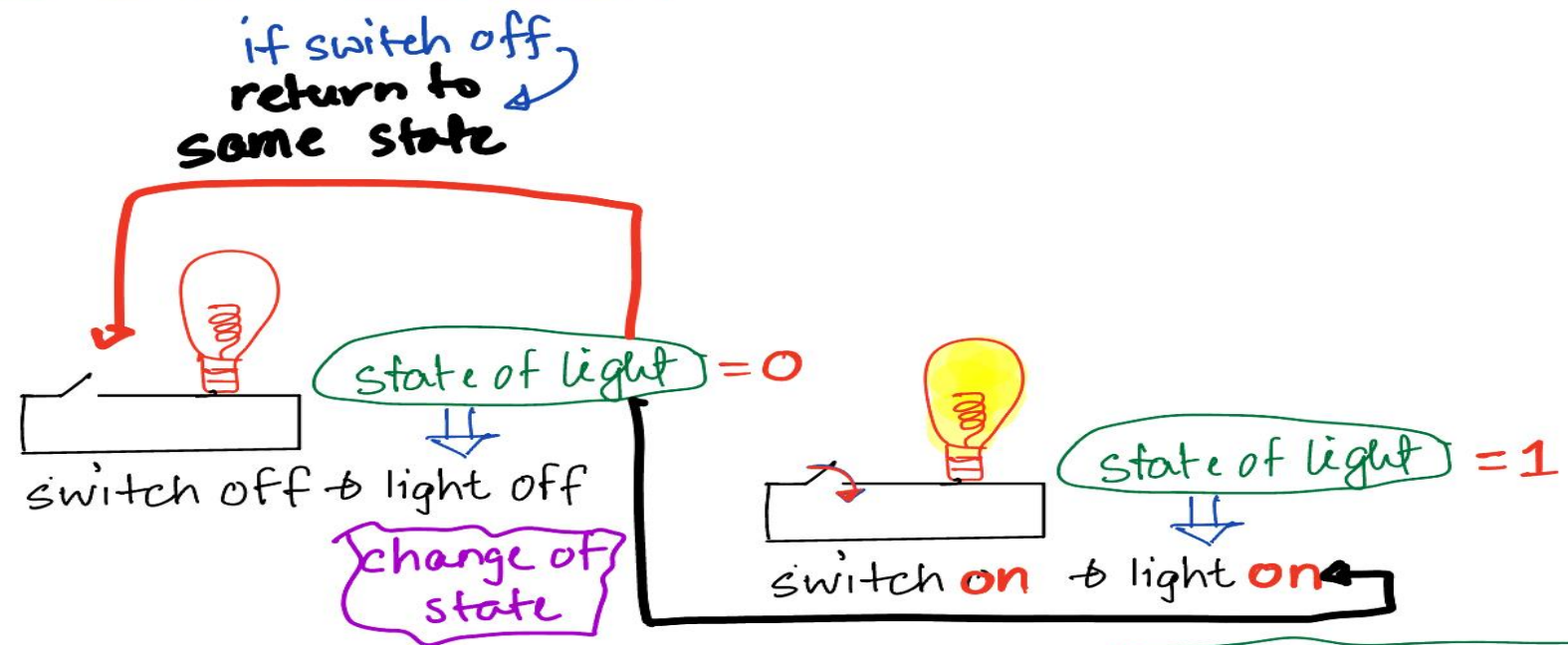


Figure: General form of a sequential circuit

Illustration of light switch problem

Illustration of previous example

Example: 1 Detector



Example: 1 Detector

State Table

state 0 $\rightarrow$ A
state 1 $\rightarrow$ B

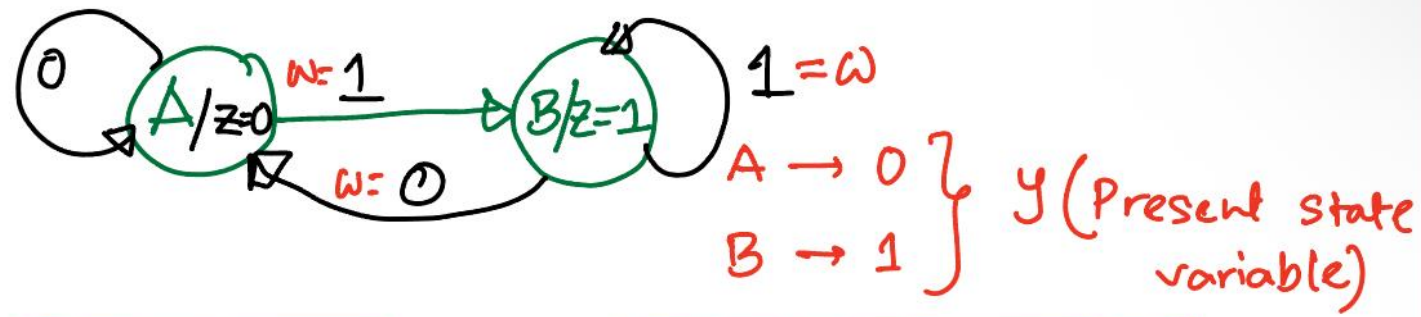
P.S	N.S		Output z
	w=0	w=1	
A	A	B	0
B	A	B	1

State Assigned Table

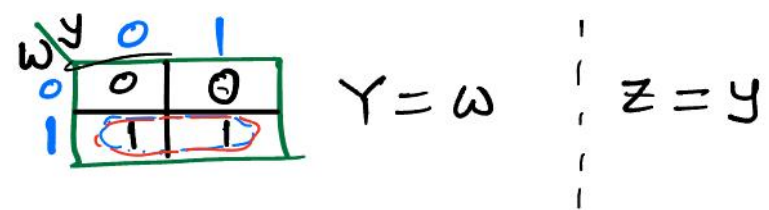
P.S _y	N.S (Y)		Output z
	w=0	w=1	
0(A)	0	1	0
1(B)	0	1	1

w = switch status = input $\rightarrow$ 0
 $\rightarrow$ 1

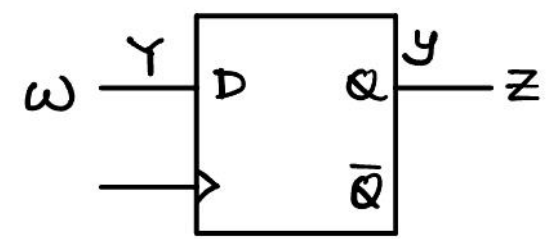
Representation of state Diagram



Find next state variable, Y & output, z



1 Detection Circuit



FSM types

- Moore type FSM

Output (Z) depends only on the “present” state (Q) of the circuit

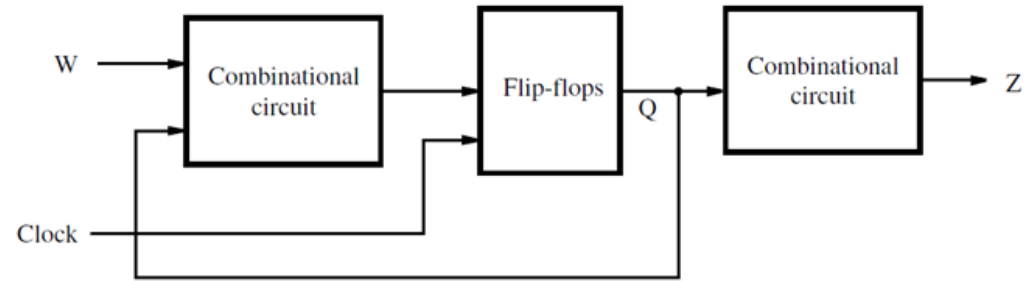


Figure: Moore type FSM

- Mealy type FSM

Output (Z) depends on both the primary inputs (W) and the “present” state (Q) of the circuit

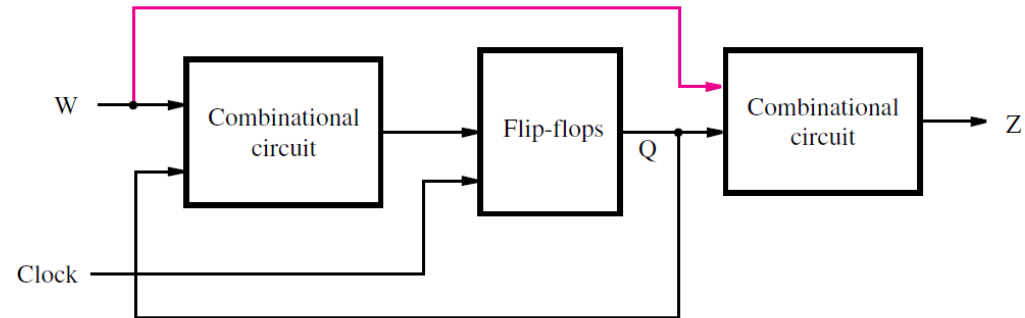


Figure: Mealy type FSM

Moore vs. Mealy type machines

Moore Machine	Mealy Machine
▪ Output depends only on the <i>present state</i>.	▪ Output depends both on the <i>present state</i> and the <i>present input</i>.
▪ Generally, it has <i>more states</i> than Mealy Machine.	▪ Generally, it has <i>fewer states</i> than Moore Machine.
▪ The output is a function of the current state and changes <i>at clock edges</i>	▪ The output is a function of the transitions and can change any instant
▪ In Moore machines, more logic is required to decode the outputs resulting in more circuit delays. They generally react one clock cycle later.	▪ Mealy machines react faster to inputs. They generally react in the same clock cycle.

4. Design a Mealy machine that simulates detecting fraudulent transactions in a stream of credit card transactions based on input categories:

- A: Low-value transaction.
- B: Medium-value transaction.
- C: High-value transaction.

The machine outputs 1 (fraud detected) if two consecutive high-value transactions (C) are followed by a low-value transaction (A). Otherwise, it outputs 0.

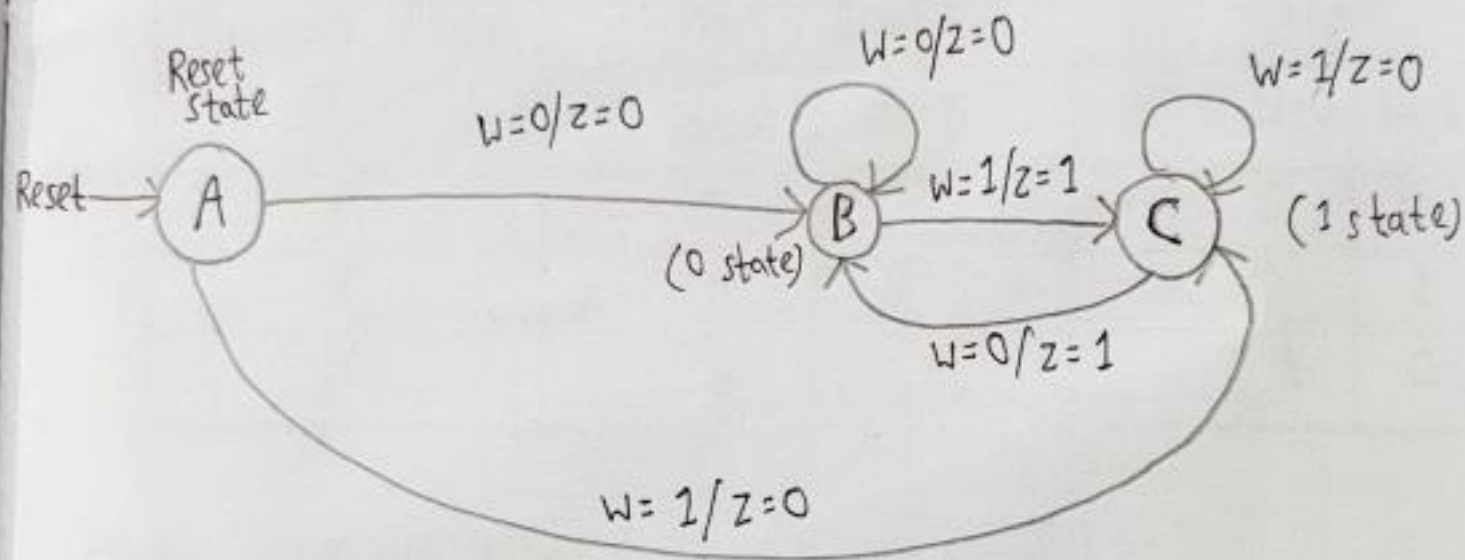
Example:

- **Input:** B C C A B
- **Output:** 00010

	(a)	Design the state diagram. Mark your transitions clearly.	3
	(b)	Derive the assigned table and write the Verilog code to implement the following FSM.	4
	(c)	Validate your answer with the appropriate input and output waveforms on a timing diagram. Include all possible combinations of the input (w).	3

(3)

(a)



This FSM here is of ~~no~~ mealy type. Here, the output(z) is dependent on both the current state and the current input.

(b)

State Table:

Present State	Next State		Output	
	W=0	W=1	W=0	W=1
A	B	C	0	0
B	B	C	0	1
C	B	C	1	0

State Assigned Table:

Present State	Next State		Output	
	w=0	w=1	w=0	w=1
00	01	10	0	0
01	01	10	0	1
10	01	10	1	0

CSE 460: VLSI Design

Lecture 9: Finite State Machines (part 2)

Example 2: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit (Mealy Type)

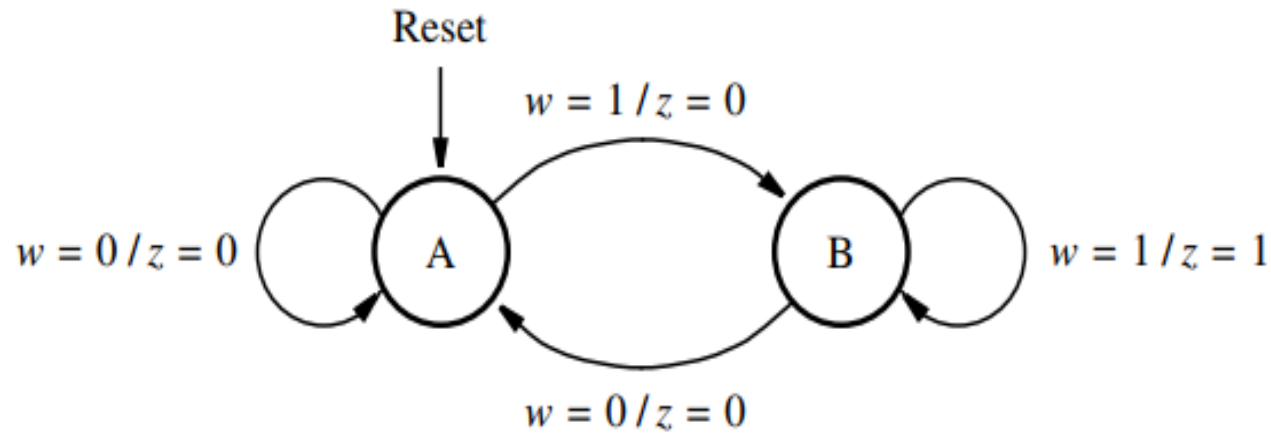
Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	1	0	0	1	1	0	0

Figure 6.22 Sequences of input and output signals.

Steps->

- State diagram
- State table
- State assigned table
- K-map
- Circuit

Example 2: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit (Mealy Type)



Steps->

- State diagram
- State table
- State assigned table
- K-map
- Circuit

Figure 6.23 State diagram of an FSM that realizes the task in Figure 6.22.

Example 2: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit (Mealy Type)

Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

Figure 6.24 State table for the FSM in Figure 6.23.

Steps->

- State diagram
- **State table**
- **State assigned table**
- K-map
- Circuit

	Present state	Next state		Output	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

Figure 6.25 State-assigned table for the FSM in Figure 6.24.

Note that,

➤ $Y = f(w, y)$

➤ $z = f(w, y)$

Example 2: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit (Mealy Type)

Y =>

y \ w	0	1
0	0	1
1	0	1

$$Y = w$$

z =>

y \ w	0	1
0	0	0
1	0	1

$$z = wy$$

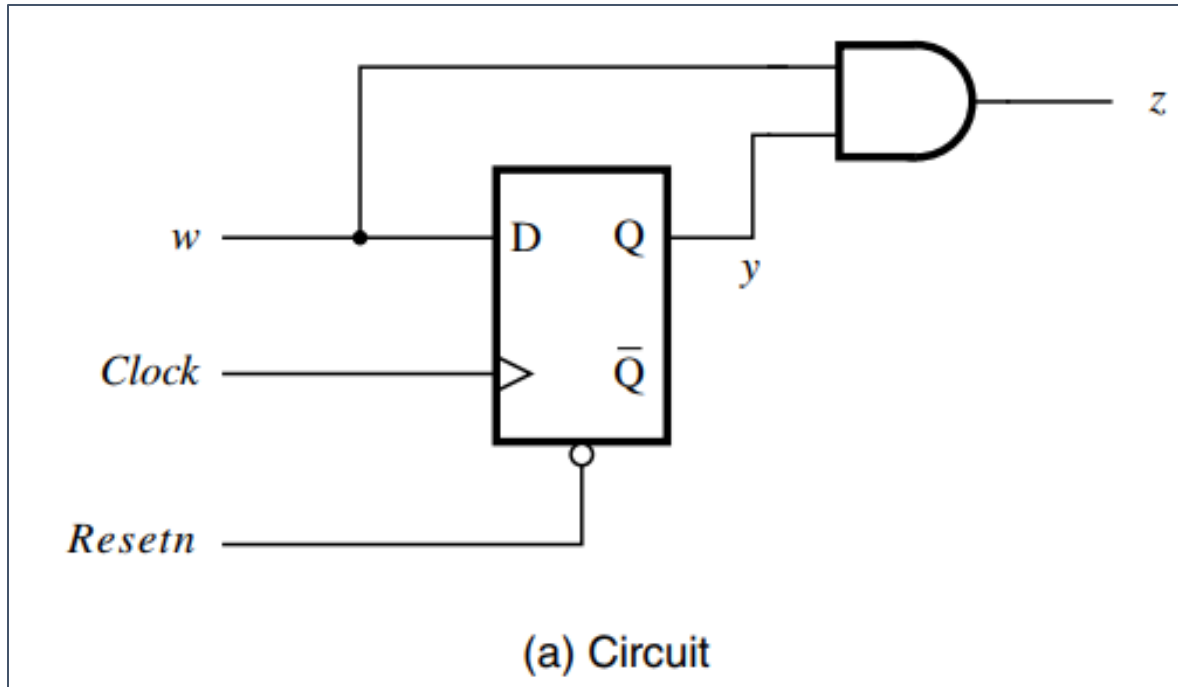
Steps->

- State diagram
- State table
- State assigned table
- **K-map** (Y and z)
- Circuit

Note that,

- $Y = f(w, y)$
- $z = f(w, y)$

Example 2: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit (Mealy Type)

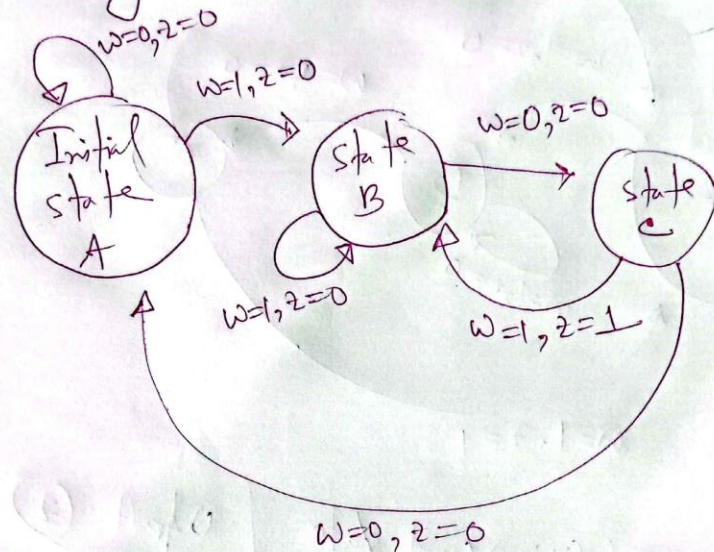


Steps->

- State diagram
- State table
- State assigned table
- K-map
- **Circuit**

Mealy Type FSM

④ Detecting Sequence: "101" (Overlapping)



Present state (x ₂ x ₁)	Next state (y ₂ y ₁)		Output (z)	
	w=0	w=1	w=0	w=1
(00) A	A(00)	B(01)	0	0
(01) B	C(10)	B(01)	0	0
(10) C	A(00)	B(01)	0	1

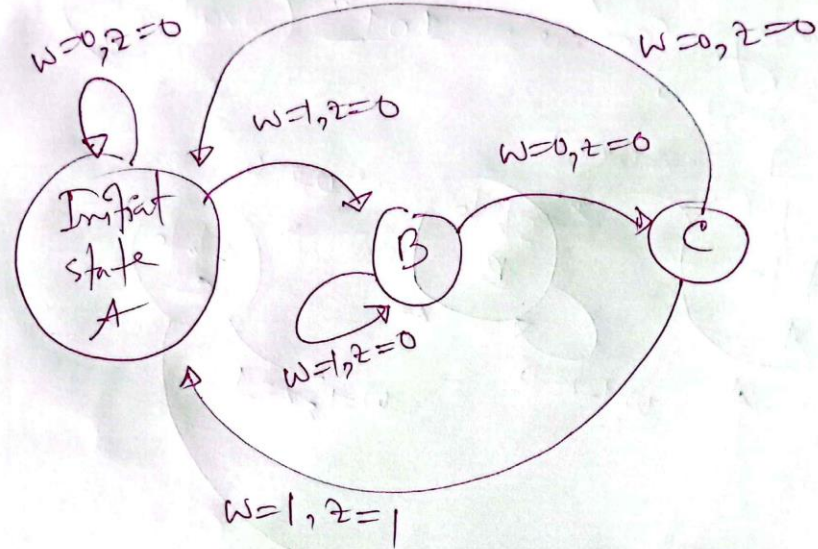
Example 3: A Simple Input Pattern ('101' Overlapping Sequence) Detection Circuit (Mealy Type)

Important:

In Mealy Type FSM, you do not need to declare a separate state for your final output (for this example: no separate state for '101' is required). After getting your output $z = 1$, you will start your 2nd loop (Start finding $z=1$ for the 2nd time).

∴ No of Flip-Flop = No of bits in Present state
= 3

Detecting Sequence "101" (Non-overlapping)



**Example 4: A Simple Input Pattern
(‘101’ Non-Overlapping Sequence)
Detection Circuit (Mealy Type)**

Present state ($y_2 y_1$)	Next state ($y_2 y_1$)		Output (z)	
	w=0	w=1	w=0	w=1
(00) A	A(00)	B(01)	0	0
(01) B	C(10)	B(01)	0	0
(10) C	A(00)	A(00)	0	1

Example 5:

1. 1. A finite state machine that has to generate $z = 1$ when the previous four values of input w were 1001 or 1111; otherwise, $z = 0$. Overlapping input patterns are allowed. The machine also has a negative reset. An example of the desired behavior is

w : 0101 1110 0110 0111 11

z : 0000 0010 0100 0100 11

(a)	Design the state diagram. Mark your reset states and transitions clearly. Is this machine Mealy type or Moore type?	3
(b)	Derive the assigned table and write the Verilog code to implement the following FSM.	4
(c)	Validate your answer with the appropriate input and output waveforms on a timing diagram. Include all possible combinations of the input (w).	3

Solution:

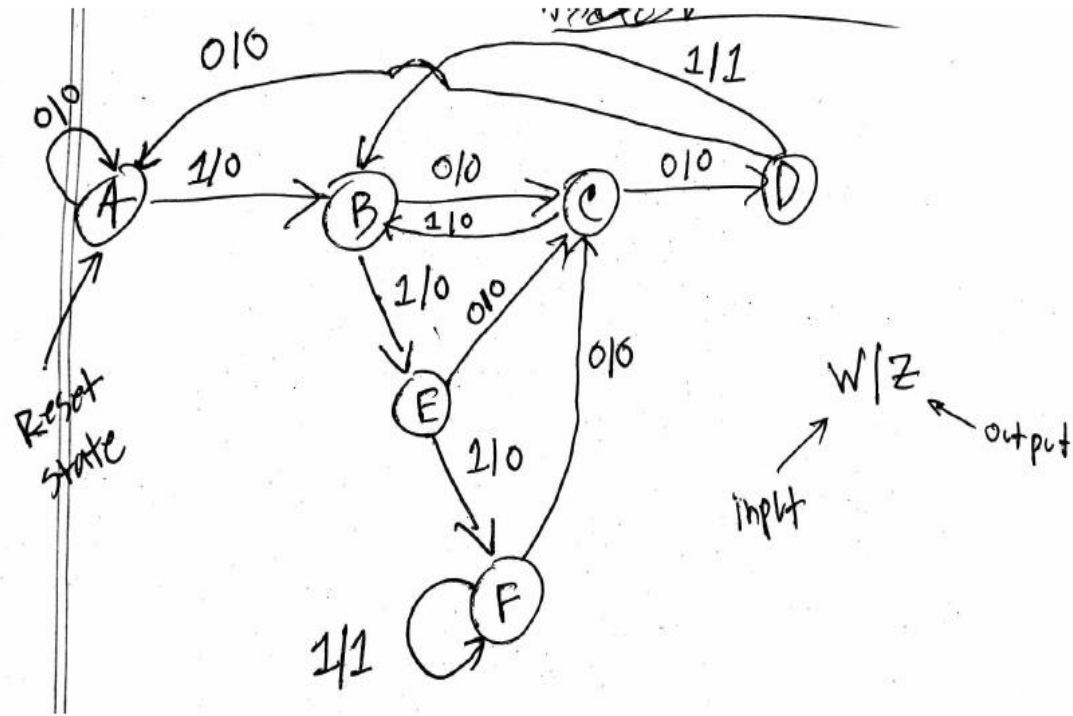
The Machine is Mealy type

My Reset state is A State

W = Input Sequence

Z = Output

Format is W/Z



Present State	Next State		Output	
	w=0	w=1	w=0	w=1
A	A	B	0	0
B	C	E	0	0
C	D	B	0	0
D	A	B	0	1
E	C	F	0	0
F	C	F	0	1

State Assigned Table:

~~A = 000~~
~~B = 010~~
~~C = 0~~
 A = 000
 B = 001
 C = 010
 D = 011
 E = 100
 F = 101

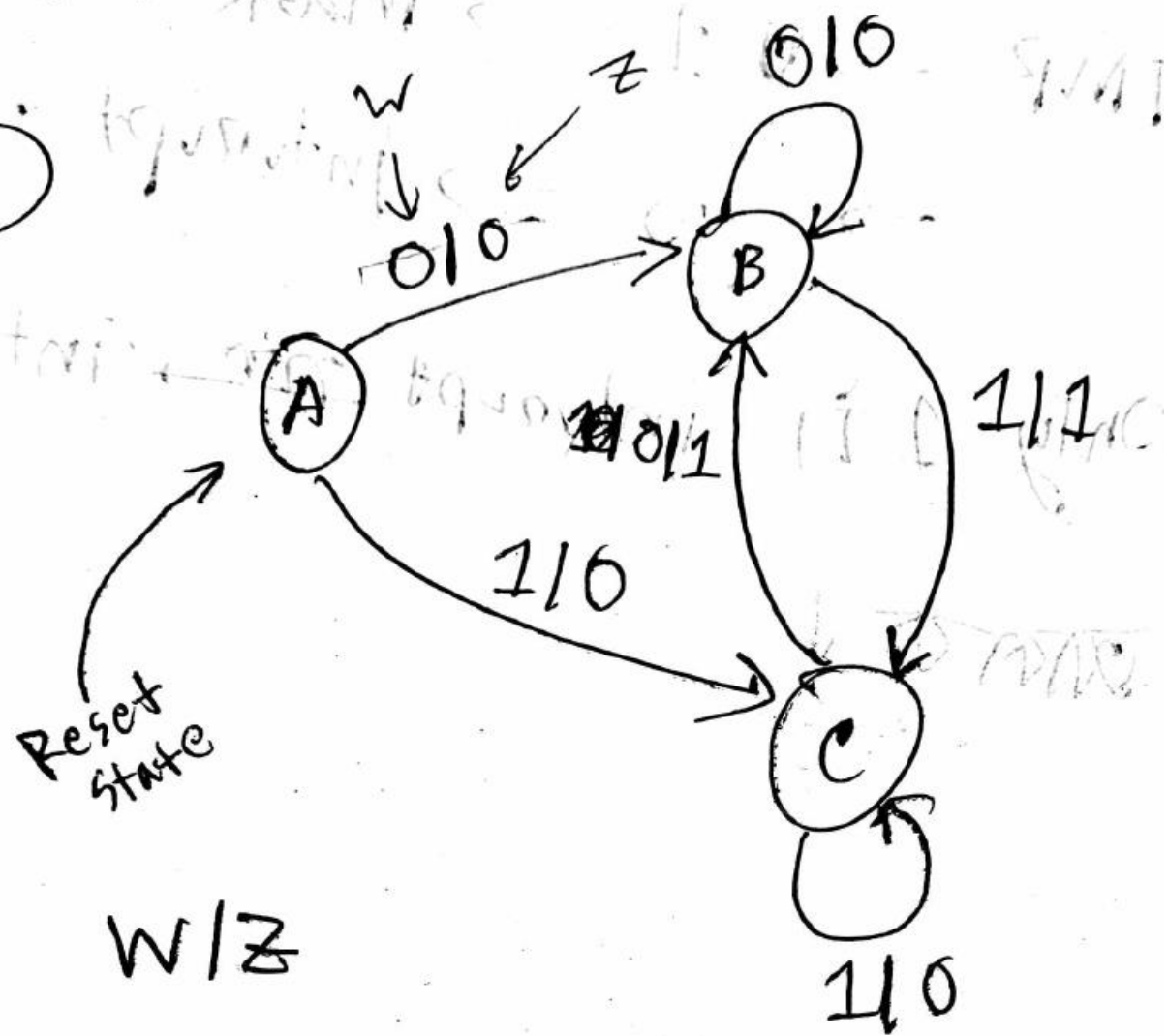
Present state	Next State		Out put	
	w=0	w=1	w=0	w=1
000	000	001	0	0
001	010	100	0	0
010	011	001	0	0
011	000	001	0	1
100	010	101	0	0
101	010	101	0	1

Example 6:

The input-output behavior of a sequence detector FSM is given below.																			
<i>CLK</i>	<i>t</i> ₁	<i>t</i> ₂	<i>t</i> ₃	<i>t</i> ₄	<i>t</i> ₅	<i>t</i> ₆	<i>t</i> ₇	<i>t</i> ₈	<i>t</i> ₉	<i>t</i> ₁₀	<i>t</i> ₁₁	<i>t</i> ₁₂	<i>t</i> ₁₃	<i>t</i> ₁₄	<i>t</i> ₁₅	<i>t</i> ₁₆	<i>t</i> ₁₇	<i>t</i> ₁₈	
<i>w</i>	1	0	0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	
<i>z</i>	0	1	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	0	
(a)	Identify the <u>sequence</u> and the <u>type</u> of FSM for the above input-output combinations. [<u>Hint</u> : The above table contains two sequences]																		1+1
(b)	Design the state diagram, the state table, and the <i>gray-encoded</i> state assigned table. [Overlapping allowed].																		3+2+2
(c)	Determine the logic expressions of the next state and output variables using k-maps.																		6

- ** Alternating 0 and 1
- ** Two consecutive inputs are different

Solution:



Present State	Next State		Output Z	
	w=0	w=1	w=0	w=1
A	B	A C	0	0
B	B	A C	0	1
C	B	C	1	0

Present state	Next State		Output Z	
	w=0	w=1	w=0	w=1
00	01	10	0	0
01	01	10	0	1
10	01	10	1	0

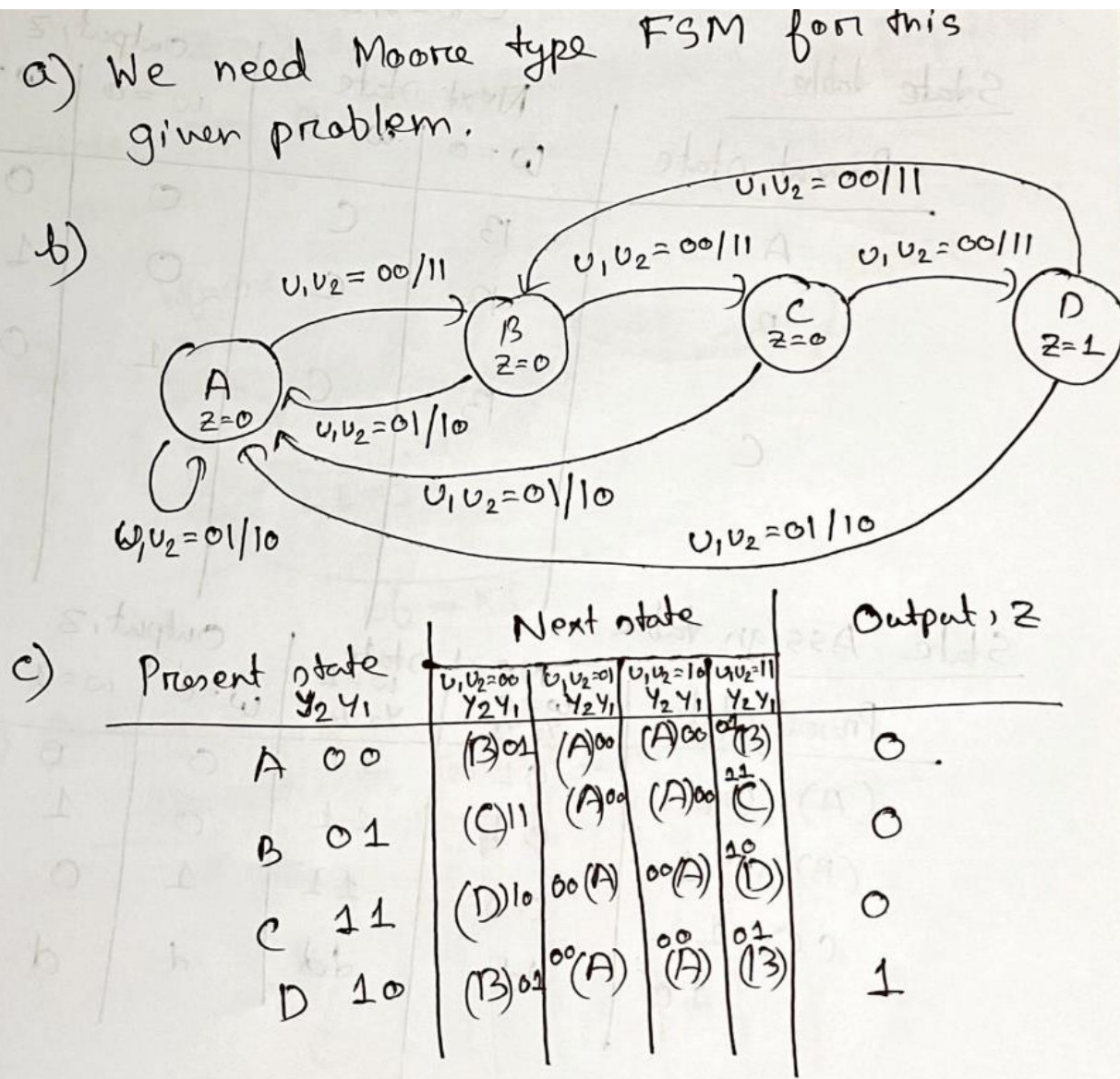
Example: 07: Double input FSM

Consider a sequential circuit with two inputs ($u1$ & $u2$) and a single output, z . CLEO is a prestigious conference where researchers from all over the world submit their research articles. These articles are handled by an **Editor** and a **Reviewer**. They review each article *independently*, and either approve a submitted article (denoted by 1) or reject it (denoted by 0). The decisions of the **Editor** are given by a bit stream, $u1$, whereas the decisions of the **Reviewer** are given by $u2$. If for three consecutive articles, both of their decisions are the same (i.e. $u1 = u2$ for three consecutive clock cycles), then they receive a consistency reward ($z = 1$) from the conference committee after a period.

Here, the decisions of the Editor & the Reviewer for a number of submitted articles & their corresponding reward instances are given below:

$u1$	0	1	1	0	1	1	1	0	0	0	1
$u2$	1	1	1	0	0	0	1	0	0	1	1
z	0	0	0	0	1	0	0	0	0	1	0

(a)	What type of FSM would you need for the given problem?	1
(b)	Design the state diagram for the corresponding FSM.	4
(c)	Derive the state-assigned table from your state diagram in (a).	4
(d)	Evaluate the <u>next state</u> & <u>output</u> logic expressions to implement the FSM & calculate the number of flip-flops required for the circuit implementation. (You don't need to draw the circuit implementation.)	5 + 1



Example: 07: Double output FSM

CLK	1	2	3	4	5	6	7	8	9	10	11	12	13
W	0	1	0	1	0	0	0	0	1	1	0	0	1
Z1	0	0	1	1	1	1	0	0	0	1	0	1	0
Z2	0	0	0	0	0	0	1	1	1	0	1	0	1

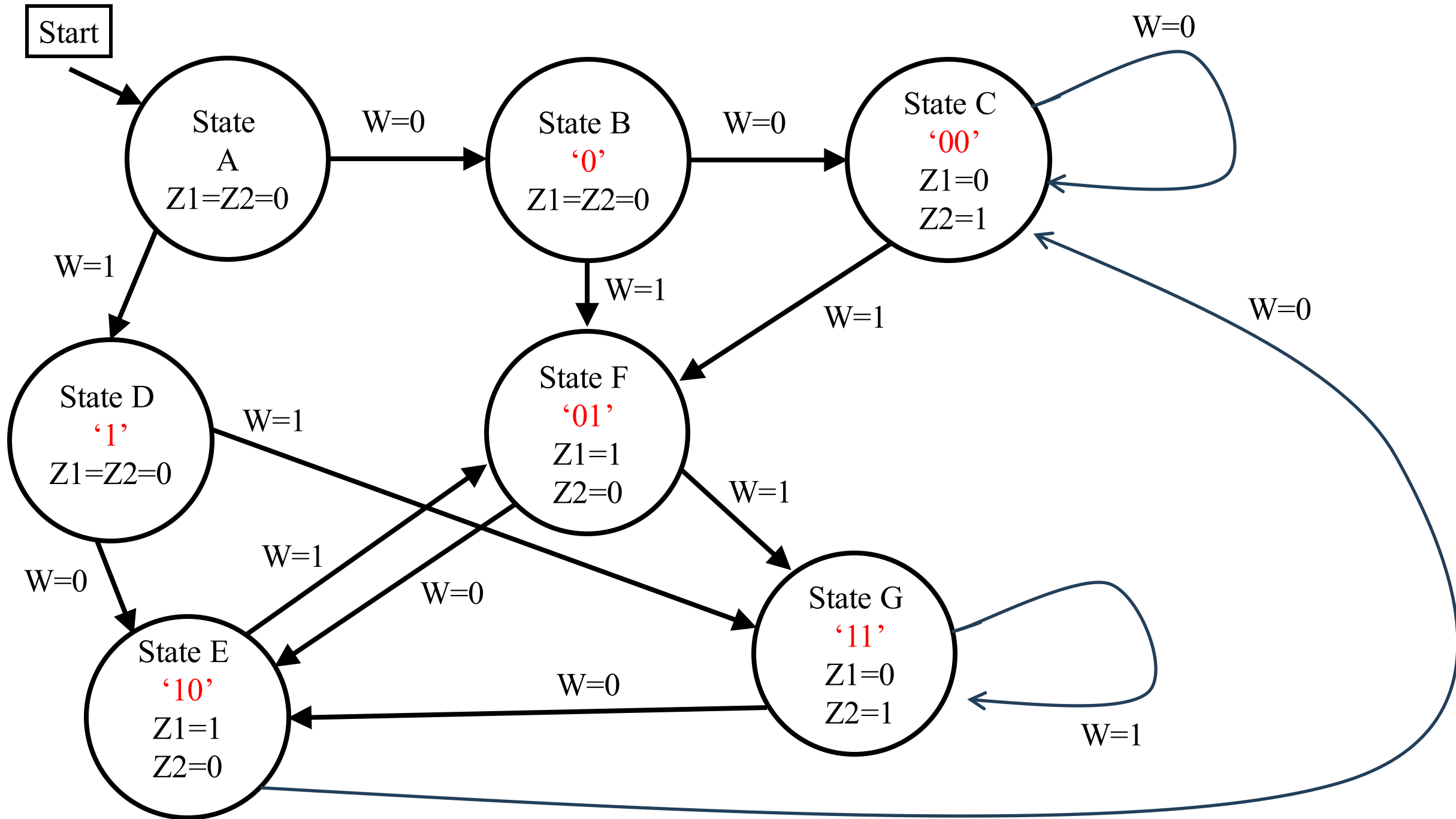
Solution: Moore Type FSM

For,

$W = 01$ or 10 , $Z1 = 1$ and $Z2 = 0$

For,

$W = 00$ or 11 , $Z1 = 0$ and $Z2 = 1$



Present State (Y ₂ Y ₁ Y ₀)	Next State (M ₂ M ₁ M ₀)		Output	
	W = 0	W = 1	Z ₂	Z ₁
A(000)	B	D	0	0
B(001)	C	F	0	0
C(010)	C	F	1	0
D(011)	E	G	0	0
E(100)	C	F	0	1
F(101)	E	G	0	1
G(110)	E	G	1	0
H(111)	d	d	d	d

Example: 08: Double output FSM

CLK	1	2	3	4	5	6	7	8	9	10	11	12	13
w	0	1	0	1	0	0	0	0	1	1	0	0	1
Z1	0	0	1	0	1	0	0	0	0	1	0	0	0
Z2	0	0	0	0	0	0	1	1	1	0	0	0	1

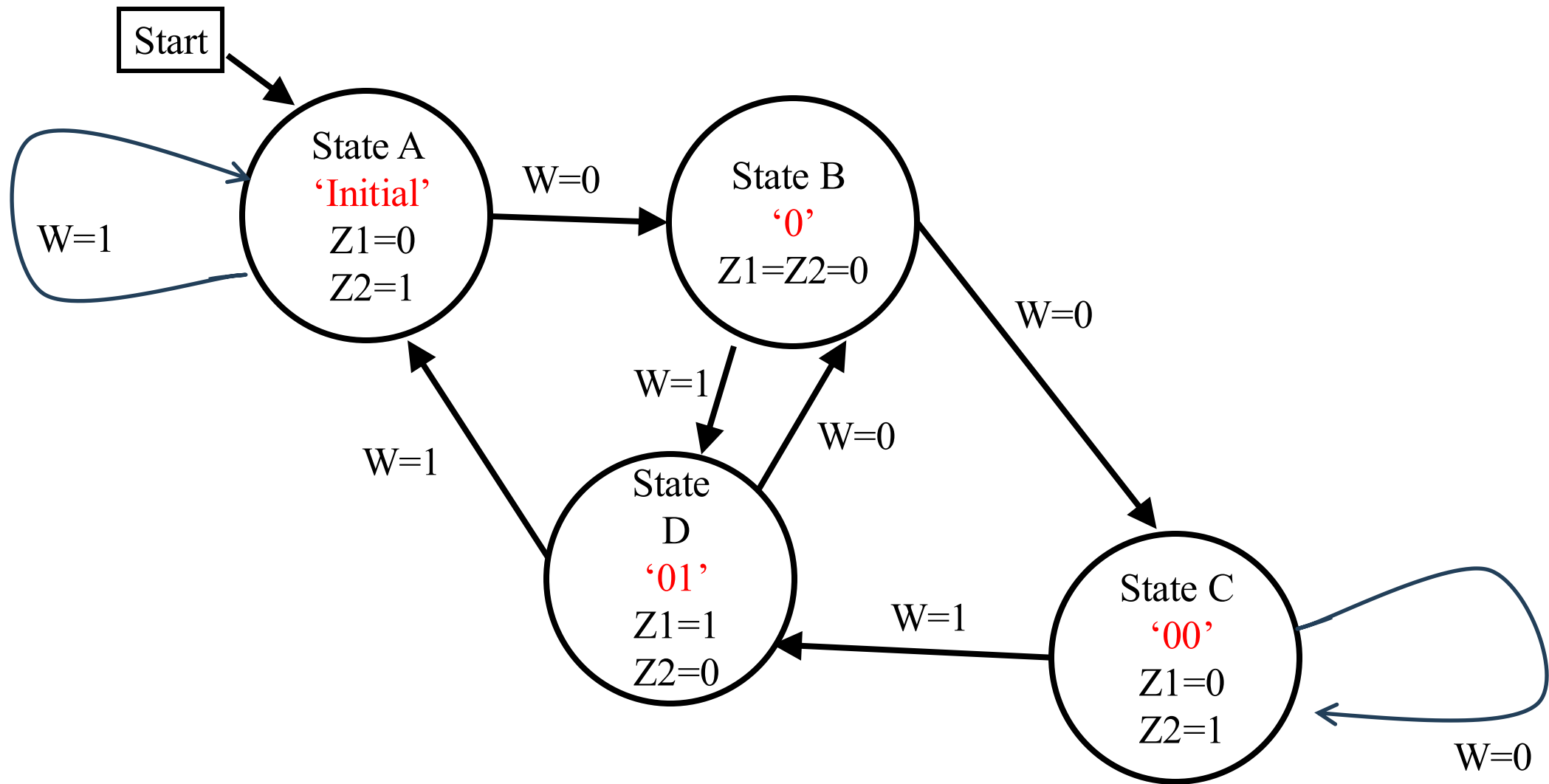
Solution: Moore Type FSM

For,

$w = 01$, $Z1 = 1$ and $Z2 = 0$

For,

$w = 00$, $Z1 = 0$ and $Z2 = 1$



Example: 09: Double output FSM

CLK	1	2	3	4	5	6	7	8	9	10	11	12	13
w	0	1	0	1	0	0	0	0	1	1	0	0	1
Z1	0	0	1	0	1	0	0	0	0	1	0	0	0
Z2	0	0	0	1	0	1	0	0	0	0	0	1	0

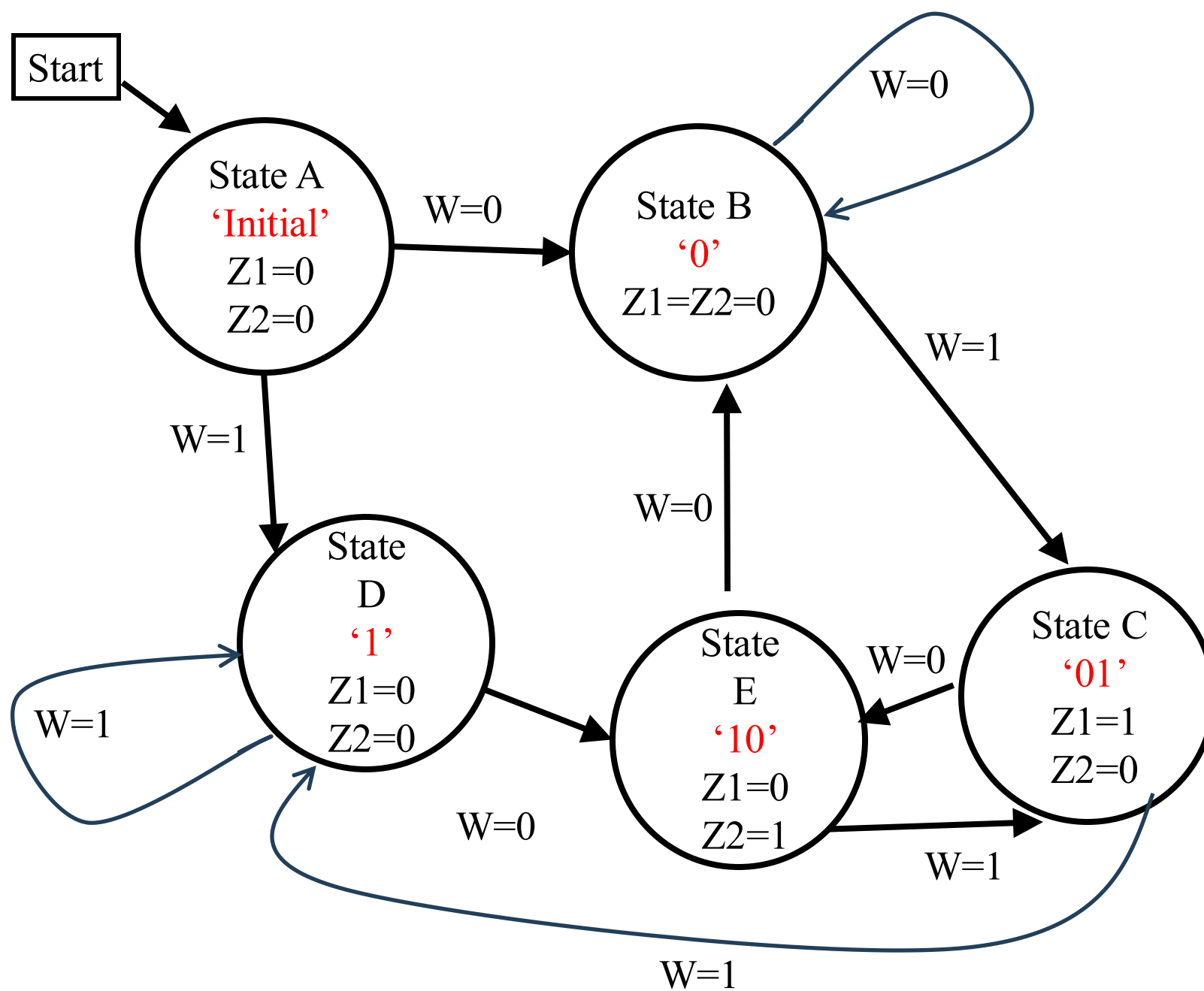
Solution: Moore Type FSM

For,

w = 01, Z1 = 1 and Z2 = 0

For,

w = 10, Z1 = 0 and Z2 = 1



Encoding Schemes (State Assignment)

Consider a state assigned table below:

	Present state	Next state		Output z
		$w=0$	$w=1$	
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

Schemes->

- **Binary encoding**
- Gray encoding
- One-hot encoding

Encoding Schemes (State Assignment)

Consider another state assigned table below:

	Present state	Next state		Output z
		$w=0$	$w=1$	
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

Schemes->

- Binary encoding
- **Gray encoding**
- One-hot encoding

Decimal Number	4 bit Binary Number	4 bit Gray Code
	<u>ABCD</u>	<u>G₁G₂G₃G₄</u>
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1
10	1 0 1 0	1 1 1 1
11	1 0 1 1	1 1 1 0
12	1 1 0 0	1 0 1 0
13	1 1 0 1	1 0 1 1
14	1 1 1 0	1 0 0 1
15	1 1 1 1	1 0 0 0

Encoding Schemes (State Assignment)

Consider another state assigned table below:

	Present state $y_3y_2y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_3Y_2Y_1$	$Y_3Y_2Y_1$	
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

Schemes->

- Binary encoding
- Gray encoding
- **One-hot encoding**

How many flipflops are required? **Ans: 3**